

Task 2 - Gender model - Google Colab

Course: DLBAIPEAI - Project Edge AI

Student: W. Pretorius

Platform: Google Colab

This notebook is the Google Colab version of the **Gender model** for Task 2. It trains MobileNetV3-Small to predict the two appearance labels supplied by FairFace: **Female** and **Male**.

This Colab version keeps the same balanced training batches, class recall tracking, balanced-accuracy checkpoint selection and TensorFlow Lite checks as the desktop version.

Before running the notebook, choose **Runtime -> Change runtime type -> GPU** in Google Colab.

Code cell 2 - output not recorded

```
# cell:folder
import os
import sys
from pathlib import Path
from datetime import datetime

PROJECT_DIR = Path("/content/project_edge_ai")
PROJECT_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

TASK = "gender"
NOTEBOOK_NAME = "02_Gender_Model_Colab"
NOTEBOOK_REVISION = "colab-gender-balanced-v1"
GPU_READY = False

OUTPUT_DIR = (
    PROJECT_DIR
    / "results"
    / TASK
    / datetime.now().strftime("%Y%m%d_%H%M%S")
)

OUTPUT_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

# Confirm that the runtime can save files before long processing starts.
probe = OUTPUT_DIR / ".write_check"
probe.write_text(
    "ok",
    encoding="utf-8",
)
probe.unlink()

os.environ["KERAS_HOME"] = str(
    PROJECT_DIR
    / "datasets"
    / "keras_cache"
)

os.environ["KAGGLEHUB_CACHE"] = str(
    PROJECT_DIR
    / "datasets"
    / "kagglehub_cache"
)

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"
print("Python:", sys.version.split()[0])
```

```
print("Notebook revision:", NOTEBOOK_REVISION)
print("Project folder:", PROJECT_DIR)
print("Results folder:", OUTPUT_DIR)
```

2. Install the extra libraries

In this cell we install only the small helper libraries. TensorFlow and GPU support come from the Colab runtime.

Code cell 4 - output not recorded

```
# cell:install
import subprocess
import sys
from importlib import metadata

# Colab already supplies TensorFlow and GPU support.
# Only the small helper libraries are installed here.
packages = {
    "kagglehub": "0.3.13",
    "tqdm": "4.67.1",
    "reportlab": "4.3.1",
    "nbformat": "5.10.4",
}

missing = []

for package, version in packages.items():
    try:
        installed = metadata.version(
            package
        )
    except metadata.PackageNotFoundError:
        installed = None

    if installed != version:
        missing.append(
            f"{package}=={version}"
        )

if missing:
    subprocess.check_call(
        [
            sys.executable,
            "-m",
            "pip",
            "install",
            "--quiet",
            "--disable-pip-version-check",
            *missing,
        ]
    )

print("Colab helper libraries are ready.")
print("TensorFlow is provided by the Colab runtime.")
```

3. Keep the outputs for the PDF

In this cell we start recording later cell outputs for the final PDF. The code is kept here, not in a separate file. Setup-cell text is included, but its earlier output is not recorded.

Code cell 6 - output not recorded

```
# cell:record
# This small tool stays inside the notebook. No helper file is needed.
import io
import re
import sys
import copy
import json
```

```

if "pdf_recorder" in globals():
    pdf_recorder.close()

class LiveNotebook:
    def __init__(self):
        self.ip = get_ipython()
        self.records = {}
        self.current = None
        self.stdout, self.stderr = sys.stdout, sys.stderr
        self.publish = self.ip.display_pub.publish
        self.format_data = self.ip.displayhook.write_format_data
        self.ip.display_pub.publish = self.capture_display
        self.ip.displayhook.write_format_data = self.capture_result
        self.ip.events.register("pre_run_cell", self.before)

        self.ip.events.register("post_run_cell", self.after)

    def before(self, info):
        self.restore()
        key = info.raw_cell.splitlines()[0].strip()
        self.current = key if key.startswith("# cell:") else None
        if self.current:
            self.records[self.current] = {
                "source": info.raw_cell,
                "execution_count": self.ip.execution_count,
                "outputs": [],
            }
        recorder = self
        class Tee:
            def __init__(self, original, name):
                self.original, self.name = original, name
            def write(self, text):
                outputs = recorder.records[recorder.current]["outputs"]
                if outputs and outputs[-1].get("name") == self.name:
                    outputs[-1]["text"] += text
                else:
                    outputs.append({"output_type": "stream", "name": self.name,
                                   "text": text})
                return self.original.write(text)
            def flush(self):
                return self.original.flush()
            def __getattr__(self, name):
                return getattr(self.original, name)
        sys.stdout, sys.stderr = Tee(self.stdout, "stdout"), Tee(self.stderr, "stderr")

    def capture_display(self, data, metadata=None, **kwargs):
        if self.current:
            self.records[self.current]["outputs"].append({
                "output_type": "display_data", "data": copy.deepcopy(data),
                "metadata": metadata or {},
            })
        return self.publish(data=data, metadata=metadata, **kwargs)

    def capture_result(self, data, metadata=None):
        if self.current:
            self.records[self.current]["outputs"].append({
                "output_type": "execute_result", "data": copy.deepcopy(data),
                "metadata": metadata or {}, "execution_count": self.ip.execution_count,
            })
        return self.format_data(data, metadata)

    def restore(self):
        sys.stdout, sys.stderr = self.stdout, self.stderr

    def after(self, result):
        if self.current:
            error = result.error_before_exec or result.error_in_exec
            if error:
                self.records[self.current]["outputs"].append({
                    "output_type": "stream", "name": "stderr",

```

```

        "text": f"{type(error).__name__}: {error}",
    })
    self.restore()
    self.current = None

    def close(self):
        self.restore()
        self.ip.display_pub.publish = self.publish
        self.ip.displayhook.write_format_data = self.format_data
        for event, function in [("pre_run_cell", self.before), ("post_run_cell", self.after)]:
            if function in self.ip.events.callbacks[event]:
                self.ip.events.unregister(event, function)

pdf_recorder = LiveNotebook()
print("Later cell outputs will be included in the PDF.")

```

4. Check that the Colab GPU can train

In this cell we check that Colab can see a GPU and run a real training calculation. We also enable mixed precision to make training faster.

Code cell 8 - recorded

```

# cell:imports
import time
import hashlib
import zipfile
import shutil
import platform
from concurrent.futures import ThreadPoolExecutor

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image
from tqdm import tqdm
import kagglehub
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    f1_score,
    classification_report,

    confusion_matrix,
    ConfusionMatrixDisplay,
)

from IPython.display import display, Markdown

get_ipython().run_line_magic(
    "matplotlib",
    "inline",
)

pd.set_option(
    "display.max_columns",
    12,
)

pd.set_option(
    "display.width",
    120,
)

print("TensorFlow:", tf.__version__)
print("System:", platform.platform())

```

```

gpus = tf.config.list_physical_devices(
    "GPU"
)

if not gpus:
    raise RuntimeError(
        "No GPU was found. In Colab choose Runtime -> Change runtime type -> GPU, "
        "then run the notebook again."
    )

for gpu in gpus:
    try:
        tf.config.experimental.set_memory_growth(
            gpu,
            True,
        )
    except RuntimeError:
        pass

GPU_NAME = tf.config.experimental.get_device_details(
    gpus[0]
).get(
    "device_name",
    str(gpus[0]),
)

print("GPU found:", GPU_NAME)

# Check a real forward and backward calculation.
previous_placement = tf.config.get_soft_device_placement()
tf.config.set_soft_device_placement(
    False
)

try:
    with tf.device("/GPU:0"):
        probe_images = tf.random.normal(
            (2, 32, 32, 3)
        )

        probe_weights = tf.Variable(
            tf.random.normal(
                (3, 3, 3, 8)
            )
        )

        with tf.GradientTape() as tape:
            probe_output = tf.nn.conv2d(
                probe_images,
                probe_weights,
                strides=1,
                padding="SAME",
            )

            probe_loss = tf.reduce_mean(
                tf.square(
                    probe_output
                )
            )

            probe_gradient = tape.gradient(
                probe_loss,
                probe_weights,
            )

        if probe_gradient is None:
            raise RuntimeError(
                "The GPU did not calculate a gradient."
            )

        if "GPU" not in probe_output.device.upper():

```

```

        raise RuntimeError(
            "The test calculation did not run on the GPU."
        )

    if not np.isfinite(
        probe_gradient.numpy()
    ).all():
        raise RuntimeError(
            "The GPU gradient contains invalid values."
        )

finally:
    tf.config.set_soft_device_placement(
        previous_placement
    )

GPU_READY = True

# MobileNetV3 trains quickly with mixed precision on modern Colab GPUs.
tf.keras.mixed_precision.set_global_policy(
    "mixed_float16"
)

print(
    "Precision policy:",
    tf.keras.mixed_precision.global_policy().name,
)

print(
    "GPU forward and backward test: PASSED"
)

TensorFlow: 2.20.0
System: Linux-6.6.122+-x86_64-with-glibc2.39
GPU found: Tesla T4
Precision policy: mixed_float16
GPU forward and backward test: PASSED

```

5. Verify or download FairFace

In this cell we check the Colab Kaggle cache first. If FairFace is already present in this runtime, we verify and reuse it. Kaggle is only used when the dataset is missing.

Code cell 10 - recorded

```

# cell:dataset
KAGGLE_HANDLE = "mehmoodsheikh/fairface-dataset"
DATASETS_DIR = PROJECT_DIR / "datasets"
DATASETS_DIR.mkdir(parents=True, exist_ok=True)
LOCATION_FILE = DATASETS_DIR / "fairface_local_location.json"
# Only change this to True when you deliberately want to replace a damaged cache.
REPAIR_DATASET = False

def relative_image_name(value):
    value = str(value).strip().replace("\\", "/")
    parts = value.split("/")
    if not value or value.startswith("/") or ":" in value or ".." in parts:
        raise ValueError("Unsafe image path in the labels: " + value)
    return value

def fairface_layout(root):
    """Check labels and all image paths without any internet request."""
    root = Path(root).resolve()
    if not root.is_dir():
        raise FileNotFoundError(str(root))

    csvs, image_roots = [], [root, root / "images"]
    # Do not walk through every image just to find two CSV files.
    for current, dirs, names in os.walk(root):

```

```

here = Path(current)
csvs.extend(here / name for name in names if name.lower().endswith(".csv"))
for directory in dirs:
    if directory.lower() == "train":
        image_roots.append(here)
dirs[:] = [d for d in dirs if d.lower() not in {"train", "val", "images", "__macosx"}]
selected = {}
for split in ("train", "val"):
    exact = [p for p in csvs if p.name.lower() == f"fairface_label_{split}.csv"]
    candidates = exact or [p for p in csvs if split in p.stem.lower()]
    valid = []
    for path in candidates:
        try:
            frame = pd.read_csv(path)
        except (OSError, ValueError):
            continue
        if {"file", "age", "gender"}.issubset(frame.columns) and not frame.empty:
            if frame[["file", "age", "gender"]].isna().any().any():
                raise ValueError(f"Missing labels in {path}")

            names = [relative_image_name(v) for v in frame["file"]]
            if len(set(names)) != len(names):
                raise ValueError(f"Repeated file rows in {path}")
            valid.append((path, frame, names))
    if len(valid) != 1:
        raise ValueError(f"Expected one {split} label CSV in {root}; found {len(valid)}")
    selected[split] = valid[0]
    image_roots.extend([valid[0][0].parent, valid[0][0].parent / "images"])
# Also support a train/val pair under a folder called images.
for base in list(image_roots):
    image_roots.append(base / "images")
image_roots = list(dict.fromkeys(p.resolve() for p in image_roots if p.is_dir()))
matched = []
for image_root in image_roots:
    if all((image_root / selected[s][2][0]).is_file() for s in ("train", "val")):
        matched.append(image_root)
if len(matched) != 1:
    raise ValueError(f"Could not find one matching image root in {root}.")
image_root = matched[0]
missing = []
for split, (_, frame, names) in selected.items():
    for name in names:
        path = image_root / name
        try:
            if not path.is_file() or path.stat().st_size <= 0:
                missing.append(str(path))
        except OSError:
            missing.append(str(path))
    if not missing:
        # This is a small readability check, not a claim that every image is perfect.
        for index in np.linspace(0, len(names)-1, min(4, len(names)), dtype=int):
            with Image.open(image_root / names[index]) as image:
                image.verify()
    if missing:
        raise FileNotFoundError(f"{len(missing)} labelled images are missing/empty. Example:
{missing[0]}")
    return {
        "root": root, "image_root": image_root,
        "train_csv": selected["train"][0], "val_csv": selected["val"][0],
        "train_count": len(selected["train"][1]), "val_count": len(selected["val"][1]),
    }

def get_local_fairface():
    candidates = []
    if LOCATION_FILE.is_file():
        try:
            candidates.append(Path(json.loads(LOCATION_FILE.read_text(encoding="utf-8"))["root"]))
        except (OSError, ValueError, KeyError):
            pass

```

```

caches = [
    Path(os.environ.get("KAGGLEHUB_CACHE", str(DATASETS_DIR / "kagglehub_cache"))),
    DATASETS_DIR / "kagglehub_cache", DATASETS_DIR / "kagglehub" / "_cache",
    DATASETS_DIR / "kagglehub", Path.home() / ".cache" / "kagglehub",
]
for cache in dict.fromkeys(caches):
    versions = cache / "datasets" / "mehmoodsheikh" / "fairface-dataset" / "versions"
    if versions.is_dir():
        candidates.extend(sorted(
            (p for p in versions.iterdir() if p.is_dir()),
            key=lambda p: int(p.name) if p.name.isdigit() else -1, reverse=True))
    # Reuse a previous extracted FairFace copy too. No images are deleted or moved.
    candidates.extend([DATASETS_DIR / "fairface", DATASETS_DIR / "fairface-dataset"])
    issues = []
    for candidate in dict.fromkeys(p.resolve() for p in candidates if p.is_dir()):
        if not any(candidate.iterdir()):
            continue
        try:
            return fairface_layout(candidate), issues
        except (OSError, ValueError) as error:
            issues.append(f"{candidate}: {error}")
    return None, issues

layout, local_issues = get_local_fairface()
if layout is not None and not REPAIR_DATASET:
    print("Verified local FairFace. Kaggle is not called; nothing is downloaded.")
else:
    if local_issues and not REPAIR_DATASET:
        print("\n".join(local_issues))
        raise RuntimeError(
            "Local FairFace files exist but verification failed. No download was started. "
            "Correct the path/files, or set REPAIR_DATASET=True to request a fresh Kaggle copy.")
    import kagglehub
    print("Downloading FairFace from Kaggle..." if not REPAIR_DATASET else "Repair download requested...")
    downloaded = Path(kagglehub.dataset_download(KAGGLE_HANDLE, force_download=REPAIR_DATASET))
    layout = fairface_layout(downloaded)

    print("Downloaded files verified.")

DATA_DIR = layout["root"]
IMAGE_ROOT = layout["image_root"]
TRAIN_CSV, VAL_CSV = layout["train_csv"], layout["val_csv"]
LOCATION_FILE.write_text(json.dumps({k: str(v) if isinstance(v, Path) else v
                                     for k, v in layout.items()}), indent=2, encoding="utf-8")
print("Dataset folder:", DATA_DIR)
print("Image root:", IMAGE_ROOT)
print("Labelled images:", {"train": layout["train_count"], "val": layout["val_count"]})

Downloading FairFace from Kaggle...
Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please
consider upgrading to the latest version (1.0.2).
Downloading from https://www.kaggle.com/api/v1/datasets/download/mehmoodsheikh/fairface-dataset?da
taset_version_number=1...

0%|          | 0.00/550M [00:00<?, ?B/s]
0%|          | 1.00M/550M [00:01<09:18, 1.03MB/s]
0%|          | 2.00M/550M [00:01<04:46, 2.01MB/s]
1%|          | 4.00M/550M [00:01<02:15, 4.22MB/s]
1%|?        | 7.00M/550M [00:01<01:11, 7.93MB/s]
2%|?        | 10.0M/550M [00:01<00:50, 11.2MB/s]
2%|?        | 13.0M/550M [00:01<00:41, 13.7MB/s]
3%|?        | 16.0M/550M [00:01<00:35, 15.9MB/s]
3%|?        | 19.0M/550M [00:02<00:31, 17.5MB/s]
4%|?        | 21.0M/550M [00:02<00:31, 17.8MB/s]
4%|?        | 24.0M/550M [00:02<00:29, 18.6MB/s]
5%|?        | 28.0M/550M [00:02<00:26, 20.5MB/s]
6%|?        | 32.0M/550M [00:02<00:25, 21.6MB/s]
6%|?        | 35.0M/550M [00:02<00:26, 20.3MB/s]
7%|?        | 37.0M/550M [00:02<00:27, 19.7MB/s]
7%|?        | 40.0M/550M [00:03<00:26, 20.0MB/s]

```


8% ?	43.0M/550M	[00:03<00:27, 19.6MB/s]
9% ?	47.0M/550M	[00:03<00:25, 20.5MB/s]
9% ?	51.0M/550M	[00:03<00:24, 21.2MB/s]
10% ?	54.0M/550M	[00:03<00:25, 20.6MB/s]
11% ?	58.0M/550M	[00:04<00:24, 21.2MB/s]
11% ??	62.0M/550M	[00:04<00:24, 21.3MB/s]
12% ??	65.0M/550M	[00:04<00:24, 20.8MB/s]
13% ??	69.0M/550M	[00:04<00:23, 21.5MB/s]
13% ??	73.0M/550M	[00:04<00:22, 21.8MB/s]
14% ??	76.0M/550M	[00:04<00:23, 20.7MB/s]
15% ??	80.0M/550M	[00:05<00:23, 21.4MB/s]
15% ??	84.0M/550M	[00:05<00:22, 21.8MB/s]
16% ??	87.0M/550M	[00:05<00:23, 21.0MB/s]
17% ??	91.0M/550M	[00:05<00:22, 21.6MB/s]
17% ??	95.0M/550M	[00:05<00:21, 21.9MB/s]
18% ??	98.0M/550M	[00:06<00:22, 21.0MB/s]
19% ??	102M/550M	[00:06<00:21, 21.4MB/s]
19% ??	105M/550M	[00:06<00:22, 20.5MB/s]
20% ??	109M/550M	[00:06<00:22, 21.0MB/s]
20% ??	111M/550M	[00:06<00:25, 18.3MB/s]
21% ??	113M/550M	[00:06<00:28, 16.3MB/s]
21% ???	117M/550M	[00:07<00:24, 18.5MB/s]
22% ???	120M/550M	[00:07<00:25, 17.7MB/s]
23% ???	124M/550M	[00:07<00:23, 18.9MB/s]
23% ???	127M/550M	[00:07<00:23, 18.9MB/s]
24% ???	131M/550M	[00:07<00:22, 19.9MB/s]
25% ???	135M/550M	[00:08<00:20, 20.8MB/s]
25% ???	138M/550M	[00:08<00:21, 20.2MB/s]
26% ???	142M/550M	[00:08<00:20, 21.0MB/s]
27% ???	146M/550M	[00:08<00:19, 21.5MB/s]
27% ???	149M/550M	[00:08<00:20, 20.8MB/s]
28% ???	153M/550M	[00:08<00:19, 21.3MB/s]
29% ???	157M/550M	[00:09<00:18, 21.8MB/s]
29% ???	160M/550M	[00:09<00:19, 20.8MB/s]
30% ???	164M/550M	[00:09<00:18, 21.4MB/s]
30% ???	167M/550M	[00:09<00:19, 20.4MB/s]
31% ???	171M/550M	[00:09<00:19, 20.9MB/s]
32% ????	175M/550M	[00:09<00:18, 21.5MB/s]
32% ????	178M/550M	[00:10<00:18, 20.8MB/s]
33% ????	182M/550M	[00:10<00:17, 21.5MB/s]
34% ????	186M/550M	[00:10<00:17, 21.9MB/s]
34% ????	189M/550M	[00:10<00:17, 21.1MB/s]
35% ????	193M/550M	[00:10<00:17, 21.5MB/s]
36% ????	197M/550M	[00:11<00:17, 21.7MB/s]
36% ????	200M/550M	[00:11<00:17, 21.1MB/s]
37% ????	204M/550M	[00:11<00:16, 21.4MB/s]
38% ????	208M/550M	[00:11<00:16, 21.6MB/s]
38% ????	211M/550M	[00:11<00:17, 20.9MB/s]
39% ????	215M/550M	[00:11<00:16, 21.5MB/s]
40% ????	218M/550M	[00:12<00:16, 20.7MB/s]
40% ????	222M/550M	[00:12<00:16, 21.5MB/s]
41% ????	226M/550M	[00:12<00:16, 20.7MB/s]
42% ?????	229M/550M	[00:12<00:22, 14.6MB/s]
42% ?????	232M/550M	[00:13<00:21, 15.6MB/s]
43% ?????	236M/550M	[00:13<00:18, 17.4MB/s]
44% ?????	240M/550M	[00:13<00:17, 18.8MB/s]
44% ?????	243M/550M	[00:13<00:17, 18.8MB/s]
45% ?????	247M/550M	[00:13<00:16, 19.8MB/s]
45% ?????	250M/550M	[00:13<00:16, 19.1MB/s]
46% ?????	254M/550M	[00:14<00:15, 20.5MB/s]
47% ?????	258M/550M	[00:14<00:14, 21.3MB/s]
47% ?????	261M/550M	[00:14<00:14, 20.3MB/s]
48% ?????	265M/550M	[00:14<00:14, 21.0MB/s]
49% ?????	269M/550M	[00:14<00:13, 21.6MB/s]
49% ?????	272M/550M	[00:15<00:14, 20.8MB/s]
50% ?????	276M/550M	[00:15<00:13, 20.8MB/s]
51% ?????	280M/550M	[00:15<00:13, 21.2MB/s]
51% ??????	283M/550M	[00:15<00:13, 21.3MB/s]
52% ??????	287M/550M	[00:15<00:12, 21.5MB/s]

53%	??????		291M/550M	[00:15<00:12, 21.3MB/s]
53%	??????		294M/550M	[00:16<00:12, 21.2MB/s]
54%	??????		298M/550M	[00:16<00:13, 20.2MB/s]
55%	??????		300M/550M	[00:16<00:14, 17.7MB/s]
55%	??????		304M/550M	[00:16<00:13, 19.2MB/s]
56%	??????		308M/550M	[00:16<00:12, 20.1MB/s]
57%	??????		311M/550M	[00:17<00:12, 19.7MB/s]
57%	??????		315M/550M	[00:17<00:11, 20.8MB/s]
58%	??????		318M/550M	[00:17<00:12, 19.8MB/s]
59%	??????		322M/550M	[00:17<00:11, 21.2MB/s]
59%	??????		326M/550M	[00:17<00:11, 20.8MB/s]
60%	??????		329M/550M	[00:17<00:11, 20.9MB/s]
61%	??????		333M/550M	[00:18<00:10, 20.7MB/s]
61%	??????		337M/550M	[00:18<00:10, 21.4MB/s]
62%	??????		340M/550M	[00:18<00:10, 21.4MB/s]
63%	??????		344M/550M	[00:18<00:10, 21.0MB/s]
63%	??????		347M/550M	[00:19<00:14, 14.7MB/s]
64%	??????		350M/550M	[00:19<00:13, 15.6MB/s]
64%	??????		354M/550M	[00:19<00:11, 17.4MB/s]
65%	??????		358M/550M	[00:19<00:11, 18.1MB/s]
66%	??????		361M/550M	[00:19<00:10, 19.0MB/s]
66%	??????		365M/550M	[00:20<00:10, 19.2MB/s]
67%	??????		369M/550M	[00:20<00:09, 20.3MB/s]
68%	??????		372M/550M	[00:20<00:09, 20.7MB/s]
68%	??????		376M/550M	[00:20<00:08, 20.4MB/s]
69%	??????		380M/550M	[00:20<00:08, 21.1MB/s]
70%	??????		383M/550M	[00:20<00:08, 21.2MB/s]
70%	??????		387M/550M	[00:21<00:08, 20.8MB/s]
71%	??????		391M/550M	[00:21<00:07, 21.4MB/s]
72%	??????		394M/550M	[00:21<00:07, 21.3MB/s]
72%	??????		398M/550M	[00:21<00:07, 21.0MB/s]
73%	??????		402M/550M	[00:21<00:07, 21.5MB/s]
74%	??????		405M/550M	[00:22<00:07, 19.8MB/s]
74%	??????		407M/550M	[00:22<00:08, 17.6MB/s]
75%	??????		411M/550M	[00:22<00:07, 19.1MB/s]
75%	??????		415M/550M	[00:22<00:07, 20.1MB/s]
76%	??????		419M/550M	[00:22<00:06, 20.9MB/s]
77%	??????		422M/550M	[00:22<00:06, 19.3MB/s]
77%	??????		425M/550M	[00:23<00:06, 19.0MB/s]
78%	??????		429M/550M	[00:23<00:06, 20.2MB/s]
79%	??????		433M/550M	[00:23<00:05, 20.8MB/s]
79%	??????		436M/550M	[00:23<00:05, 20.3MB/s]
80%	??????		440M/550M	[00:23<00:05, 20.8MB/s]
81%	??????		444M/550M	[00:24<00:05, 21.7MB/s]
81%	??????		447M/550M	[00:24<00:05, 20.8MB/s]
82%	??????		451M/550M	[00:24<00:04, 21.2MB/s]
83%	??????		455M/550M	[00:24<00:04, 21.6MB/s]
83%	??????		458M/550M	[00:24<00:04, 20.9MB/s]
84%	??????		462M/550M	[00:24<00:04, 21.5MB/s]
85%	??????		466M/550M	[00:25<00:04, 21.9MB/s]
85%	??????		469M/550M	[00:25<00:04, 21.0MB/s]
86%	??????		473M/550M	[00:25<00:03, 21.4MB/s]
87%	??????		477M/550M	[00:25<00:03, 21.8MB/s]
87%	??????		480M/550M	[00:25<00:03, 21.0MB/s]
88%	??????		484M/550M	[00:26<00:03, 21.4MB/s]
89%	??????		488M/550M	[00:26<00:02, 21.8MB/s]
89%	??????		491M/550M	[00:26<00:02, 20.9MB/s]
90%	??????		495M/550M	[00:26<00:02, 21.4MB/s]
91%	??????		498M/550M	[00:26<00:02, 20.7MB/s]
91%	??????		502M/550M	[00:26<00:02, 21.3MB/s]
92%	??????		506M/550M	[00:27<00:02, 21.7MB/s]
93%	??????		509M/550M	[00:27<00:02, 20.8MB/s]
93%	??????		513M/550M	[00:27<00:01, 21.3MB/s]
94%	??????		517M/550M	[00:27<00:01, 21.7MB/s]
95%	??????		520M/550M	[00:27<00:01, 20.9MB/s]
95%	??????		523M/550M	[00:27<00:01, 22.1MB/s]
96%	??????		526M/550M	[00:28<00:01, 21.0MB/s]
96%	??????		529M/550M	[00:28<00:00, 21.9MB/s]
97%	??????		532M/550M	[00:28<00:00, 22.1MB/s]

```

97%|??????????| 535M/550M [00:28<00:00, 20.9MB/s]
98%|??????????| 538M/550M [00:28<00:00, 22.3MB/s]
98%|??????????| 541M/550M [00:28<00:00, 21.0MB/s]
99%|??????????| 544M/550M [00:29<00:00, 17.8MB/s]
99%|??????????| 546M/550M [00:29<00:00, 17.5MB/s]
100%|??????????| 548M/550M [00:29<00:00, 17.5MB/s]
100%|??????????| 550M/550M [00:29<00:00, 19.6MB/s]

```

Extracting files...

Downloaded files verified.

Dataset folder: /content/project_edge_ai/datasets/kagglehub_cache/datasets/mehmoodsheikh/fairface-dataset/versions/1

Image root: /content/project_edge_ai/datasets/kagglehub_cache/datasets/mehmoodsheikh/fairface-dataset/versions/1/FairFace

Labelled images: {'train': 86744, 'val': 10954}

6. Read the image labels

In this cell we read the FairFace train and validation labels. The original FairFace validation data stays untouched as the final holdout test.

Code cell 12 - recorded

```

# cell:labels
parts = []

for csv_path, split in [
    (TRAIN_CSV, "development"),
    (VAL_CSV, "holdout"),
]:
    frame = pd.read_csv(csv_path)

    required = {"file", "age", "gender"}

    if not required.issubset(frame.columns):
        raise ValueError(
            f"Unexpected FairFace columns in {csv_path.name}."
        )

    relative_paths = (
        frame["file"]
        .astype(str)
        .str.replace("\\", "/", regex=False)
    )

    resolved_paths = [
        (IMAGE_ROOT / value).resolve()
        for value in relative_paths
    ]

    missing = [
        path for path in resolved_paths
        if not path.is_file()
    ]

    if missing:
        raise FileNotFoundError(
            f"{len(missing)} labelled FairFace images are missing. "
            f"Example: {missing[0]}"
        )

    frame["path"] = [
        str(path)
        for path in resolved_paths
    ]

    frame["source_split"] = split

    frame[
        ["face_x", "face_y", "face_width", "face_height"]
    ] = -1

```

```

parts.append(frame)

raw = pd.concat(
    parts,
    ignore_index=True,
)

print("Actual FairFace labels loaded:", len(raw))

print("\nOriginal source split counts:")
display(
    raw["source_split"]
    .value_counts()
    .rename("images")
    .to_frame()
)

display(raw.head())
Actual FairFace labels loaded: 97698

Original source split counts:

```

source_split	images
development	86744
holdout	10954

```


```

	file	age	gender	race	service_test	path \
0	train/1.jpg	50-59	Male	East Asian	True	/content/project_edge_ai/datasets/kagglehub_ca...
1	train/2.jpg	30-39	Female	Indian	False	/content/project_edge_ai/datasets/kagglehub_ca...
2	train/3.jpg	3-9	Female	Black	False	/content/project_edge_ai/datasets/kagglehub_ca...
3	train/4.jpg	20-29	Female	Indian	True	/content/project_edge_ai/datasets/kagglehub_ca...
4	train/5.jpg	20-29	Female	Indian	True	/content/project_edge_ai/datasets/kagglehub_ca...

```


```

	source_split	face_x	face_y	face_width	face_height
0	development	-1	-1	-1	-1
1	development	-1	-1	-1	-1
2	development	-1	-1	-1	-1
3	development	-1	-1	-1	-1
4	development	-1	-1	-1	-1

7. Choose the training settings

In this cell we choose the model settings and create the two gender classes used by FairFace. The label order is Female first and Male second.

Code cell 14 - recorded

```

# cell:settings
SEED = 42
IMAGE_SIZE = 224
BATCH_SIZE = 32
HEAD_EPOCHS = 8
FINE_TUNE_EPOCHS = 20

CLASS_NAMES = ["Female", "Male"]
N_CLASSES = len(CLASS_NAMES)
TARGET_COLUMN = "gender"

# Set this to True only for a quick pipeline test.
# Keep it False for the final model and report.
FAST_MODE = False

if FAST_MODE:

```

```

HEAD_EPOCHS = 2
FINE_TUNE_EPOCHS = 3
print(
    "FAST_MODE is ON. This is a quick pipeline test, not the final experiment."
)
else:
    print(
        "FAST_MODE is OFF. Full training settings are active."
    )

tf.keras.utils.set_random_seed(SEED)

label_map = {
    "Female": 0,
    "Male": 1,
}

raw["label"] = raw[TARGET_COLUMN].map(label_map)

if raw["label"].isna().any():
    bad_values = sorted(raw.loc[raw["label"].isna(), TARGET_COLUMN].astype(str).unique())
    raise ValueError("Unexpected FairFace gender labels: " + ", ".join(bad_values))

raw["label"] = raw["label"].astype(int)

if set(raw["label"].unique()) != {0, 1}:
    raise ValueError("Both Female and Male classes must be present.")

print("Class order:", dict(enumerate(CLASS_NAMES)))
print("Images kept:", len(raw))

print("\nFairFace gender counts:")
display(
    raw["label"]
    .map(dict(enumerate(CLASS_NAMES)))
    .value_counts()
    .rename("images")
    .to_frame()
)

# Set these checks before training, not after looking at holdout results.
MOBILE_IMAGES_PER_CLASS = 256

FP32_LIMITS = {
    "max_abs": 0.01,
    "mean_abs": 0.001,
    "min_agreement": 0.99,
    "max_balanced_accuracy_drop": 0.01,
    "max_class_recall_drop": 0.02,
}

INT8_LIMITS = {
    "max_abs": 0.15,
    "mean_abs": 0.02,
    "min_agreement": 0.97,
    "max_balanced_accuracy_drop": 0.02,
    "max_class_recall_drop": 0.03,
}

if BATCH_SIZE < 2 or BATCH_SIZE % 2:
    raise ValueError("Use an even batch size so each class gets half the batch.")

FAST_MODE is OFF. Full training settings are active.
Class order: {0: 'Female', 1: 'Male'}
Images kept: 97698

FairFace gender counts:

      images
label
Male      51778
Female    45920

```

8. Check the images

In this cell we open every labelled face image and check that it can be read. We also create a fingerprint so exact repeated images can be found.

Code cell 16 - recorded

```
# cell:audit
BOX_COLUMNS = ["face_x", "face_y", "face_width", "face_height"]

def open_face(path, box):
    with Image.open(path) as original:
        original.load()
        image = original.convert("RGB")
    x, y, width, height = map(float, box)
    if not (x == y == width == height == -1):
        if not np.isfinite([x, y, width, height]).all() or width <= 0 or height <= 0:
            raise ValueError("Invalid face box.")
        bounds = (max(0, int(x)), max(0, int(y)),
                  min(image.width, int(x + width)), min(image.height, int(y + height)))
        if bounds[2] <= bounds[0] or bounds[3] <= bounds[1]:
            raise ValueError("Face box is outside the image.")
        image = image.crop(bounds)
    if min(image.size) < 16:
        raise ValueError("Face crop is smaller than 16 pixels.")
    return image

def check_image(row):
    try:
        image = open_face(row["path"], [row[column] for column in BOX_COLUMNS])
        digest = hashlib.sha256(str(image.size).encode() + image.tobytes()).hexdigest()
        return {"sha256": digest, "image_error": ""}
    except Exception as error:
        return {"sha256": "", "image_error": str(error)}

with ThreadPoolExecutor(max_workers=min(8, os.cpu_count() or 1)) as pool:
    checks = list(tqdm(pool.map(check_image, raw.to_dict("records")),
                       total=len(raw), desc="Checking face images"))
checked = pd.concat([raw.reset_index(drop=True), pd.DataFrame(checks)], axis=1)

# The image check can take several minutes. Make sure the results folder
# still exists before we save the file.
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

image_check_file = OUTPUT_DIR / "image_checks.csv"
checked.to_csv(image_check_file, index=False)

print("Image check saved to:", image_check_file)
print("Unreadable or invalid images:", int(checked["image_error"].ne("").sum()))
clean_data = checked.loc[checked["image_error"].eq("")]
if clean_data.empty:
    raise ValueError("No usable face images were found.")

Checking face images: 0%|          | 0/97698 [00:00<?, ?it/s]
Checking face images: 0%|          | 314/97698 [00:00<00:31, 3120.11it/s]
Checking face images: 1%|          | 627/97698 [00:00<00:59, 1618.94it/s]
Checking face images: 1%|          | 827/97698 [00:00<01:07, 1444.39it/s]
Checking face images: 1%|          | 991/97698 [00:00<01:12, 1342.47it/s]
Checking face images: 1%|          | 1136/97698 [00:00<01:12, 1328.65it/s]
Checking face images: 1%|?        | 1275/97698 [00:00<01:13, 1312.16it/s]
Checking face images: 1%|?        | 1411/97698 [00:00<01:12, 1323.64it/s]
Checking face images: 2%|?        | 1546/97698 [00:01<01:12, 1327.58it/s]
Checking face images: 2%|?        | 1681/97698 [00:01<01:12, 1330.71it/s]
Checking face images: 2%|?        | 1816/97698 [00:01<01:11, 1333.90it/s]
Checking face images: 2%|?        | 1954/97698 [00:01<01:11, 1342.92it/s]
Checking face images: 2%|?        | 2089/97698 [00:01<01:12, 1320.33it/s]
Checking face images: 2%|?        | 2222/97698 [00:01<01:14, 1289.50it/s]
Checking face images: 2%|?        | 2352/97698 [00:01<01:15, 1267.84it/s]
Checking face images: 3%|?        | 2480/97698 [00:01<01:16, 1243.59it/s]
```

Checking face images:	3% ?		2605/97698	[00:01<01:18,	1204.08it/s]
Checking face images:	3% ?		2735/97698	[00:02<01:17,	1229.67it/s]
Checking face images:	3% ?		2868/97698	[00:02<01:15,	1249.03it/s]
Checking face images:	3% ?		3000/97698	[00:02<01:14,	1266.50it/s]
Checking face images:	3% ?		3128/97698	[00:02<01:14,	1270.29it/s]
Checking face images:	3% ?		3256/97698	[00:02<01:14,	1263.37it/s]
Checking face images:	3% ?		3397/97698	[00:02<01:12,	1304.36it/s]
Checking face images:	4% ?		3528/97698	[00:02<01:13,	1288.41it/s]
Checking face images:	4% ?		3657/97698	[00:02<01:14,	1262.57it/s]
Checking face images:	4% ?		3784/97698	[00:02<01:14,	1262.69it/s]
Checking face images:	4% ?		3911/97698	[00:02<01:19,	1180.59it/s]
Checking face images:	4% ?		4034/97698	[00:03<01:18,	1193.07it/s]
Checking face images:	4% ?		4156/97698	[00:03<01:18,	1197.19it/s]
Checking face images:	4% ?		4282/97698	[00:03<01:17,	1210.53it/s]
Checking face images:	5% ?		4409/97698	[00:03<01:16,	1224.09it/s]
Checking face images:	5% ?		4543/97698	[00:03<01:15,	1240.35it/s]
Checking face images:	5% ?		4681/97698	[00:03<01:13,	1262.61it/s]
Checking face images:	5% ?		4820/97698	[00:03<01:11,	1299.01it/s]
Checking face images:	5% ?		4951/97698	[00:03<01:12,	1286.75it/s]
Checking face images:	5% ?		5080/97698	[00:03<01:12,	1280.92it/s]
Checking face images:	5% ?		5209/97698	[00:04<01:19,	1170.66it/s]
Checking face images:	5% ?		5359/97698	[00:04<01:13,	1255.01it/s]
Checking face images:	6% ?		5487/97698	[00:04<01:13,	1261.82it/s]
Checking face images:	6% ?		5623/97698	[00:04<01:11,	1286.35it/s]
Checking face images:	6% ?		5758/97698	[00:04<01:10,	1303.26it/s]
Checking face images:	6% ?		5893/97698	[00:04<01:09,	1315.85it/s]
Checking face images:	6% ?		6026/97698	[00:04<01:09,	1310.88it/s]
Checking face images:	6% ?		6158/97698	[00:04<01:10,	1302.14it/s]
Checking face images:	6% ?		6295/97698	[00:04<01:09,	1317.98it/s]
Checking face images:	7% ?		6428/97698	[00:04<01:09,	1315.76it/s]
Checking face images:	7% ?		6560/97698	[00:05<01:14,	1215.51it/s]
Checking face images:	7% ?		6708/97698	[00:05<01:10,	1287.03it/s]
Checking face images:	7% ?		6839/97698	[00:05<01:10,	1288.07it/s]
Checking face images:	7% ?		6972/97698	[00:05<01:09,	1299.70it/s]
Checking face images:	7% ?		7103/97698	[00:05<01:10,	1292.11it/s]
Checking face images:	7% ?		7233/97698	[00:05<01:10,	1279.08it/s]
Checking face images:	8% ?		7362/97698	[00:05<01:10,	1275.05it/s]
Checking face images:	8% ?		7491/97698	[00:05<01:10,	1274.09it/s]
Checking face images:	8% ?		7622/97698	[00:05<01:10,	1280.42it/s]
Checking face images:	8% ?		7756/97698	[00:05<01:09,	1291.54it/s]
Checking face images:	8% ?		7886/97698	[00:06<01:12,	1239.79it/s]
Checking face images:	8% ?		8014/97698	[00:06<01:11,	1250.94it/s]
Checking face images:	8% ?		8147/97698	[00:06<01:10,	1273.05it/s]
Checking face images:	8% ?		8278/97698	[00:06<01:09,	1278.84it/s]
Checking face images:	9% ?		8416/97698	[00:06<01:08,	1306.68it/s]
Checking face images:	9% ?		8549/97698	[00:06<01:08,	1304.71it/s]
Checking face images:	9% ?		8680/97698	[00:06<01:09,	1276.25it/s]
Checking face images:	9% ?		8808/97698	[00:06<01:10,	1258.08it/s]
Checking face images:	9% ?		8934/97698	[00:06<01:10,	1258.12it/s]
Checking face images:	9% ?		9064/97698	[00:07<01:09,	1267.31it/s]
Checking face images:	9% ?		9191/97698	[00:07<01:12,	1226.59it/s]
Checking face images:	10% ?		9314/97698	[00:07<01:13,	1207.05it/s]
Checking face images:	10% ?		9454/97698	[00:07<01:10,	1253.33it/s]
Checking face images:	10% ?		9591/97698	[00:07<01:08,	1286.89it/s]
Checking face images:	10% ?		9726/97698	[00:07<01:07,	1303.61it/s]
Checking face images:	10% ?		9857/97698	[00:07<01:07,	1301.38it/s]
Checking face images:	10% ?		9988/97698	[00:07<01:08,	1287.57it/s]
Checking face images:	10% ?		10117/97698	[00:07<01:09,	1262.46it/s]
Checking face images:	10% ?		10248/97698	[00:07<01:08,	1276.13it/s]
Checking face images:	11% ?		10381/97698	[00:08<01:08,	1281.41it/s]
Checking face images:	11% ?		10510/97698	[00:08<01:15,	1150.80it/s]
Checking face images:	11% ?		10628/97698	[00:08<01:28,	986.91it/s]
Checking face images:	11% ?		10732/97698	[00:08<01:36,	902.74it/s]
Checking face images:	11% ?		10827/97698	[00:08<01:43,	841.52it/s]
Checking face images:	11% ?		10915/97698	[00:08<01:49,	795.01it/s]
Checking face images:	11% ??		10997/97698	[00:08<01:52,	768.69it/s]
Checking face images:	11% ??		11076/97698	[00:08<01:54,	755.17it/s]
Checking face images:	11% ??		11153/97698	[00:09<01:55,	749.73it/s]
Checking face images:	11% ??		11229/97698	[00:09<02:02,	708.03it/s]

Checking face images: 12% ??	11301/97698	[00:09<02:05, 686.74it/s]
Checking face images: 12% ??	11374/97698	[00:09<02:04, 694.73it/s]
Checking face images: 12% ??	11444/97698	[00:09<02:04, 694.03it/s]
Checking face images: 12% ??	11514/97698	[00:09<02:05, 688.51it/s]
Checking face images: 12% ??	11583/97698	[00:09<02:06, 681.18it/s]
Checking face images: 12% ??	11654/97698	[00:09<02:05, 684.35it/s]
Checking face images: 12% ??	11731/97698	[00:09<02:01, 705.17it/s]
Checking face images: 12% ??	11802/97698	[00:10<02:03, 694.71it/s]
Checking face images: 12% ??	11873/97698	[00:10<02:03, 696.73it/s]
Checking face images: 12% ??	11943/97698	[00:10<02:08, 665.60it/s]
Checking face images: 12% ??	12016/97698	[00:10<02:05, 680.86it/s]
Checking face images: 12% ??	12096/97698	[00:10<01:59, 714.63it/s]
Checking face images: 12% ??	12168/97698	[00:10<01:59, 712.81it/s]
Checking face images: 13% ??	12240/97698	[00:10<02:02, 695.75it/s]
Checking face images: 13% ??	12310/97698	[00:10<02:02, 695.57it/s]
Checking face images: 13% ??	12438/97698	[00:10<01:38, 866.14it/s]
Checking face images: 13% ??	12566/97698	[00:10<01:26, 985.64it/s]
Checking face images: 13% ??	12693/97698	[00:11<01:19, 1068.77it/s]
Checking face images: 13% ??	12819/97698	[00:11<01:15, 1119.08it/s]
Checking face images: 13% ??	12955/97698	[00:11<01:11, 1185.47it/s]
Checking face images: 13% ??	13074/97698	[00:11<01:13, 1152.51it/s]
Checking face images: 14% ??	13215/97698	[00:11<01:09, 1222.85it/s]
Checking face images: 14% ??	13356/97698	[00:11<01:06, 1276.58it/s]
Checking face images: 14% ??	13485/97698	[00:11<01:05, 1277.04it/s]
Checking face images: 14% ??	13615/97698	[00:11<01:05, 1279.87it/s]
Checking face images: 14% ??	13744/97698	[00:11<01:06, 1264.69it/s]
Checking face images: 14% ??	13871/97698	[00:12<01:07, 1233.73it/s]
Checking face images: 14% ??	13995/97698	[00:12<01:08, 1218.44it/s]
Checking face images: 14% ??	14118/97698	[00:12<01:10, 1191.59it/s]
Checking face images: 15% ??	14249/97698	[00:12<01:08, 1221.78it/s]
Checking face images: 15% ??	14372/97698	[00:12<01:09, 1203.48it/s]
Checking face images: 15% ??	14505/97698	[00:12<01:07, 1236.70it/s]
Checking face images: 15% ??	14637/97698	[00:12<01:06, 1253.36it/s]
Checking face images: 15% ??	14763/97698	[00:12<01:07, 1237.39it/s]
Checking face images: 15% ??	14887/97698	[00:12<01:08, 1210.15it/s]
Checking face images: 15% ??	15026/97698	[00:12<01:05, 1258.39it/s]
Checking face images: 16% ??	15159/97698	[00:13<01:04, 1275.71it/s]
Checking face images: 16% ??	15298/97698	[00:13<01:03, 1306.61it/s]
Checking face images: 16% ??	15431/97698	[00:13<01:02, 1312.31it/s]
Checking face images: 16% ??	15563/97698	[00:13<01:02, 1313.15it/s]
Checking face images: 16% ??	15695/97698	[00:13<01:04, 1265.39it/s]
Checking face images: 16% ??	15826/97698	[00:13<01:04, 1276.79it/s]
Checking face images: 16% ??	15955/97698	[00:13<01:03, 1280.60it/s]
Checking face images: 16% ??	16093/97698	[00:13<01:02, 1306.97it/s]
Checking face images: 17% ??	16226/97698	[00:13<01:02, 1311.08it/s]
Checking face images: 17% ??	16360/97698	[00:13<01:02, 1309.43it/s]
Checking face images: 17% ??	16499/97698	[00:14<01:04, 1251.41it/s]
Checking face images: 17% ??	16667/97698	[00:14<00:59, 1372.83it/s]
Checking face images: 17% ??	16806/97698	[00:14<01:00, 1345.84it/s]
Checking face images: 17% ??	16942/97698	[00:14<01:00, 1344.27it/s]
Checking face images: 17% ??	17078/97698	[00:14<01:02, 1285.54it/s]
Checking face images: 18% ??	17214/97698	[00:14<01:01, 1304.64it/s]
Checking face images: 18% ??	17346/97698	[00:14<01:01, 1303.33it/s]
Checking face images: 18% ??	17486/97698	[00:14<01:00, 1330.67it/s]
Checking face images: 18% ??	17622/97698	[00:14<00:59, 1339.07it/s]
Checking face images: 18% ??	17757/97698	[00:15<01:01, 1296.87it/s]
Checking face images: 18% ??	17888/97698	[00:15<01:01, 1290.10it/s]
Checking face images: 18% ??	18031/97698	[00:15<01:00, 1317.63it/s]
Checking face images: 19% ??	18164/97698	[00:15<01:05, 1208.90it/s]
Checking face images: 19% ??	18287/97698	[00:15<01:09, 1143.50it/s]
Checking face images: 19% ??	18427/97698	[00:15<01:05, 1211.10it/s]
Checking face images: 19% ??	18556/97698	[00:15<01:06, 1188.19it/s]
Checking face images: 19% ??	18705/97698	[00:15<01:02, 1271.05it/s]
Checking face images: 19% ??	18834/97698	[00:15<01:06, 1192.35it/s]
Checking face images: 19% ??	18985/97698	[00:16<01:01, 1273.56it/s]
Checking face images: 20% ??	19115/97698	[00:16<01:02, 1265.76it/s]
Checking face images: 20% ??	19243/97698	[00:16<01:02, 1247.02it/s]
Checking face images: 20% ??	19369/97698	[00:16<01:03, 1231.10it/s]
Checking face images: 20% ??	19493/97698	[00:16<01:03, 1231.82it/s]

Checking face images:	20% ??	19617/97698	[00:16<01:06,	1181.62it/s]
Checking face images:	20% ??	19739/97698	[00:16<01:05,	1189.84it/s]
Checking face images:	20% ??	19865/97698	[00:16<01:04,	1210.02it/s]
Checking face images:	20% ??	19992/97698	[00:16<01:03,	1223.74it/s]
Checking face images:	21% ??	20115/97698	[00:16<01:03,	1225.19it/s]
Checking face images:	21% ??	20238/97698	[00:17<01:04,	1208.60it/s]
Checking face images:	21% ??	20360/97698	[00:17<01:04,	1192.09it/s]
Checking face images:	21% ??	20480/97698	[00:17<01:04,	1190.69it/s]
Checking face images:	21% ??	20605/97698	[00:17<01:03,	1207.10it/s]
Checking face images:	21% ??	20738/97698	[00:17<01:02,	1233.36it/s]
Checking face images:	21% ???	20862/97698	[00:17<01:05,	1181.16it/s]
Checking face images:	21% ???	20985/97698	[00:17<01:05,	1168.57it/s]
Checking face images:	22% ???	21117/97698	[00:17<01:03,	1208.75it/s]
Checking face images:	22% ???	21242/97698	[00:17<01:02,	1214.38it/s]
Checking face images:	22% ???	21366/97698	[00:18<01:02,	1220.80it/s]
Checking face images:	22% ???	21489/97698	[00:18<01:02,	1222.00it/s]
Checking face images:	22% ???	21616/97698	[00:18<01:01,	1235.78it/s]
Checking face images:	22% ???	21740/97698	[00:18<01:02,	1214.67it/s]
Checking face images:	22% ???	21870/97698	[00:18<01:01,	1237.43it/s]
Checking face images:	23% ???	21996/97698	[00:18<01:00,	1243.34it/s]
Checking face images:	23% ???	22121/97698	[00:18<01:04,	1173.39it/s]
Checking face images:	23% ???	22242/97698	[00:18<01:03,	1183.30it/s]
Checking face images:	23% ???	22366/97698	[00:18<01:02,	1196.56it/s]
Checking face images:	23% ???	22495/97698	[00:18<01:01,	1221.90it/s]
Checking face images:	23% ???	22623/97698	[00:19<01:00,	1237.51it/s]
Checking face images:	23% ???	22748/97698	[00:19<01:00,	1235.00it/s]
Checking face images:	23% ???	22872/97698	[00:19<01:01,	1223.55it/s]
Checking face images:	24% ???	22995/97698	[00:19<01:01,	1204.97it/s]
Checking face images:	24% ???	23130/97698	[00:19<01:00,	1240.29it/s]
Checking face images:	24% ???	23271/97698	[00:19<00:57,	1289.50it/s]
Checking face images:	24% ???	23401/97698	[00:19<01:00,	1238.17it/s]
Checking face images:	24% ???	23532/97698	[00:19<00:58,	1258.43it/s]
Checking face images:	24% ???	23667/97698	[00:19<00:57,	1285.10it/s]
Checking face images:	24% ???	23801/97698	[00:19<00:56,	1299.54it/s]
Checking face images:	24% ???	23932/97698	[00:20<00:57,	1291.61it/s]
Checking face images:	25% ???	24065/97698	[00:20<00:56,	1299.53it/s]
Checking face images:	25% ???	24199/97698	[00:20<00:56,	1310.88it/s]
Checking face images:	25% ???	24331/97698	[00:20<00:56,	1308.67it/s]
Checking face images:	25% ???	24464/97698	[00:20<00:55,	1314.62it/s]
Checking face images:	25% ???	24596/97698	[00:20<00:56,	1301.43it/s]
Checking face images:	25% ???	24727/97698	[00:20<00:58,	1247.09it/s]
Checking face images:	25% ???	24853/97698	[00:20<01:00,	1204.17it/s]
Checking face images:	26% ???	24974/97698	[00:20<01:12,	997.73it/s]
Checking face images:	26% ???	25080/97698	[00:21<01:20,	902.24it/s]
Checking face images:	26% ???	25176/97698	[00:21<01:25,	843.30it/s]
Checking face images:	26% ???	25264/97698	[00:21<01:29,	809.06it/s]
Checking face images:	26% ???	25348/97698	[00:21<01:32,	778.58it/s]
Checking face images:	26% ???	25428/97698	[00:21<01:34,	763.96it/s]
Checking face images:	26% ???	25506/97698	[00:21<01:39,	727.27it/s]
Checking face images:	26% ???	25580/97698	[00:21<01:39,	723.81it/s]
Checking face images:	26% ???	25653/97698	[00:21<01:40,	719.18it/s]
Checking face images:	26% ???	25726/97698	[00:22<01:41,	708.64it/s]
Checking face images:	26% ???	25797/97698	[00:22<01:41,	707.38it/s]
Checking face images:	26% ???	25868/97698	[00:22<01:42,	703.60it/s]
Checking face images:	27% ???	25939/97698	[00:22<01:45,	683.22it/s]
Checking face images:	27% ???	26011/97698	[00:22<01:44,	683.65it/s]
Checking face images:	27% ???	26081/97698	[00:22<01:44,	687.80it/s]
Checking face images:	27% ???	26150/97698	[00:22<01:44,	685.97it/s]
Checking face images:	27% ???	26219/97698	[00:22<01:45,	677.76it/s]
Checking face images:	27% ???	26287/97698	[00:22<01:46,	669.84it/s]
Checking face images:	27% ???	26355/97698	[00:23<01:49,	651.65it/s]
Checking face images:	27% ???	26425/97698	[00:23<01:47,	665.51it/s]
Checking face images:	27% ???	26492/97698	[00:23<01:46,	665.71it/s]
Checking face images:	27% ???	26564/97698	[00:23<01:44,	681.43it/s]
Checking face images:	27% ???	26633/97698	[00:23<01:46,	669.88it/s]
Checking face images:	27% ???	26708/97698	[00:23<01:42,	691.86it/s]
Checking face images:	27% ???	26827/97698	[00:23<01:24,	835.19it/s]
Checking face images:	28% ???	26949/97698	[00:23<01:14,	947.69it/s]
Checking face images:	28% ???	27066/97698	[00:23<01:09,	1011.99it/s]

Checking face images:	28% ???	27168/97698	[00:23<01:11, 983.72it/s]
Checking face images:	28% ???	27281/97698	[00:24<01:09, 1007.67it/s]
Checking face images:	28% ???	27407/97698	[00:24<01:05, 1079.18it/s]
Checking face images:	28% ???	27524/97698	[00:24<01:03, 1102.28it/s]
Checking face images:	28% ???	27636/97698	[00:24<01:03, 1105.16it/s]
Checking face images:	28% ???	27750/97698	[00:24<01:02, 1114.69it/s]
Checking face images:	29% ???	27874/97698	[00:24<01:00, 1148.95it/s]
Checking face images:	29% ???	27992/97698	[00:24<01:00, 1155.22it/s]
Checking face images:	29% ???	28118/97698	[00:24<00:58, 1180.74it/s]
Checking face images:	29% ???	28239/97698	[00:24<00:58, 1186.09it/s]
Checking face images:	29% ???	28358/97698	[00:24<01:01, 1125.91it/s]
Checking face images:	29% ???	28472/97698	[00:25<01:05, 1049.23it/s]
Checking face images:	29% ???	28579/97698	[00:25<01:09, 1000.08it/s]
Checking face images:	29% ???	28701/97698	[00:25<01:05, 1057.58it/s]
Checking face images:	29% ???	28809/97698	[00:25<01:04, 1060.28it/s]
Checking face images:	30% ???	28950/97698	[00:25<00:59, 1158.06it/s]
Checking face images:	30% ???	29067/97698	[00:25<00:59, 1155.49it/s]
Checking face images:	30% ???	29208/97698	[00:25<00:55, 1227.06it/s]
Checking face images:	30% ???	29332/97698	[00:25<00:56, 1211.02it/s]
Checking face images:	30% ???	29454/97698	[00:25<00:57, 1197.26it/s]
Checking face images:	30% ???	29575/97698	[00:26<00:58, 1168.17it/s]
Checking face images:	30% ???	29706/97698	[00:26<00:56, 1202.74it/s]
Checking face images:	31% ???	29827/97698	[00:26<00:56, 1198.52it/s]
Checking face images:	31% ???	29948/97698	[00:26<00:56, 1200.40it/s]
Checking face images:	31% ???	30069/97698	[00:26<00:59, 1135.80it/s]
Checking face images:	31% ???	30217/97698	[00:26<00:55, 1221.69it/s]
Checking face images:	31% ???	30346/97698	[00:26<00:54, 1236.36it/s]
Checking face images:	31% ???	30471/97698	[00:26<00:54, 1238.97it/s]
Checking face images:	31% ????	30596/97698	[00:26<00:55, 1207.55it/s]
Checking face images:	31% ????	30718/97698	[00:26<00:58, 1145.32it/s]
Checking face images:	32% ????	30842/97698	[00:27<00:57, 1170.30it/s]
Checking face images:	32% ????	30970/97698	[00:27<00:55, 1199.47it/s]
Checking face images:	32% ????	31095/97698	[00:27<00:55, 1210.69it/s]
Checking face images:	32% ????	31217/97698	[00:27<00:55, 1207.85it/s]
Checking face images:	32% ????	31348/97698	[00:27<00:53, 1236.62it/s]
Checking face images:	32% ????	31475/97698	[00:27<00:53, 1243.50it/s]
Checking face images:	32% ????	31600/97698	[00:27<00:54, 1222.16it/s]
Checking face images:	32% ????	31723/97698	[00:27<00:54, 1218.08it/s]
Checking face images:	33% ????	31845/97698	[00:27<00:55, 1196.37it/s]
Checking face images:	33% ????	31965/97698	[00:28<00:57, 1143.36it/s]
Checking face images:	33% ????	32089/97698	[00:28<00:56, 1164.52it/s]
Checking face images:	33% ????	32213/97698	[00:28<00:55, 1181.26it/s]
Checking face images:	33% ????	32335/97698	[00:28<00:54, 1192.26it/s]
Checking face images:	33% ????	32455/97698	[00:28<00:54, 1189.02it/s]
Checking face images:	33% ????	32593/97698	[00:28<00:52, 1240.48it/s]
Checking face images:	33% ????	32724/97698	[00:28<00:51, 1260.98it/s]
Checking face images:	34% ????	32851/97698	[00:28<00:51, 1256.90it/s]
Checking face images:	34% ????	32979/97698	[00:28<00:54, 1188.67it/s]
Checking face images:	34% ????	33130/97698	[00:28<00:50, 1277.24it/s]
Checking face images:	34% ????	33259/97698	[00:29<00:52, 1220.31it/s]
Checking face images:	34% ????	33385/97698	[00:29<00:52, 1230.60it/s]
Checking face images:	34% ????	33513/97698	[00:29<00:51, 1244.39it/s]
Checking face images:	34% ????	33639/97698	[00:29<00:51, 1243.93it/s]
Checking face images:	35% ????	33771/97698	[00:29<00:50, 1258.92it/s]
Checking face images:	35% ????	33906/97698	[00:29<00:49, 1282.15it/s]
Checking face images:	35% ????	34039/97698	[00:29<00:49, 1294.20it/s]
Checking face images:	35% ????	34169/97698	[00:29<00:49, 1281.73it/s]
Checking face images:	35% ????	34304/97698	[00:29<00:48, 1300.26it/s]
Checking face images:	35% ????	34435/97698	[00:29<00:49, 1282.28it/s]
Checking face images:	35% ????	34564/97698	[00:30<00:52, 1203.05it/s]
Checking face images:	36% ????	34691/97698	[00:30<00:52, 1208.19it/s]
Checking face images:	36% ????	34833/97698	[00:30<00:49, 1261.66it/s]
Checking face images:	36% ????	34961/97698	[00:30<00:49, 1266.89it/s]
Checking face images:	36% ????	35089/97698	[00:30<00:49, 1269.51it/s]
Checking face images:	36% ????	35217/97698	[00:30<00:51, 1219.98it/s]
Checking face images:	36% ????	35343/97698	[00:30<00:50, 1231.21it/s]
Checking face images:	36% ????	35470/97698	[00:30<00:50, 1240.25it/s]
Checking face images:	36% ????	35595/97698	[00:30<00:50, 1238.00it/s]
Checking face images:	37% ????	35721/97698	[00:31<00:49, 1243.89it/s]

Checking face images:	37% ????	35846/97698	[00:31<00:51, 1195.68it/s]
Checking face images:	37% ????	35978/97698	[00:31<00:50, 1230.62it/s]
Checking face images:	37% ????	36111/97698	[00:31<00:49, 1244.60it/s]
Checking face images:	37% ????	36242/97698	[00:31<00:48, 1257.03it/s]
Checking face images:	37% ????	36369/97698	[00:31<00:49, 1247.20it/s]
Checking face images:	37% ????	36494/97698	[00:31<00:49, 1247.02it/s]
Checking face images:	38% ????	36643/97698	[00:31<00:46, 1308.31it/s]
Checking face images:	38% ????	36776/97698	[00:31<00:46, 1308.79it/s]
Checking face images:	38% ????	36907/97698	[00:31<00:46, 1296.97it/s]
Checking face images:	38% ????	37037/97698	[00:32<00:46, 1291.89it/s]
Checking face images:	38% ????	37167/97698	[00:32<00:48, 1243.44it/s]
Checking face images:	38% ????	37293/97698	[00:32<00:48, 1246.94it/s]
Checking face images:	38% ????	37418/97698	[00:32<00:48, 1247.02it/s]
Checking face images:	38% ????	37543/97698	[00:32<00:48, 1243.21it/s]
Checking face images:	39% ????	37675/97698	[00:32<00:47, 1264.54it/s]
Checking face images:	39% ????	37802/97698	[00:32<00:47, 1255.04it/s]
Checking face images:	39% ????	37928/97698	[00:32<00:47, 1255.96it/s]
Checking face images:	39% ????	38054/97698	[00:32<00:47, 1251.41it/s]
Checking face images:	39% ????	38180/97698	[00:32<00:47, 1243.90it/s]
Checking face images:	39% ????	38308/97698	[00:33<00:47, 1253.47it/s]
Checking face images:	39% ????	38434/97698	[00:33<00:48, 1233.54it/s]
Checking face images:	39% ????	38558/97698	[00:33<00:48, 1214.73it/s]
Checking face images:	40% ????	38680/97698	[00:33<00:48, 1205.97it/s]
Checking face images:	40% ????	38801/97698	[00:33<00:50, 1177.83it/s]
Checking face images:	40% ????	38919/97698	[00:33<01:01, 957.49it/s]
Checking face images:	40% ????	39022/97698	[00:33<01:11, 822.46it/s]
Checking face images:	40% ????	39112/97698	[00:34<01:15, 774.70it/s]
Checking face images:	40% ????	39195/97698	[00:34<01:17, 750.97it/s]
Checking face images:	40% ????	39274/97698	[00:34<01:20, 726.20it/s]
Checking face images:	40% ????	39349/97698	[00:34<01:22, 704.68it/s]
Checking face images:	40% ????	39421/97698	[00:34<01:25, 684.39it/s]
Checking face images:	40% ????	39491/97698	[00:34<01:27, 665.71it/s]
Checking face images:	40% ????	39558/97698	[00:34<01:28, 658.99it/s]
Checking face images:	41% ????	39625/97698	[00:34<01:30, 643.85it/s]
Checking face images:	41% ????	39690/97698	[00:34<01:32, 626.88it/s]
Checking face images:	41% ????	39753/97698	[00:35<01:33, 622.83it/s]
Checking face images:	41% ????	39816/97698	[00:35<01:32, 622.85it/s]
Checking face images:	41% ????	39881/97698	[00:35<01:32, 627.44it/s]
Checking face images:	41% ????	39948/97698	[00:35<01:31, 632.25it/s]
Checking face images:	41% ????	40017/97698	[00:35<01:29, 644.73it/s]
Checking face images:	41% ????	40085/97698	[00:35<01:28, 651.71it/s]
Checking face images:	41% ????	40151/97698	[00:35<01:28, 648.72it/s]
Checking face images:	41% ????	40220/97698	[00:35<01:27, 657.64it/s]
Checking face images:	41% ????	40286/97698	[00:35<01:29, 643.80it/s]
Checking face images:	41% ?????	40351/97698	[00:35<01:28, 644.49it/s]
Checking face images:	41% ?????	40418/97698	[00:36<01:28, 649.57it/s]
Checking face images:	41% ?????	40488/97698	[00:36<01:26, 663.99it/s]
Checking face images:	42% ?????	40555/97698	[00:36<01:27, 653.06it/s]
Checking face images:	42% ?????	40637/97698	[00:36<01:21, 701.07it/s]
Checking face images:	42% ?????	40738/97698	[00:36<01:12, 789.97it/s]
Checking face images:	42% ?????	40862/97698	[00:36<01:01, 922.95it/s]
Checking face images:	42% ?????	40992/97698	[00:36<00:54, 1033.22it/s]
Checking face images:	42% ?????	41117/97698	[00:36<00:51, 1096.99it/s]
Checking face images:	42% ?????	41233/97698	[00:36<00:50, 1114.90it/s]
Checking face images:	42% ?????	41349/97698	[00:36<00:50, 1126.63it/s]
Checking face images:	42% ?????	41468/97698	[00:37<00:49, 1142.50it/s]
Checking face images:	43% ?????	41598/97698	[00:37<00:47, 1187.46it/s]
Checking face images:	43% ?????	41731/97698	[00:37<00:45, 1227.38it/s]
Checking face images:	43% ?????	41861/97698	[00:37<00:44, 1247.79it/s]
Checking face images:	43% ?????	41986/97698	[00:37<00:45, 1213.14it/s]
Checking face images:	43% ?????	42120/97698	[00:37<00:44, 1246.46it/s]
Checking face images:	43% ?????	42247/97698	[00:37<00:44, 1248.08it/s]
Checking face images:	43% ?????	42378/97698	[00:37<00:43, 1262.95it/s]
Checking face images:	44% ?????	42505/97698	[00:37<00:43, 1263.21it/s]
Checking face images:	44% ?????	42632/97698	[00:37<00:43, 1255.83it/s]
Checking face images:	44% ?????	42758/97698	[00:38<00:44, 1248.11it/s]
Checking face images:	44% ?????	42883/97698	[00:38<00:44, 1244.01it/s]
Checking face images:	44% ?????	43020/97698	[00:38<00:42, 1272.51it/s]
Checking face images:	44% ?????	43150/97698	[00:38<00:42, 1280.41it/s]

Checking face images:	44%	?????		43279/97698	[00:38<00:45,	1199.13it/s]
Checking face images:	44%	?????		43402/97698	[00:38<00:44,	1207.03it/s]
Checking face images:	45%	?????		43525/97698	[00:38<00:44,	1213.36it/s]
Checking face images:	45%	?????		43655/97698	[00:38<00:43,	1235.05it/s]
Checking face images:	45%	?????		43784/97698	[00:38<00:43,	1250.59it/s]
Checking face images:	45%	?????		43914/97698	[00:38<00:42,	1265.13it/s]
Checking face images:	45%	?????		44045/97698	[00:39<00:42,	1268.41it/s]
Checking face images:	45%	?????		44173/97698	[00:39<00:44,	1206.84it/s]
Checking face images:	45%	?????		44331/97698	[00:39<00:40,	1306.38it/s]
Checking face images:	46%	?????		44463/97698	[00:39<00:41,	1284.06it/s]
Checking face images:	46%	?????		44593/97698	[00:39<00:42,	1249.07it/s]
Checking face images:	46%	?????		44722/97698	[00:39<00:42,	1260.68it/s]
Checking face images:	46%	?????		44850/97698	[00:39<00:42,	1256.55it/s]
Checking face images:	46%	?????		44984/97698	[00:39<00:41,	1276.14it/s]
Checking face images:	46%	?????		45114/97698	[00:39<00:42,	1230.16it/s]
Checking face images:	46%	?????		45257/97698	[00:40<00:41,	1269.71it/s]
Checking face images:	46%	?????		45388/97698	[00:40<00:40,	1275.86it/s]
Checking face images:	47%	?????		45517/97698	[00:40<00:40,	1275.40it/s]
Checking face images:	47%	?????		45646/97698	[00:40<00:42,	1236.10it/s]
Checking face images:	47%	?????		45776/97698	[00:40<00:42,	1213.81it/s]
Checking face images:	47%	?????		45909/97698	[00:40<00:41,	1246.02it/s]
Checking face images:	47%	?????		46034/97698	[00:40<00:41,	1238.42it/s]
Checking face images:	47%	?????		46161/97698	[00:40<00:41,	1235.61it/s]
Checking face images:	47%	?????		46299/97698	[00:40<00:40,	1274.47it/s]
Checking face images:	48%	?????		46427/97698	[00:40<00:40,	1274.64it/s]
Checking face images:	48%	?????		46555/97698	[00:41<00:40,	1265.94it/s]
Checking face images:	48%	?????		46691/97698	[00:41<00:40,	1268.22it/s]
Checking face images:	48%	?????		46831/97698	[00:41<00:39,	1286.94it/s]
Checking face images:	48%	?????		46971/97698	[00:41<00:38,	1317.28it/s]
Checking face images:	48%	?????		47103/97698	[00:41<00:38,	1317.10it/s]
Checking face images:	48%	?????		47235/97698	[00:41<00:40,	1259.34it/s]
Checking face images:	48%	?????		47366/97698	[00:41<00:39,	1263.45it/s]
Checking face images:	49%	?????		47494/97698	[00:41<00:39,	1266.78it/s]
Checking face images:	49%	?????		47627/97698	[00:41<00:39,	1283.26it/s]
Checking face images:	49%	?????		47756/97698	[00:42<00:39,	1269.66it/s]
Checking face images:	49%	?????		47890/97698	[00:42<00:38,	1282.46it/s]
Checking face images:	49%	?????		48027/97698	[00:42<00:37,	1307.60it/s]
Checking face images:	49%	?????		48162/97698	[00:42<00:37,	1318.70it/s]
Checking face images:	49%	?????		48294/97698	[00:42<00:38,	1296.41it/s]
Checking face images:	50%	?????		48426/97698	[00:42<00:37,	1296.84it/s]
Checking face images:	50%	?????		48556/97698	[00:42<00:39,	1251.86it/s]
Checking face images:	50%	?????		48688/97698	[00:42<00:38,	1269.38it/s]
Checking face images:	50%	?????		48822/97698	[00:42<00:37,	1286.26it/s]
Checking face images:	50%	?????		48951/97698	[00:42<00:38,	1276.18it/s]
Checking face images:	50%	?????		49081/97698	[00:43<00:37,	1282.72it/s]
Checking face images:	50%	?????		49213/97698	[00:43<00:37,	1290.86it/s]
Checking face images:	51%	?????		49345/97698	[00:43<00:37,	1296.93it/s]
Checking face images:	51%	?????		49475/97698	[00:43<00:37,	1297.42it/s]
Checking face images:	51%	?????		49605/97698	[00:43<00:37,	1296.84it/s]
Checking face images:	51%	?????		49735/97698	[00:43<00:37,	1286.09it/s]
Checking face images:	51%	?????		49864/97698	[00:43<00:39,	1224.85it/s]
Checking face images:	51%	?????		49997/97698	[00:43<00:38,	1250.99it/s]
Checking face images:	51%	??????		50130/97698	[00:43<00:37,	1271.05it/s]
Checking face images:	51%	??????		50264/97698	[00:43<00:36,	1290.71it/s]
Checking face images:	52%	??????		50394/97698	[00:44<00:36,	1279.27it/s]
Checking face images:	52%	??????		50523/97698	[00:44<00:36,	1281.24it/s]
Checking face images:	52%	??????		50652/97698	[00:44<00:36,	1276.00it/s]
Checking face images:	52%	??????		50795/97698	[00:44<00:35,	1317.59it/s]
Checking face images:	52%	??????		50927/97698	[00:44<00:35,	1311.25it/s]
Checking face images:	52%	??????		51059/97698	[00:44<00:35,	1309.79it/s]
Checking face images:	52%	??????		51191/97698	[00:44<00:37,	1250.52it/s]
Checking face images:	53%	??????		51323/97698	[00:44<00:37,	1246.15it/s]
Checking face images:	53%	??????		51469/97698	[00:44<00:35,	1294.17it/s]
Checking face images:	53%	??????		51604/97698	[00:45<00:35,	1308.05it/s]
Checking face images:	53%	??????		51736/97698	[00:45<00:35,	1295.90it/s]
Checking face images:	53%	??????		51866/97698	[00:45<00:35,	1292.52it/s]
Checking face images:	53%	??????		51998/97698	[00:45<00:35,	1296.76it/s]
Checking face images:	53%	??????		52128/97698	[00:45<00:35,	1288.04it/s]
Checking face images:	53%	??????		52259/97698	[00:45<00:35,	1294.42it/s]

Checking face images:	54%	??????		52391/97698	[00:45<00:34,	1298.78it/s]
Checking face images:	54%	??????		52521/97698	[00:45<00:36,	1243.31it/s]
Checking face images:	54%	??????		52656/97698	[00:45<00:35,	1270.39it/s]
Checking face images:	54%	??????		52791/97698	[00:45<00:35,	1279.08it/s]
Checking face images:	54%	??????		52922/97698	[00:46<00:34,	1284.92it/s]
Checking face images:	54%	??????		53054/97698	[00:46<00:34,	1293.87it/s]
Checking face images:	54%	??????		53184/97698	[00:46<00:34,	1294.70it/s]
Checking face images:	55%	??????		53314/97698	[00:46<00:37,	1193.01it/s]
Checking face images:	55%	??????		53435/97698	[00:46<00:43,	1013.17it/s]
Checking face images:	55%	??????		53542/97698	[00:46<00:49,	900.15it/s]
Checking face images:	55%	??????		53638/97698	[00:46<00:53,	830.27it/s]
Checking face images:	55%	??????		53725/97698	[00:46<00:56,	777.26it/s]
Checking face images:	55%	??????		53806/97698	[00:47<00:58,	749.96it/s]
Checking face images:	55%	??????		53883/97698	[00:47<00:59,	736.67it/s]
Checking face images:	55%	??????		53958/97698	[00:47<01:01,	714.56it/s]
Checking face images:	55%	??????		54030/97698	[00:47<01:02,	700.76it/s]
Checking face images:	55%	??????		54101/97698	[00:47<01:02,	696.34it/s]
Checking face images:	55%	??????		54171/97698	[00:47<01:02,	690.94it/s]
Checking face images:	56%	??????		54241/97698	[00:47<01:03,	682.12it/s]
Checking face images:	56%	??????		54310/97698	[00:47<01:04,	675.60it/s]
Checking face images:	56%	??????		54378/97698	[00:47<01:04,	673.02it/s]
Checking face images:	56%	??????		54446/97698	[00:48<01:04,	670.62it/s]
Checking face images:	56%	??????		54517/97698	[00:48<01:03,	680.05it/s]
Checking face images:	56%	??????		54589/97698	[00:48<01:02,	688.08it/s]
Checking face images:	56%	??????		54658/97698	[00:48<01:02,	683.41it/s]
Checking face images:	56%	??????		54730/97698	[00:48<01:02,	692.87it/s]
Checking face images:	56%	??????		54800/97698	[00:48<01:03,	680.01it/s]
Checking face images:	56%	??????		54874/97698	[00:48<01:01,	694.16it/s]
Checking face images:	56%	??????		54944/97698	[00:48<01:01,	694.38it/s]
Checking face images:	56%	??????		55017/97698	[00:48<01:01,	698.19it/s]
Checking face images:	56%	??????		55087/97698	[00:48<01:02,	680.23it/s]
Checking face images:	56%	??????		55174/97698	[00:49<00:57,	734.86it/s]
Checking face images:	57%	??????		55304/97698	[00:49<00:47,	900.05it/s]
Checking face images:	57%	??????		55407/97698	[00:49<00:45,	936.78it/s]
Checking face images:	57%	??????		55502/97698	[00:49<01:10,	597.04it/s]
Checking face images:	57%	??????		55578/97698	[00:49<01:22,	512.29it/s]
Checking face images:	57%	??????		55642/97698	[00:50<01:35,	440.97it/s]
Checking face images:	57%	??????		55696/97698	[00:50<01:41,	412.40it/s]
Checking face images:	57%	??????		55744/97698	[00:50<01:43,	403.59it/s]
Checking face images:	57%	??????		55789/97698	[00:50<01:45,	398.73it/s]
Checking face images:	57%	??????		55833/97698	[00:50<01:43,	403.90it/s]
Checking face images:	57%	??????		55881/97698	[00:50<01:40,	415.18it/s]
Checking face images:	57%	??????		56019/97698	[00:50<01:03,	655.88it/s]
Checking face images:	57%	??????		56152/97698	[00:50<00:49,	832.03it/s]
Checking face images:	58%	??????		56269/97698	[00:50<00:45,	917.44it/s]
Checking face images:	58%	??????		56366/97698	[00:51<01:03,	646.69it/s]
Checking face images:	58%	??????		56445/97698	[00:51<01:13,	563.06it/s]
Checking face images:	58%	??????		56513/97698	[00:51<01:22,	497.00it/s]
Checking face images:	58%	??????		56572/97698	[00:51<01:28,	463.99it/s]
Checking face images:	58%	??????		56625/97698	[00:51<01:31,	446.85it/s]
Checking face images:	58%	??????		56674/97698	[00:51<01:35,	428.59it/s]
Checking face images:	58%	??????		56731/97698	[00:52<01:29,	458.02it/s]
Checking face images:	58%	??????		56780/97698	[00:52<01:38,	417.23it/s]
Checking face images:	58%	??????		56860/97698	[00:52<01:20,	507.13it/s]
Checking face images:	58%	??????		56976/97698	[00:52<01:00,	670.42it/s]
Checking face images:	58%	??????		57101/97698	[00:52<00:49,	822.28it/s]
Checking face images:	59%	??????		57190/97698	[00:52<00:51,	785.53it/s]
Checking face images:	59%	??????		57274/97698	[00:52<01:11,	564.93it/s]
Checking face images:	59%	??????		57342/97698	[00:53<01:19,	508.31it/s]
Checking face images:	59%	??????		57402/97698	[00:53<01:26,	467.41it/s]
Checking face images:	59%	??????		57455/97698	[00:53<01:30,	444.47it/s]
Checking face images:	59%	??????		57504/97698	[00:53<01:38,	409.84it/s]
Checking face images:	59%	??????		57548/97698	[00:53<01:41,	397.26it/s]
Checking face images:	59%	??????		57602/97698	[00:53<01:33,	428.81it/s]
Checking face images:	59%	??????		57647/97698	[00:53<01:36,	413.00it/s]
Checking face images:	59%	??????		57772/97698	[00:53<01:04,	619.53it/s]
Checking face images:	59%	??????		57885/97698	[00:54<00:52,	753.12it/s]
Checking face images:	59%	??????		58016/97698	[00:54<00:43,	903.26it/s]
Checking face images:	60%	??????		58149/97698	[00:54<00:39,	1011.53it/s]

Checking face images:	60%	??????		58286/97698	[00:54<00:35,	1110.11it/s]
Checking face images:	60%	??????		58413/97698	[00:54<00:34,	1153.58it/s]
Checking face images:	60%	??????		58543/97698	[00:54<00:32,	1192.60it/s]
Checking face images:	60%	??????		58672/97698	[00:54<00:32,	1212.55it/s]
Checking face images:	60%	??????		58803/97698	[00:54<00:31,	1239.38it/s]
Checking face images:	60%	??????		58928/97698	[00:54<00:31,	1238.74it/s]
Checking face images:	60%	??????		59053/97698	[00:55<00:31,	1240.21it/s]
Checking face images:	61%	??????		59179/97698	[00:55<00:31,	1240.62it/s]
Checking face images:	61%	??????		59304/97698	[00:55<00:31,	1206.84it/s]
Checking face images:	61%	??????		59426/97698	[00:55<00:31,	1202.34it/s]
Checking face images:	61%	??????		59558/97698	[00:55<00:31,	1226.34it/s]
Checking face images:	61%	??????		59690/97698	[00:55<00:30,	1246.55it/s]
Checking face images:	61%	??????		59822/97698	[00:55<00:29,	1267.31it/s]
Checking face images:	61%	??????		59953/97698	[00:55<00:29,	1274.65it/s]
Checking face images:	61%	??????		60084/97698	[00:55<00:29,	1281.55it/s]
Checking face images:	62%	??????		60213/97698	[00:55<00:29,	1281.47it/s]
Checking face images:	62%	??????		60342/97698	[00:56<00:30,	1224.06it/s]
Checking face images:	62%	??????		60473/97698	[00:56<00:29,	1241.10it/s]
Checking face images:	62%	??????		60598/97698	[00:56<00:30,	1204.70it/s]
Checking face images:	62%	??????		60736/97698	[00:56<00:30,	1195.25it/s]
Checking face images:	62%	??????		60881/97698	[00:56<00:29,	1261.27it/s]
Checking face images:	62%	??????		61012/97698	[00:56<00:28,	1274.94it/s]
Checking face images:	63%	??????		61141/97698	[00:56<00:28,	1263.89it/s]
Checking face images:	63%	??????		61268/97698	[00:56<00:29,	1225.33it/s]
Checking face images:	63%	??????		61391/97698	[00:56<00:29,	1210.86it/s]
Checking face images:	63%	??????		61514/97698	[00:56<00:29,	1215.03it/s]
Checking face images:	63%	??????		61638/97698	[00:57<00:29,	1221.69it/s]
Checking face images:	63%	??????		61767/97698	[00:57<00:29,	1236.93it/s]
Checking face images:	63%	??????		61891/97698	[00:57<00:30,	1182.99it/s]
Checking face images:	63%	??????		62014/97698	[00:57<00:29,	1195.61it/s]
Checking face images:	64%	??????		62138/97698	[00:57<00:29,	1208.02it/s]
Checking face images:	64%	??????		62261/97698	[00:57<00:29,	1210.88it/s]
Checking face images:	64%	??????		62386/97698	[00:57<00:29,	1214.85it/s]
Checking face images:	64%	??????		62515/97698	[00:57<00:28,	1234.63it/s]
Checking face images:	64%	??????		62647/97698	[00:57<00:27,	1255.39it/s]
Checking face images:	64%	??????		62773/97698	[00:58<00:27,	1247.62it/s]
Checking face images:	64%	??????		62900/97698	[00:58<00:27,	1252.80it/s]
Checking face images:	65%	??????		63034/97698	[00:58<00:27,	1277.49it/s]
Checking face images:	65%	??????		63162/97698	[00:58<00:28,	1220.88it/s]
Checking face images:	65%	??????		63289/97698	[00:58<00:29,	1171.24it/s]
Checking face images:	65%	??????		63446/97698	[00:58<00:26,	1282.36it/s]
Checking face images:	65%	??????		63576/97698	[00:58<00:26,	1283.70it/s]
Checking face images:	65%	??????		63713/97698	[00:58<00:26,	1304.74it/s]
Checking face images:	65%	??????		63845/97698	[00:58<00:26,	1295.26it/s]
Checking face images:	65%	??????		63978/97698	[00:58<00:25,	1303.52it/s]
Checking face images:	66%	??????		64109/97698	[00:59<00:26,	1273.10it/s]
Checking face images:	66%	??????		64237/97698	[00:59<00:32,	1034.78it/s]
Checking face images:	66%	??????		64348/97698	[00:59<00:36,	920.20it/s]
Checking face images:	66%	??????		64447/97698	[00:59<00:39,	849.98it/s]
Checking face images:	66%	??????		64537/97698	[00:59<00:41,	802.62it/s]
Checking face images:	66%	??????		64621/97698	[00:59<00:42,	773.33it/s]
Checking face images:	66%	??????		64701/97698	[00:59<00:44,	745.33it/s]
Checking face images:	66%	??????		64777/97698	[01:00<00:45,	728.05it/s]
Checking face images:	66%	??????		64851/97698	[01:00<00:45,	722.97it/s]
Checking face images:	66%	??????		64924/97698	[01:00<00:46,	702.20it/s]
Checking face images:	67%	??????		64995/97698	[01:00<00:46,	697.64it/s]
Checking face images:	67%	??????		65065/97698	[01:00<00:47,	692.41it/s]
Checking face images:	67%	??????		65135/97698	[01:00<00:47,	691.05it/s]
Checking face images:	67%	??????		65205/97698	[01:00<00:47,	678.97it/s]
Checking face images:	67%	??????		65273/97698	[01:00<00:47,	677.49it/s]
Checking face images:	67%	??????		65345/97698	[01:00<00:47,	677.66it/s]
Checking face images:	67%	??????		65422/97698	[01:00<00:46,	696.74it/s]
Checking face images:	67%	??????		65493/97698	[01:01<00:46,	691.41it/s]
Checking face images:	67%	??????		65569/97698	[01:01<00:45,	709.49it/s]
Checking face images:	67%	??????		65641/97698	[01:01<00:46,	693.59it/s]
Checking face images:	67%	??????		65716/97698	[01:01<00:45,	708.95it/s]
Checking face images:	67%	??????		65788/97698	[01:01<00:44,	710.78it/s]
Checking face images:	67%	??????		65860/97698	[01:01<00:44,	710.94it/s]
Checking face images:	67%	??????		65932/97698	[01:01<00:44,	712.54it/s]

Checking face images:	68%	???????		66004/97698	[01:01<00:44,	709.36it/s]
Checking face images:	68%	???????		66120/97698	[01:01<00:37,	839.57it/s]
Checking face images:	68%	???????		66252/97698	[01:01<00:32,	980.75it/s]
Checking face images:	68%	???????		66375/97698	[01:02<00:29,	1049.42it/s]
Checking face images:	68%	???????		66515/97698	[01:02<00:27,	1152.38it/s]
Checking face images:	68%	???????		66644/97698	[01:02<00:26,	1192.29it/s]
Checking face images:	68%	???????		66773/97698	[01:02<00:25,	1219.48it/s]
Checking face images:	68%	???????		66896/97698	[01:02<00:25,	1220.46it/s]
Checking face images:	69%	???????		67019/97698	[01:02<00:25,	1182.50it/s]
Checking face images:	69%	???????		67138/97698	[01:02<00:26,	1164.83it/s]
Checking face images:	69%	???????		67271/97698	[01:02<00:25,	1208.33it/s]
Checking face images:	69%	???????		67405/97698	[01:02<00:25,	1208.59it/s]
Checking face images:	69%	???????		67554/97698	[01:03<00:23,	1277.38it/s]
Checking face images:	69%	???????		67683/97698	[01:03<00:23,	1279.86it/s]
Checking face images:	69%	???????		67812/97698	[01:03<00:23,	1278.37it/s]
Checking face images:	70%	???????		67940/97698	[01:03<00:23,	1275.63it/s]
Checking face images:	70%	???????		68070/97698	[01:03<00:23,	1272.93it/s]
Checking face images:	70%	???????		68198/97698	[01:03<00:23,	1252.47it/s]
Checking face images:	70%	???????		68324/97698	[01:03<00:24,	1205.64it/s]
Checking face images:	70%	???????		68451/97698	[01:03<00:23,	1220.15it/s]
Checking face images:	70%	???????		68583/97698	[01:03<00:23,	1238.78it/s]
Checking face images:	70%	???????		68723/97698	[01:03<00:22,	1285.48it/s]
Checking face images:	70%	???????		68854/97698	[01:04<00:22,	1291.21it/s]
Checking face images:	71%	???????		68984/97698	[01:04<00:22,	1291.07it/s]
Checking face images:	71%	???????		69115/97698	[01:04<00:22,	1294.17it/s]
Checking face images:	71%	???????		69248/97698	[01:04<00:21,	1302.91it/s]
Checking face images:	71%	???????		69381/97698	[01:04<00:21,	1308.79it/s]
Checking face images:	71%	???????		69512/97698	[01:04<00:21,	1281.84it/s]
Checking face images:	71%	???????		69641/97698	[01:04<00:22,	1232.33it/s]
Checking face images:	71%	???????		69775/97698	[01:04<00:22,	1259.47it/s]
Checking face images:	72%	???????		69909/97698	[01:04<00:21,	1280.68it/s]
Checking face images:	72%	???????		70038/97698	[01:04<00:21,	1278.04it/s]
Checking face images:	72%	???????		70167/97698	[01:05<00:21,	1263.77it/s]
Checking face images:	72%	???????		70294/97698	[01:05<00:21,	1258.40it/s]
Checking face images:	72%	???????		70422/97698	[01:05<00:21,	1263.27it/s]
Checking face images:	72%	???????		70552/97698	[01:05<00:21,	1271.60it/s]
Checking face images:	72%	???????		70682/97698	[01:05<00:21,	1277.23it/s]
Checking face images:	72%	???????		70810/97698	[01:05<00:21,	1238.65it/s]
Checking face images:	73%	???????		70935/97698	[01:05<00:22,	1201.60it/s]
Checking face images:	73%	???????		71066/97698	[01:05<00:21,	1229.95it/s]
Checking face images:	73%	???????		71192/97698	[01:05<00:21,	1238.43it/s]
Checking face images:	73%	???????		71321/97698	[01:06<00:21,	1252.32it/s]
Checking face images:	73%	???????		71450/97698	[01:06<00:20,	1260.38it/s]
Checking face images:	73%	???????		71577/97698	[01:06<00:20,	1260.33it/s]
Checking face images:	73%	???????		71706/97698	[01:06<00:20,	1265.15it/s]
Checking face images:	74%	???????		71838/97698	[01:06<00:20,	1280.88it/s]
Checking face images:	74%	???????		71967/97698	[01:06<00:20,	1280.53it/s]
Checking face images:	74%	???????		72097/97698	[01:06<00:19,	1284.54it/s]
Checking face images:	74%	???????		72226/97698	[01:06<00:20,	1215.19it/s]
Checking face images:	74%	???????		72352/97698	[01:06<00:20,	1225.66it/s]
Checking face images:	74%	???????		72486/97698	[01:06<00:20,	1255.23it/s]
Checking face images:	74%	???????		72619/97698	[01:07<00:19,	1264.81it/s]
Checking face images:	74%	???????		72746/97698	[01:07<00:19,	1256.76it/s]
Checking face images:	75%	???????		72872/97698	[01:07<00:19,	1245.72it/s]
Checking face images:	75%	???????		73005/97698	[01:07<00:19,	1263.83it/s]
Checking face images:	75%	???????		73133/97698	[01:07<00:19,	1267.68it/s]
Checking face images:	75%	???????		73260/97698	[01:07<00:19,	1263.56it/s]
Checking face images:	75%	???????		73387/97698	[01:07<00:19,	1252.41it/s]
Checking face images:	75%	???????		73513/97698	[01:07<00:19,	1211.50it/s]
Checking face images:	75%	???????		73645/97698	[01:07<00:19,	1238.85it/s]
Checking face images:	76%	???????		73772/97698	[01:07<00:19,	1247.73it/s]
Checking face images:	76%	???????		73902/97698	[01:08<00:18,	1262.95it/s]
Checking face images:	76%	???????		74036/97698	[01:08<00:18,	1282.95it/s]
Checking face images:	76%	???????		74165/97698	[01:08<00:18,	1269.28it/s]
Checking face images:	76%	???????		74300/97698	[01:08<00:18,	1291.61it/s]
Checking face images:	76%	???????		74430/97698	[01:08<00:18,	1280.30it/s]
Checking face images:	76%	???????		74563/97698	[01:08<00:17,	1290.85it/s]
Checking face images:	76%	???????		74693/97698	[01:08<00:17,	1288.63it/s]
Checking face images:	77%	???????		74822/97698	[01:08<00:19,	1178.59it/s]

Checking face images:	77%	????????	74945/97698	[01:08<00:19,	1191.38it/s]
Checking face images:	77%	????????	75066/97698	[01:09<00:18,	1194.69it/s]
Checking face images:	77%	????????	75191/97698	[01:09<00:18,	1202.64it/s]
Checking face images:	77%	????????	75321/97698	[01:09<00:18,	1228.64it/s]
Checking face images:	77%	????????	75445/97698	[01:09<00:18,	1226.42it/s]
Checking face images:	77%	????????	75570/97698	[01:09<00:17,	1232.69it/s]
Checking face images:	77%	????????	75701/97698	[01:09<00:17,	1254.25it/s]
Checking face images:	78%	????????	75827/97698	[01:09<00:17,	1252.74it/s]
Checking face images:	78%	????????	75964/97698	[01:09<00:16,	1286.99it/s]
Checking face images:	78%	????????	76093/97698	[01:09<00:17,	1229.60it/s]
Checking face images:	78%	????????	76218/97698	[01:09<00:17,	1233.65it/s]
Checking face images:	78%	????????	76342/97698	[01:10<00:17,	1228.83it/s]
Checking face images:	78%	????????	76478/97698	[01:10<00:16,	1265.19it/s]
Checking face images:	78%	????????	76614/97698	[01:10<00:16,	1256.08it/s]
Checking face images:	79%	????????	76758/97698	[01:10<00:16,	1298.68it/s]
Checking face images:	79%	????????	76889/97698	[01:10<00:15,	1301.24it/s]
Checking face images:	79%	????????	77022/97698	[01:10<00:15,	1307.06it/s]
Checking face images:	79%	????????	77153/97698	[01:10<00:15,	1304.11it/s]
Checking face images:	79%	????????	77284/97698	[01:10<00:15,	1305.78it/s]
Checking face images:	79%	????????	77415/97698	[01:10<00:16,	1232.90it/s]
Checking face images:	79%	????????	77547/97698	[01:10<00:16,	1256.20it/s]
Checking face images:	80%	????????	77674/97698	[01:11<00:16,	1223.29it/s]
Checking face images:	80%	????????	77826/97698	[01:11<00:15,	1306.29it/s]
Checking face images:	80%	????????	77958/97698	[01:11<00:15,	1290.26it/s]
Checking face images:	80%	????????	78088/97698	[01:11<00:15,	1290.09it/s]
Checking face images:	80%	????????	78219/97698	[01:11<00:15,	1285.59it/s]
Checking face images:	80%	????????	78350/97698	[01:11<00:14,	1291.84it/s]
Checking face images:	80%	????????	78480/97698	[01:11<00:14,	1287.12it/s]
Checking face images:	80%	????????	78610/97698	[01:11<00:14,	1281.19it/s]
Checking face images:	81%	????????	78739/97698	[01:11<00:18,	1016.09it/s]
Checking face images:	81%	????????	78850/97698	[01:12<00:20,	920.05it/s]
Checking face images:	81%	????????	78950/97698	[01:12<00:22,	847.27it/s]
Checking face images:	81%	????????	79041/97698	[01:12<00:23,	794.20it/s]
Checking face images:	81%	????????	79125/97698	[01:12<00:24,	759.84it/s]
Checking face images:	81%	????????	79204/97698	[01:12<00:25,	732.06it/s]
Checking face images:	81%	????????	79279/97698	[01:12<00:26,	700.29it/s]
Checking face images:	81%	????????	79350/97698	[01:12<00:26,	688.72it/s]
Checking face images:	81%	????????	79420/97698	[01:13<00:27,	674.35it/s]
Checking face images:	81%	????????	79488/97698	[01:13<00:27,	669.36it/s]
Checking face images:	81%	????????	79556/97698	[01:13<00:27,	669.71it/s]
Checking face images:	82%	????????	79624/97698	[01:13<00:27,	668.90it/s]
Checking face images:	82%	????????	79691/97698	[01:13<00:26,	668.84it/s]
Checking face images:	82%	????????	79759/97698	[01:13<00:26,	668.73it/s]
Checking face images:	82%	????????	79832/97698	[01:13<00:26,	684.60it/s]
Checking face images:	82%	????????	79906/97698	[01:13<00:25,	695.88it/s]
Checking face images:	82%	????????	79977/97698	[01:13<00:25,	699.01it/s]
Checking face images:	82%	????????	80047/97698	[01:13<00:25,	688.62it/s]
Checking face images:	82%	????????	80116/97698	[01:14<00:25,	686.48it/s]
Checking face images:	82%	????????	80189/97698	[01:14<00:25,	697.18it/s]
Checking face images:	82%	????????	80261/97698	[01:14<00:24,	703.10it/s]
Checking face images:	82%	????????	80332/97698	[01:14<00:24,	700.11it/s]
Checking face images:	82%	????????	80405/97698	[01:14<00:24,	703.09it/s]
Checking face images:	82%	????????	80477/97698	[01:14<00:24,	704.67it/s]
Checking face images:	83%	????????	80614/97698	[01:14<00:18,	899.77it/s]
Checking face images:	83%	????????	80742/97698	[01:14<00:16,	1010.28it/s]
Checking face images:	83%	????????	80877/97698	[01:14<00:15,	1108.42it/s]
Checking face images:	83%	????????	81010/97698	[01:14<00:14,	1160.32it/s]
Checking face images:	83%	????????	81149/97698	[01:15<00:13,	1225.68it/s]
Checking face images:	83%	????????	81272/97698	[01:15<00:14,	1154.79it/s]
Checking face images:	83%	????????	81395/97698	[01:15<00:13,	1166.32it/s]
Checking face images:	83%	????????	81552/97698	[01:15<00:12,	1278.97it/s]
Checking face images:	84%	????????	81684/97698	[01:15<00:12,	1289.98it/s]
Checking face images:	84%	????????	81814/97698	[01:15<00:12,	1281.56it/s]
Checking face images:	84%	????????	81943/97698	[01:15<00:12,	1268.74it/s]
Checking face images:	84%	????????	82071/97698	[01:15<00:12,	1264.57it/s]
Checking face images:	84%	????????	82198/97698	[01:15<00:12,	1261.36it/s]
Checking face images:	84%	????????	82326/97698	[01:15<00:12,	1263.76it/s]
Checking face images:	84%	????????	82453/97698	[01:16<00:12,	1259.55it/s]
Checking face images:	85%	????????	82580/97698	[01:16<00:12,	1225.50it/s]

Checking face images:	85%	?????????	82711/97698	[01:16<00:12,	1229.79it/s]
Checking face images:	85%	?????????	82848/97698	[01:16<00:11,	1263.21it/s]
Checking face images:	85%	?????????	82975/97698	[01:16<00:11,	1264.62it/s]
Checking face images:	85%	?????????	83102/97698	[01:16<00:11,	1226.86it/s]
Checking face images:	85%	?????????	83250/97698	[01:16<00:11,	1297.29it/s]
Checking face images:	85%	?????????	83381/97698	[01:16<00:11,	1297.85it/s]
Checking face images:	85%	?????????	83514/97698	[01:16<00:10,	1298.28it/s]
Checking face images:	86%	?????????	83647/97698	[01:17<00:10,	1306.56it/s]
Checking face images:	86%	?????????	83781/97698	[01:17<00:10,	1313.56it/s]
Checking face images:	86%	?????????	83913/97698	[01:17<00:11,	1249.28it/s]
Checking face images:	86%	?????????	84039/97698	[01:17<00:11,	1147.49it/s]
Checking face images:	86%	?????????	84171/97698	[01:17<00:11,	1193.64it/s]
Checking face images:	86%	?????????	84304/97698	[01:17<00:10,	1230.65it/s]
Checking face images:	86%	?????????	84432/97698	[01:17<00:10,	1243.10it/s]
Checking face images:	87%	?????????	84558/97698	[01:17<00:10,	1241.86it/s]
Checking face images:	87%	?????????	84690/97698	[01:17<00:10,	1260.14it/s]
Checking face images:	87%	?????????	84822/97698	[01:17<00:10,	1275.56it/s]
Checking face images:	87%	?????????	84950/97698	[01:18<00:10,	1271.82it/s]
Checking face images:	87%	?????????	85082/97698	[01:18<00:09,	1281.02it/s]
Checking face images:	87%	?????????	85211/97698	[01:18<00:10,	1233.88it/s]
Checking face images:	87%	?????????	85337/97698	[01:18<00:09,	1240.83it/s]
Checking face images:	87%	?????????	85462/97698	[01:18<00:09,	1238.20it/s]
Checking face images:	88%	?????????	85587/97698	[01:18<00:09,	1241.59it/s]
Checking face images:	88%	?????????	85717/97698	[01:18<00:09,	1253.67it/s]
Checking face images:	88%	?????????	85843/97698	[01:18<00:09,	1251.19it/s]
Checking face images:	88%	?????????	85980/97698	[01:18<00:09,	1283.61it/s]
Checking face images:	88%	?????????	86109/97698	[01:18<00:09,	1278.55it/s]
Checking face images:	88%	?????????	86246/97698	[01:19<00:08,	1302.41it/s]
Checking face images:	88%	?????????	86377/97698	[01:19<00:08,	1289.87it/s]
Checking face images:	89%	?????????	86507/97698	[01:19<00:08,	1256.57it/s]
Checking face images:	89%	?????????	86637/97698	[01:19<00:08,	1252.61it/s]
Checking face images:	89%	?????????	86772/97698	[01:19<00:08,	1275.74it/s]
Checking face images:	89%	?????????	86901/97698	[01:19<00:08,	1277.61it/s]
Checking face images:	89%	?????????	87031/97698	[01:19<00:08,	1283.05it/s]
Checking face images:	89%	?????????	87160/97698	[01:19<00:08,	1269.88it/s]
Checking face images:	89%	?????????	87291/97698	[01:19<00:08,	1262.59it/s]
Checking face images:	89%	?????????	87423/97698	[01:20<00:08,	1278.59it/s]
Checking face images:	90%	?????????	87551/97698	[01:20<00:07,	1272.37it/s]
Checking face images:	90%	?????????	87685/97698	[01:20<00:07,	1291.12it/s]
Checking face images:	90%	?????????	87815/97698	[01:20<00:08,	1235.18it/s]
Checking face images:	90%	?????????	87942/97698	[01:20<00:07,	1240.11it/s]
Checking face images:	90%	?????????	88075/97698	[01:20<00:07,	1255.33it/s]
Checking face images:	90%	?????????	88210/97698	[01:20<00:07,	1281.99it/s]
Checking face images:	90%	?????????	88339/97698	[01:20<00:07,	1282.86it/s]
Checking face images:	91%	?????????	88468/97698	[01:20<00:07,	1268.95it/s]
Checking face images:	91%	?????????	88599/97698	[01:20<00:07,	1280.43it/s]
Checking face images:	91%	?????????	88728/97698	[01:21<00:07,	1271.09it/s]
Checking face images:	91%	?????????	88857/97698	[01:21<00:06,	1273.94it/s]
Checking face images:	91%	?????????	88991/97698	[01:21<00:06,	1286.21it/s]
Checking face images:	91%	?????????	89120/97698	[01:21<00:06,	1240.38it/s]
Checking face images:	91%	?????????	89252/97698	[01:21<00:06,	1261.61it/s]
Checking face images:	91%	?????????	89382/97698	[01:21<00:06,	1269.85it/s]
Checking face images:	92%	?????????	89516/97698	[01:21<00:06,	1286.83it/s]
Checking face images:	92%	?????????	89645/97698	[01:21<00:06,	1257.06it/s]
Checking face images:	92%	?????????	89771/97698	[01:21<00:06,	1252.41it/s]
Checking face images:	92%	?????????	89897/97698	[01:21<00:06,	1254.26it/s]
Checking face images:	92%	?????????	90023/97698	[01:22<00:06,	1240.55it/s]
Checking face images:	92%	?????????	90148/97698	[01:22<00:06,	1242.20it/s]
Checking face images:	92%	?????????	90275/97698	[01:22<00:05,	1246.68it/s]
Checking face images:	93%	?????????	90400/97698	[01:22<00:06,	1205.56it/s]
Checking face images:	93%	?????????	90532/97698	[01:22<00:05,	1237.29it/s]
Checking face images:	93%	?????????	90664/97698	[01:22<00:05,	1254.99it/s]
Checking face images:	93%	?????????	90793/97698	[01:22<00:05,	1263.62it/s]
Checking face images:	93%	?????????	90923/97698	[01:22<00:05,	1273.33it/s]
Checking face images:	93%	?????????	91051/97698	[01:22<00:05,	1268.66it/s]
Checking face images:	93%	?????????	91185/97698	[01:23<00:05,	1286.53it/s]
Checking face images:	93%	?????????	91316/97698	[01:23<00:04,	1292.20it/s]
Checking face images:	94%	?????????	91446/97698	[01:23<00:04,	1287.53it/s]
Checking face images:	94%	?????????	91575/97698	[01:23<00:04,	1254.17it/s]

Checking face images:	94%	??????????	91701/97698	[01:23<00:04,	1212.52it/s]
Checking face images:	94%	??????????	91831/97698	[01:23<00:04,	1236.64it/s]
Checking face images:	94%	??????????	91957/97698	[01:23<00:04,	1241.10it/s]
Checking face images:	94%	??????????	92085/97698	[01:23<00:04,	1250.07it/s]
Checking face images:	94%	??????????	92217/97698	[01:23<00:04,	1268.62it/s]
Checking face images:	95%	??????????	92345/97698	[01:23<00:04,	1269.88it/s]
Checking face images:	95%	??????????	92475/97698	[01:24<00:04,	1277.00it/s]
Checking face images:	95%	??????????	92606/97698	[01:24<00:03,	1283.56it/s]
Checking face images:	95%	??????????	92736/97698	[01:24<00:03,	1287.69it/s]
Checking face images:	95%	??????????	92865/97698	[01:24<00:03,	1232.87it/s]
Checking face images:	95%	??????????	92989/97698	[01:24<00:03,	1228.34it/s]
Checking face images:	95%	??????????	93113/97698	[01:24<00:03,	1193.28it/s]
Checking face images:	95%	??????????	93233/97698	[01:24<00:04,	990.10it/s]
Checking face images:	96%	??????????	93338/97698	[01:24<00:04,	888.95it/s]
Checking face images:	96%	??????????	93433/97698	[01:25<00:05,	827.17it/s]
Checking face images:	96%	??????????	93520/97698	[01:25<00:05,	784.33it/s]
Checking face images:	96%	??????????	93601/97698	[01:25<00:05,	748.94it/s]
Checking face images:	96%	??????????	93678/97698	[01:25<00:05,	723.96it/s]
Checking face images:	96%	??????????	93752/97698	[01:25<00:05,	703.25it/s]
Checking face images:	96%	??????????	93825/97698	[01:25<00:05,	707.41it/s]
Checking face images:	96%	??????????	93897/97698	[01:25<00:05,	690.08it/s]
Checking face images:	96%	??????????	93967/97698	[01:25<00:05,	670.57it/s]
Checking face images:	96%	??????????	94035/97698	[01:25<00:05,	654.29it/s]
Checking face images:	96%	??????????	94102/97698	[01:26<00:05,	657.72it/s]
Checking face images:	96%	??????????	94168/97698	[01:26<00:05,	650.61it/s]
Checking face images:	96%	??????????	94236/97698	[01:26<00:05,	656.25it/s]
Checking face images:	97%	??????????	94306/97698	[01:26<00:05,	668.18it/s]
Checking face images:	97%	??????????	94378/97698	[01:26<00:04,	679.35it/s]
Checking face images:	97%	??????????	94447/97698	[01:26<00:05,	646.79it/s]
Checking face images:	97%	??????????	94513/97698	[01:26<00:04,	643.00it/s]
Checking face images:	97%	??????????	94578/97698	[01:26<00:04,	643.49it/s]
Checking face images:	97%	??????????	94649/97698	[01:26<00:04,	662.04it/s]
Checking face images:	97%	??????????	94720/97698	[01:26<00:04,	673.11it/s]
Checking face images:	97%	??????????	94788/97698	[01:27<00:04,	665.20it/s]
Checking face images:	97%	??????????	94857/97698	[01:27<00:04,	671.63it/s]
Checking face images:	97%	??????????	94945/97698	[01:27<00:03,	732.53it/s]
Checking face images:	97%	??????????	95063/97698	[01:27<00:03,	864.60it/s]
Checking face images:	97%	??????????	95196/97698	[01:27<00:02,	999.52it/s]
Checking face images:	98%	??????????	95326/97698	[01:27<00:02,	1087.27it/s]
Checking face images:	98%	??????????	95436/97698	[01:27<00:02,	1090.80it/s]
Checking face images:	98%	??????????	95560/97698	[01:27<00:01,	1134.79it/s]
Checking face images:	98%	??????????	95691/97698	[01:27<00:01,	1184.50it/s]
Checking face images:	98%	??????????	95823/97698	[01:27<00:01,	1223.48it/s]
Checking face images:	98%	??????????	95955/97698	[01:28<00:01,	1251.66it/s]
Checking face images:	98%	??????????	96089/97698	[01:28<00:01,	1277.04it/s]
Checking face images:	98%	??????????	96220/97698	[01:28<00:01,	1284.42it/s]
Checking face images:	99%	??????????	96349/97698	[01:28<00:01,	1279.45it/s]
Checking face images:	99%	??????????	96477/97698	[01:28<00:00,	1271.98it/s]
Checking face images:	99%	??????????	96609/97698	[01:28<00:00,	1285.60it/s]
Checking face images:	99%	??????????	96738/97698	[01:28<00:00,	1226.57it/s]
Checking face images:	99%	??????????	96864/97698	[01:28<00:00,	1229.86it/s]
Checking face images:	99%	??????????	97000/97698	[01:28<00:00,	1259.50it/s]
Checking face images:	99%	??????????	97131/97698	[01:29<00:00,	1272.91it/s]
Checking face images:	100%	??????????	97264/97698	[01:29<00:00,	1285.89it/s]
Checking face images:	100%	??????????	97401/97698	[01:29<00:00,	1301.50it/s]
Checking face images:	100%	??????????	97540/97698	[01:29<00:00,	1324.93it/s]
Checking face images:	100%	??????????	97673/97698	[01:29<00:00,	1319.51it/s]
Checking face images:	100%	??????????	97698/97698	[01:29<00:00,	1092.37it/s]

Image check saved to: /content/project_edge_ai/results/gender/20260915_030921/image_checks.csv
 Unreadable or invalid images: 0

9. Remove exact repeats

In this cell we remove exact duplicate images and conflicting labels. If the same image appears in both development and holdout data, the holdout copy is kept.

Code cell 18 - recorded

```
# cell:duplicates
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
label_columns = ['age', 'gender']
conflicts = clean_data.groupby("sha256")[label_columns].nunique().max(axis=1)
conflict_hashes = set(conflicts[conflicts > 1].index)
conflicting = clean_data["sha256"].isin(conflict_hashes)
print("Images with conflicting labels:", int(conflicting.sum()))
clean_data = clean_data.loc[~conflicting].copy()
clean_data["priority"] = clean_data["source_split"].map({"holdout": 0, "development": 1})
clean_data = clean_data.sort_values(["priority", "path"])
repeats = clean_data.duplicated("sha256", keep="first")
clean_data.loc[repeats].to_csv(OUTPUT_DIR / "duplicate_images.csv", index=False)
print("Exact repeated images removed:", int(repeats.sum()))
clean_data = clean_data.loc[~repeats].copy()

Images with conflicting labels: 115
Exact repeated images removed: 102
```

10. Keep the same training, validation and holdout splits

In this cell we keep the original FairFace validation set as the final holdout. We take 15% of the development data for validation and keep age and gender groups balanced as well as possible.

Code cell 20 - recorded

```
# cell:split
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
development = clean_data.loc[clean_data["source_split"].eq("development")].copy()
test_df = clean_data.loc[clean_data["source_split"].eq("holdout")].copy()
strata = (
    development["age"].astype(str)
    + "_"
    + development["gender"].astype(str)
)
if strata.value_counts().min() < 2:
    raise ValueError("A group has fewer than two images. Review the data before splitting.")
train_df, val_df = train_test_split(development, test_size=0.15, random_state=SEED,
                                    stratify=strata)
frames = {"train": train_df, "validation": val_df, "holdout": test_df}
for name, frame in frames.items():
    if set(frame["label"]) != set(range(N_CLASSES)):
        raise ValueError(f"The {name} set does not contain every required class.")
    frame.to_csv(OUTPUT_DIR / f"{name}_images.csv", index=False)
for first, second in [(train_df, val_df), (train_df, test_df), (val_df, test_df)]:
    assert set(first["sha256"]).isdisjoint(set(second["sha256"]))
print({name: len(frame) for name, frame in frames.items()})

# Optional smaller sets for a quick Colab pipeline test.
# The final model must use FAST_MODE=False.
if FAST_MODE:
    def quick_subset(frame, per_class):
        parts = []

        for label, group in frame.groupby(
            "label"
        ):
            parts.append(
                group.sample(
                    min(
                        per_class,
                        len(group),
                    ),
                    random_state=(
                        SEED
                        + int(label)
                    ),
                )
            )

        return pd.concat(
            parts,
```

```

        ignore_index=True,
    )

    train_df = quick_subset(
        train_df,
        1500,
    )

    val_df = quick_subset(
        val_df,
        300,
    )

    test_df = quick_subset(
        test_df,
        300,
    )

    frames = {
        "train": train_df,
        "validation": val_df,
        "holdout": test_df,
    }

    print(
        "FAST_MODE subset sizes:",
        {
            name: len(frame)
            for name, frame in frames.items()
        },
    )

{'train': 73555, 'validation': 12981, 'holdout': 10945}

```

11. Count each class

In this cell we count Female and Male images in the training, validation and holdout sets. The final validation and holdout counts are not changed by balanced training.

Code cell 22 - recorded

```

# cell:counts
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
counts = pd.DataFrame({name: frame["label"].value_counts().reindex(range(N_CLASSES), fill_value=0)
                       for name, frame in frames.items()})
counts.index = CLASS_NAMES
display(counts)
counts.to_csv(OUTPUT_DIR / "class_counts.csv")
fig, ax = plt.subplots(figsize=(9, 4))
counts["train"].plot.bar(ax=ax)
ax.set(title="Training images per class", xlabel="Class", ylabel="Images")
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "class_counts.png", dpi=150)
plt.show()

```

	train	validation	holdout
Female	34558	6099	5159
Male	38997	6882	5786



12. Look at a few training images

In this cell we show a few training faces and their FairFace labels. This is a quick check that the image and label pipeline looks correct.

Code cell 24 - recorded

```
# cell:samples
SHOW_FACE_IMAGES = True
if SHOW_FACE_IMAGES:
    for _, row in train_df.sample(3, random_state=SEED).iterrows():
        face = open_face(row["path"], row[BOX_COLUMNS])
        fig, ax = plt.subplots(figsize=(3, 3))
        ax.imshow(face)
        ax.set_title(CLASS_NAMES[int(row["label"])])
        ax.axis("off")
        plt.show()
```

Female



Female



Male



13. Prepare the model input

In this cell we load each face as a 224 by 224 RGB image. Validation and holdout images are not augmented.

Code cell 26 - recorded

```
# cell:input
def load_pixels(path, box):
    path = path.item() if isinstance(path, np.ndarray) else path
    path = path.decode("utf-8") if isinstance(path, bytes) else str(path)
    image = open_face(path, box).resize((IMAGE_SIZE, IMAGE_SIZE), Image.Resampling.BILINEAR)
    return np.asarray(image, dtype=np.float32)

def read_image(path, box, label):
    image = tf.numpy_function(load_pixels, [path, box], tf.float32)
```

```

image.set_shape((IMAGE_SIZE, IMAGE_SIZE, 3))
return image, label

def make_dataset(frame):
    if frame.empty:
        raise ValueError("Cannot make an empty image dataset.")
    dataset = tf.data.Dataset.from_tensor_slices((
        frame["path"].astype(str).to_numpy(),
        frame[BOX_COLUMNS].to_numpy(dtype=np.float32),
        frame["label"].to_numpy(dtype=np.int32),
    ))

    options = tf.data.Options()
    options.experimental_deterministic = True
    dataset = dataset.with_options(options)
    dataset = dataset.map(read_image, num_parallel_calls=tf.data.AUTOTUNE)
    return dataset.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

val_ds = make_dataset(val_df)
test_ds = make_dataset(test_df)
images, labels = next(iter(val_ds))
print("Input shape:", images.shape)
print("Pixel range:", float(tf.reduce_min(images)), "to", float(tf.reduce_max(images)))

Input shape: (32, 224, 224, 3)
Pixel range: 0.0 to 255.0

```

14. Make small changes to training images

In this cell we make small random changes to the training images. We use horizontal flips, brightness and contrast changes. The function is compiled once so it does not create the same TensorFlow warnings for every batch.

Code cell 28 - recorded

```

# cell:augmentation
# This function is compiled once and then reused for every batch.
# It avoids the repeated Keras Random* conversion warnings.

@tf.function(
    input_signature=[
        tf.TensorSpec(
            shape=[None, IMAGE_SIZE, IMAGE_SIZE, 3],
            dtype=tf.float32,
        )
    ],
    reduce_retracing=True,
)
def augment_batch(images):
    images = tf.image.random_flip_left_right(
        images,
        seed=SEED,
    )
    images = tf.image.random_brightness(
        images,
        max_delta=10.0,
        seed=SEED + 1,
    )
    images = tf.image.random_contrast(
        images,
        lower=0.90,
        upper=1.10,
        seed=SEED + 2,
    )
    return tf.clip_by_value(images, 0.0, 255.0)

print(
    "Training augmentation ready: horizontal flip, "
    "brightness and contrast."
)

```

```
)
print(
    "The augmentation is compiled once, so repeated "
    "Random* conversion warnings should not appear."
)
```

Training augmentation ready: horizontal flip, brightness and contrast.
 The augmentation is compiled once, so repeated Random* conversion warnings should not appear.

15. Make balanced training batches

In this cell every training batch contains the same number of Female and Male faces. The smaller class is reused with augmentation when needed. Validation and holdout data are not oversampled.

Code cell 30 - recorded

```
# cell:weights
def balanced_epoch_indices(labels, batch_size, seed):
    """Make exact half/half batches; visit each class in shuffled cycles."""
    labels = np.asarray(labels, dtype=np.int32)
    if batch_size < 2 or batch_size % 2:
        raise ValueError("Batch size must be positive and even.")
    if set(np.unique(labels)) != {0, 1}:
        raise ValueError("Both gender classes must be present.")
    half = batch_size // 2
    members = [np.flatnonzero(labels == class_id) for class_id in (0, 1)]
    steps = int(np.ceil(max(map(len, members)) / half))
    needed = steps * half
    rng = np.random.default_rng(seed)
    selections = []
    for indices in members:
        cycles = [rng.permutation(indices) for _ in range(int(np.ceil(needed / len(indices))))]
        selections.append(np.concatenate(cycles)[:needed].reshape(steps, half))
    batches = np.concatenate(selections, axis=1)
    for row in batches:
        rng.shuffle(row)
    return batches

class BalancedGenderBatches(tf.keras.utils.Sequence):
    def __init__(self, frame, batch_size, seed, use_augmentation=True):
        self.frame = frame.reset_index(drop=True).copy()
        self.batch_size, self.seed = batch_size, seed
        self.use_augmentation = bool(use_augmentation)
        self.epoch = 0
        self.labels = self.frame["label"].to_numpy(dtype=np.int32)
        self.paths = self.frame["path"].astype(str).to_numpy()
        self.bboxes = self.frame[BBOX_COLUMNS].to_numpy(dtype=np.float32)
        self.indices = balanced_epoch_indices(self.labels, batch_size, seed)

    def __len__(self):
        return len(self.indices)

    def __getitem__(self, index):
        positions = self.indices[index]
        images = np.stack(
            [
                load_pixels(self.paths[i], self.bboxes[i])
                for i in positions
            ]
        ).astype(np.float32)

        if self.use_augmentation:
            images = augment_batch(
                tf.convert_to_tensor(
                    images,
                    dtype=tf.float32,
                )
            ).numpy()

        return images, self.labels[positions]
```



```

def on_epoch_end(self):
    self.epoch += 1
    self.indices = balanced_epoch_indices(self.labels, self.batch_size, self.seed+self.epoch)

train_ds = BalancedGenderBatches(train_df, BATCH_SIZE, SEED, use_augmentation=True)
CLASS_WEIGHTS = None
STEPS_PER_EPOCH = len(train_ds)
# Check all planned batches, not just one example.
planned_labels = train_ds.labels[train_ds.indices]

assert np.all(np.sum(planned_labels == 0, axis=1) == BATCH_SIZE//2)
assert np.all(np.sum(planned_labels == 1, axis=1) == BATCH_SIZE//2)
assert set(train_ds.indices.ravel()) == set(range(len(train_df)))
exposure = pd.DataFrame({
    "class": CLASS_NAMES,
    "unique_training_images": [int((train_ds.labels == i).sum()) for i in (0, 1)],
    "samples_per_epoch": [STEPS_PER_EPOCH*(BATCH_SIZE//2)]*2,
})
exposure["average_uses_per_epoch"] = exposure["samples_per_epoch"] /
exposure["unique_training_images"]
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
exposure.to_csv(OUTPUT_DIR / "balanced_training_exposure.csv", index=False)
display(exposure)
print("Every batch:", BATCH_SIZE//2, "Female +", BATCH_SIZE//2, "Male")
print("Batches per epoch:", STEPS_PER_EPOCH)
print("Class weights: off. Validation and holdout counts are unchanged.")

   class  unique_training_images  samples_per_epoch  average_uses_per_epoch
0  Female                34558             39008             1.128769
1   Male                 38997             39008             1.000282

Every batch: 16 Female + 16 Male
Batches per epoch: 2438
Class weights: off. Validation and holdout counts are unchanged.

```

16. Build the model

In this cell we build MobileNetV3Small with ImageNet weights. We add a new output layer with two choices, Female and Male. The feature layers stay frozen at first.

Code cell 32 - recorded

```

# cell:model
if not globals().get(
    "GPU_READY",
    False,
):
    raise RuntimeError(
        "Pass the GPU test before building the model."
    )

def build_gender_model(
    weights="imagenet",
):
    backbone = tf.keras.applications.MobileNetV3Small(
        input_shape=(
            IMAGE_SIZE,
            IMAGE_SIZE,
            3,
        ),
        include_top=False,
        weights=weights,
        include_preprocessing=True,
    )

    backbone.trainable = False

    inputs = tf.keras.Input(
        shape=(

```



```

        return dict(super().get_config(), class_id=self.class_id)

def compile_model(network, learning_rate):
    network.compile(
        optimizer=tf.keras.optimizers.AdamW(
            learning_rate=learning_rate,
            weight_decay=1e-4,
        ),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy", GenderRecall(0, name="recall_female"),
                 GenderRecall(1, name="recall_male"), GenderRecall(name="balanced_accuracy")],
        jit_compile=False,
    )

class PrintGenderRecall(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs=None):
        values = logs or {}
        print(
            f"Validation - Female recall: {values.get('val_recall_female', float('nan')):.2%} | "
            f"Male recall: {values.get('val_recall_male', float('nan')):.2%} | "
            f"Balanced accuracy: {values.get('val_balanced_accuracy', float('nan')):.2%}",
            flush=True,
        )

def callbacks(stage):
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    return [
        tf.keras.callbacks.ModelCheckpoint(
            str(OUTPUT_DIR / f"best_{stage}.weights.h5"), monitor="val_balanced_accuracy",
            mode="max", save_best_only=True, save_weights_only=True),
        tf.keras.callbacks.EarlyStopping(monitor="val_balanced_accuracy", mode="max",
                                         patience=5, restore_best_weights=True),
        tf.keras.callbacks.ReduceLROnPlateau(monitor="val_balanced_accuracy", mode="max",
                                              patience=2, factor=0.5, min_lr=1e-7),
        tf.keras.callbacks.CSVLogger(str(OUTPUT_DIR / f"{stage}_training.csv")),
        PrintGenderRecall(), tf.keras.callbacks.TerminateOnNaN(),
    ]

compile_model(model, 1e-3)
print("Each epoch reports train and validation recall for both classes.")
print("Checkpoint files save model weights only; optimizer-state HDF5 warnings are avoided.")

Each epoch reports train and validation recall for both classes.
Checkpoint files save model weights only; optimizer-state HDF5 warnings are avoided.

```

18. Train the new output layer

In this cell we train the new Female/Male output layer while the MobileNetV3 feature extractor stays frozen.

Code cell 36 - recorded

```

# cell:train_head
if not globals().get("GPU_READY", False):
    raise RuntimeError("The GPU test must pass before training.")

OUTPUT_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

started = time.perf_counter()

head_history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=HEAD_EPOCHS,
    callbacks=callbacks("head"),
    verbose=2,
)

```

```

head_seconds = time.perf_counter() - started

print(
    "Training time, minutes:",
    round(head_seconds / 60, 2),
)

/usr/local/lib/python3.13/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121
: UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`, `max_queue_size`. Do not pass these
arguments to `fit()`, as they will be ignored.
    self._warn_if_super_not_called()

Epoch 1/8
Validation - Female recall: 74.16% | Male recall: 78.19% | Balanced accuracy: 76.17%
2438/2438 - 249s - 102ms/step - accuracy: 0.7197 - balanced_accuracy: 0.7197 - loss: 0.5474 -
recall_female: 0.7124 - recall_male: 0.7271 - val_accuracy: 0.7630 - val_balanced_accuracy: 0.7617
- val_loss: 0.4870 - val_recall_female: 0.7416 - val_recall_male: 0.7819 - learning_rate: 0.0010
Epoch 2/8
Validation - Female recall: 73.68% | Male recall: 78.48% | Balanced accuracy: 76.08%
2438/2438 - 199s - 82ms/step - accuracy: 0.7345 - balanced_accuracy: 0.7345 - loss: 0.5292 -
recall_female: 0.7302 - recall_male: 0.7388 - val_accuracy: 0.7623 - val_balanced_accuracy: 0.7608
- val_loss: 0.4844 - val_recall_female: 0.7368 - val_recall_male: 0.7848 - learning_rate: 0.0010
Epoch 3/8
Validation - Female recall: 70.37% | Male recall: 82.16% | Balanced accuracy: 76.26%
2438/2438 - 190s - 78ms/step - accuracy: 0.7307 - balanced_accuracy: 0.7307 - loss: 0.5340 -
recall_female: 0.7243 - recall_male: 0.7371 - val_accuracy: 0.7662 - val_balanced_accuracy: 0.7626
- val_loss: 0.4862 - val_recall_female: 0.7037 - val_recall_male: 0.8216 - learning_rate: 0.0010
Epoch 4/8
Validation - Female recall: 70.08% | Male recall: 82.01% | Balanced accuracy: 76.04%
2438/2438 - 196s - 80ms/step - accuracy: 0.7333 - balanced_accuracy: 0.7333 - loss: 0.5316 -
recall_female: 0.7272 - recall_male: 0.7394 - val_accuracy: 0.7640 - val_balanced_accuracy: 0.7604
- val_loss: 0.4860 - val_recall_female: 0.7008 - val_recall_male: 0.8201 - learning_rate: 0.0010
Epoch 5/8
Validation - Female recall: 73.70% | Male recall: 79.16% | Balanced accuracy: 76.43%
2438/2438 - 193s - 79ms/step - accuracy: 0.7343 - balanced_accuracy: 0.7343 - loss: 0.5310 -
recall_female: 0.7292 - recall_male: 0.7394 - val_accuracy: 0.7660 - val_balanced_accuracy: 0.7643
- val_loss: 0.4839 - val_recall_female: 0.7370 - val_recall_male: 0.7916 - learning_rate: 0.0010
Epoch 6/8
Validation - Female recall: 76.05% | Male recall: 76.36% | Balanced accuracy: 76.20%
2438/2438 - 212s - 87ms/step - accuracy: 0.7329 - balanced_accuracy: 0.7329 - loss: 0.5316 -
recall_female: 0.7274 - recall_male: 0.7384 - val_accuracy: 0.7621 - val_balanced_accuracy: 0.7620
- val_loss: 0.4872 - val_recall_female: 0.7605 - val_recall_male: 0.7636 - learning_rate: 0.0010
Epoch 7/8
Validation - Female recall: 71.04% | Male recall: 81.23% | Balanced accuracy: 76.14%
2438/2438 - 199s - 82ms/step - accuracy: 0.7314 - balanced_accuracy: 0.7314 - loss: 0.5317 -
recall_female: 0.7264 - recall_male: 0.7364 - val_accuracy: 0.7644 - val_balanced_accuracy: 0.7614
- val_loss: 0.4874 - val_recall_female: 0.7104 - val_recall_male: 0.8123 - learning_rate: 0.0010
Epoch 8/8
Validation - Female recall: 78.11% | Male recall: 75.07% | Balanced accuracy: 76.59%
2438/2438 - 192s - 79ms/step - accuracy: 0.7320 - balanced_accuracy: 0.7320 - loss: 0.5282 -
recall_female: 0.7261 - recall_male: 0.7380 - val_accuracy: 0.7650 - val_balanced_accuracy: 0.7659
- val_loss: 0.4873 - val_recall_female: 0.7811 - val_recall_male: 0.7507 - learning_rate:
5.0000e-04
Training time, minutes: 27.19

```

19. Prepare fine-tuning

In this cell we load the best first-stage weights and unlock the last part of MobileNetV3. Batch-normalisation layers stay frozen for a more stable fine-tuning stage.

Code cell 38 - recorded

```

# cell:unfreeze
def set_feature_stage(stage):
    features = model.get_layer(BACKBONE_NAME)
    if stage == "head":
        features.trainable = False
    elif stage == "fine":
        features.trainable = True

```

```

        cutoff = int(len(features.layers)*2/3)
        for index, layer in enumerate(features.layers):
            layer.trainable = index >= cutoff and not isinstance(layer,
tf.keras.layers.BatchNormalization)
        else:
            raise ValueError("Unknown training stage.")
        return features

# Restore the checkpoint with the same trainable flags used when it was saved.
set_feature_stage("head")
model.load_weights(str(OUTPUT_DIR / "best_head.weights.h5"))
backbone = set_feature_stage("fine")
compile_model(model, 1e-5)
print("Trainable feature layers:", sum(layer.trainable for layer in backbone.layers))

Trainable feature layers: 42

```

20. Fine-tune the model

In this cell we train again with a smaller learning rate. The output layer and selected feature layers learn together.

Code cell 40 - recorded

```

# cell:train_fine
if not globals().get("GPU_READY", False):
    raise RuntimeError("The GPU test must pass before training.")

OUTPUT_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

started = time.perf_counter()

fine_history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=FINE_TUNE_EPOCHS,
    callbacks=callbacks("fine"),
    verbose=2,
)

fine_seconds = time.perf_counter() - started

print(
    "Fine-tuning time, minutes:",
    round(fine_seconds / 60, 2),
)

Epoch 1/20
Validation - Female recall: 82.72% | Male recall: 81.68% | Balanced accuracy: 82.20%
2438/2438 - 216s - 89ms/step - accuracy: 0.7771 - balanced_accuracy: 0.7771 - loss: 0.4577 -
recall_female: 0.7727 - recall_male: 0.7815 - val_accuracy: 0.8217 - val_balanced_accuracy: 0.8220
- val_loss: 0.3852 - val_recall_female: 0.8272 - val_recall_male: 0.8168 - learning_rate:
1.0000e-05
Epoch 2/20
Validation - Female recall: 83.19% | Male recall: 85.32% | Balanced accuracy: 84.26%
2438/2438 - 191s - 78ms/step - accuracy: 0.8213 - balanced_accuracy: 0.8213 - loss: 0.3831 -
recall_female: 0.8180 - recall_male: 0.8247 - val_accuracy: 0.8432 - val_balanced_accuracy: 0.8426
- val_loss: 0.3440 - val_recall_female: 0.8319 - val_recall_male: 0.8532 - learning_rate:
1.0000e-05
Epoch 3/20
Validation - Female recall: 85.54% | Male recall: 85.18% | Balanced accuracy: 85.36%
2438/2438 - 207s - 85ms/step - accuracy: 0.8443 - balanced_accuracy: 0.8443 - loss: 0.3444 -
recall_female: 0.8426 - recall_male: 0.8460 - val_accuracy: 0.8535 - val_balanced_accuracy: 0.8536
- val_loss: 0.3219 - val_recall_female: 0.8554 - val_recall_male: 0.8518 - learning_rate:
1.0000e-05
Epoch 4/20
Validation - Female recall: 86.78% | Male recall: 85.25% | Balanced accuracy: 86.02%
2438/2438 - 207s - 85ms/step - accuracy: 0.8557 - balanced_accuracy: 0.8557 - loss: 0.3181 -
recall_female: 0.8542 - recall_male: 0.8572 - val_accuracy: 0.8597 - val_balanced_accuracy: 0.8602

```

- val_loss: 0.3063 - val_recall_female: 0.8678 - val_recall_male: 0.8525 - learning_rate: 1.0000e-05

Epoch 5/20

Validation - Female recall: 87.39% | Male recall: 85.95% | Balanced accuracy: 86.67%

2438/2438 - 206s - 84ms/step - accuracy: 0.8664 - balanced_accuracy: 0.8664 - loss: 0.2994 - recall_female: 0.8674 - recall_male: 0.8655 - val_accuracy: 0.8663 - val_balanced_accuracy: 0.8667 - val_loss: 0.2950 - val_recall_female: 0.8739 - val_recall_male: 0.8595 - learning_rate: 1.0000e-05

Epoch 6/20

Validation - Female recall: 86.69% | Male recall: 87.36% | Balanced accuracy: 87.02%

2438/2438 - 217s - 89ms/step - accuracy: 0.8732 - balanced_accuracy: 0.8732 - loss: 0.2854 - recall_female: 0.8745 - recall_male: 0.8720 - val_accuracy: 0.8704 - val_balanced_accuracy: 0.8702 - val_loss: 0.2856 - val_recall_female: 0.8669 - val_recall_male: 0.8736 - learning_rate: 1.0000e-05

Epoch 7/20

Validation - Female recall: 87.10% | Male recall: 87.66% | Balanced accuracy: 87.38%

2438/2438 - 197s - 81ms/step - accuracy: 0.8793 - balanced_accuracy: 0.8793 - loss: 0.2719 - recall_female: 0.8821 - recall_male: 0.8765 - val_accuracy: 0.8740 - val_balanced_accuracy: 0.8738 - val_loss: 0.2789 - val_recall_female: 0.8710 - val_recall_male: 0.8766 - learning_rate: 1.0000e-05

Epoch 8/20

Validation - Female recall: 87.85% | Male recall: 87.72% | Balanced accuracy: 87.79%

2438/2438 - 193s - 79ms/step - accuracy: 0.8859 - balanced_accuracy: 0.8859 - loss: 0.2611 - recall_female: 0.8882 - recall_male: 0.8836 - val_accuracy: 0.8778 - val_balanced_accuracy: 0.8779 - val_loss: 0.2730 - val_recall_female: 0.8785 - val_recall_male: 0.8772 - learning_rate: 1.0000e-05

Epoch 9/20

Validation - Female recall: 88.75% | Male recall: 87.36% | Balanced accuracy: 88.06%

2438/2438 - 206s - 84ms/step - accuracy: 0.8904 - balanced_accuracy: 0.8904 - loss: 0.2509 - recall_female: 0.8933 - recall_male: 0.8875 - val_accuracy: 0.8801 - val_balanced_accuracy: 0.8806 - val_loss: 0.2683 - val_recall_female: 0.8875 - val_recall_male: 0.8736 - learning_rate: 1.0000e-05

Epoch 10/20

Validation - Female recall: 88.41% | Male recall: 88.14% | Balanced accuracy: 88.28%

2438/2438 - 198s - 81ms/step - accuracy: 0.8942 - balanced_accuracy: 0.8942 - loss: 0.2424 - recall_female: 0.8972 - recall_male: 0.8912 - val_accuracy: 0.8827 - val_balanced_accuracy: 0.8828 - val_loss: 0.2645 - val_recall_female: 0.8841 - val_recall_male: 0.8814 - learning_rate: 1.0000e-05

Epoch 11/20

Validation - Female recall: 90.18% | Male recall: 86.33% | Balanced accuracy: 88.25%

2438/2438 - 237s - 97ms/step - accuracy: 0.8981 - balanced_accuracy: 0.8981 - loss: 0.2356 - recall_female: 0.9019 - recall_male: 0.8944 - val_accuracy: 0.8814 - val_balanced_accuracy: 0.8825 - val_loss: 0.2656 - val_recall_female: 0.9018 - val_recall_male: 0.8633 - learning_rate: 1.0000e-05

Epoch 12/20

Validation - Female recall: 90.44% | Male recall: 86.46% | Balanced accuracy: 88.45%

2438/2438 - 212s - 87ms/step - accuracy: 0.9019 - balanced_accuracy: 0.9019 - loss: 0.2265 - recall_female: 0.9041 - recall_male: 0.8996 - val_accuracy: 0.8833 - val_balanced_accuracy: 0.8845 - val_loss: 0.2622 - val_recall_female: 0.9044 - val_recall_male: 0.8646 - learning_rate: 1.0000e-05

Epoch 13/20

Validation - Female recall: 88.23% | Male recall: 89.00% | Balanced accuracy: 88.61%

2438/2438 - 189s - 77ms/step - accuracy: 0.9049 - balanced_accuracy: 0.9049 - loss: 0.2201 - recall_female: 0.9086 - recall_male: 0.9011 - val_accuracy: 0.8864 - val_balanced_accuracy: 0.8861 - val_loss: 0.2583 - val_recall_female: 0.8823 - val_recall_male: 0.8900 - learning_rate: 1.0000e-05

Epoch 14/20

Validation - Female recall: 88.64% | Male recall: 88.42% | Balanced accuracy: 88.53%

2438/2438 - 205s - 84ms/step - accuracy: 0.9078 - balanced_accuracy: 0.9078 - loss: 0.2132 - recall_female: 0.9115 - recall_male: 0.9040 - val_accuracy: 0.8852 - val_balanced_accuracy: 0.8853 - val_loss: 0.2583 - val_recall_female: 0.8864 - val_recall_male: 0.8842 - learning_rate: 1.0000e-05

Epoch 15/20

Validation - Female recall: 90.79% | Male recall: 86.68% | Balanced accuracy: 88.73%

2438/2438 - 198s - 81ms/step - accuracy: 0.9112 - balanced_accuracy: 0.9112 - loss: 0.2071 - recall_female: 0.9147 - recall_male: 0.9078 - val_accuracy: 0.8861 - val_balanced_accuracy: 0.8873 - val_loss: 0.2558 - val_recall_female: 0.9079 - val_recall_male: 0.8668 - learning_rate: 1.0000e-05

Epoch 16/20

```

Validation - Female recall: 88.00% | Male recall: 89.57% | Balanced accuracy: 88.78%
2438/2438 - 203s - 83ms/step - accuracy: 0.9132 - balanced_accuracy: 0.9132 - loss: 0.2017 -
recall_female: 0.9170 - recall_male: 0.9093 - val_accuracy: 0.8883 - val_balanced_accuracy: 0.8878
- val_loss: 0.2569 - val_recall_female: 0.8800 - val_recall_male: 0.8957 - learning_rate:
1.0000e-05
Epoch 17/20
Validation - Female recall: 88.11% | Male recall: 89.97% | Balanced accuracy: 89.04%
2438/2438 - 301s - 124ms/step - accuracy: 0.9150 - balanced_accuracy: 0.9150 - loss: 0.1968 -
recall_female: 0.9195 - recall_male: 0.9105 - val_accuracy: 0.8910 - val_balanced_accuracy: 0.8904
- val_loss: 0.2522 - val_recall_female: 0.8811 - val_recall_male: 0.8997 - learning_rate:
1.0000e-05
Epoch 18/20
Validation - Female recall: 89.72% | Male recall: 88.38% | Balanced accuracy: 89.05%
2438/2438 - 213s - 87ms/step - accuracy: 0.9198 - balanced_accuracy: 0.9198 - loss: 0.1901 -
recall_female: 0.9238 - recall_male: 0.9158 - val_accuracy: 0.8901 - val_balanced_accuracy: 0.8905
- val_loss: 0.2545 - val_recall_female: 0.8972 - val_recall_male: 0.8838 - learning_rate:
1.0000e-05
Epoch 19/20
Validation - Female recall: 90.92% | Male recall: 87.04% | Balanced accuracy: 88.98%
2438/2438 - 186s - 76ms/step - accuracy: 0.9206 - balanced_accuracy: 0.9206 - loss: 0.1847 -
recall_female: 0.9241 - recall_male: 0.9170 - val_accuracy: 0.8886 - val_balanced_accuracy: 0.8898
- val_loss: 0.2538 - val_recall_female: 0.9092 - val_recall_male: 0.8704 - learning_rate:
1.0000e-05
Epoch 20/20
Validation - Female recall: 90.54% | Male recall: 87.59% | Balanced accuracy: 89.07%
2438/2438 - 202s - 83ms/step - accuracy: 0.9240 - balanced_accuracy: 0.9240 - loss: 0.1786 -
recall_female: 0.9285 - recall_male: 0.9194 - val_accuracy: 0.8898 - val_balanced_accuracy: 0.8907
- val_loss: 0.2536 - val_recall_female: 0.9054 - val_recall_male: 0.8759 - learning_rate:
5.0000e-06
Fine-tuning time, minutes: 69.76

```

21. Keep the best model

In this cell we compare the best first-stage and fine-tuned checkpoints on validation data. Validation balanced accuracy chooses the model, with validation loss used only to break a tie.

Code cell 42 - recorded

```

# cell:select
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
choices = []
for stage in ("head", "fine"):
    set_feature_stage(stage)
    model.load_weights(str(OUTPUT_DIR / f"best_{stage}.weights.h5"))
    compile_model(model, 1e-5)
    result = model.evaluate(val_ds, verbose=0, return_dict=True)
    choices.append({"stage": stage, "validation_loss": float(result["loss"]),
                    "validation_accuracy": float(result["accuracy"]),
                    "validation_recall_female": float(result["recall_female"]),
                    "validation_recall_male": float(result["recall_male"]),
                    "validation_balanced_accuracy": float(result["balanced_accuracy"])})
selection = pd.DataFrame(choices).sort_values(
    ["validation_balanced_accuracy", "validation_loss"], ascending=[False, True])
display(selection)
selection.to_csv(OUTPUT_DIR / "checkpoint_selection.csv", index=False)
BEST_STAGE = str(selection.iloc[0]["stage"])
set_feature_stage(BEST_STAGE)
model.load_weights(str(OUTPUT_DIR / f"best_{BEST_STAGE}.weights.h5"))
model.save(OUTPUT_DIR / "best_model.h5", include_optimizer=False)
print("Chosen stage:", BEST_STAGE)

/usr/local/lib/python3.13/dist-packages/keras/src/saving/saving_lib.py:797: UserWarning: Skipping
variable loading for optimizer 'loss_scale_optimizer', because it has 56 variables whereas the
saved optimizer has 10 variables.
    saveable.load_own_variables(weights_store.get(inner_path))
/usr/local/lib/python3.13/dist-packages/keras/src/saving/saving_lib.py:797: UserWarning: Skipping
variable loading for optimizer 'adamw', because it has 52 variables whereas the saved optimizer
has 6 variables.
    saveable.load_own_variables(weights_store.get(inner_path))
/usr/local/lib/python3.13/dist-packages/keras/src/saving/saving_lib.py:797: UserWarning: Skipping

```



```

variable loading for optimizer 'loss_scale_optimizer', because it has 4 variables whereas the
saved optimizer has 56 variables.
    saveable.load_own_variables(weights_store.get(inner_path))
/usr/local/lib/python3.13/dist-packages/keras/src/saving/saving_lib.py:797: UserWarning: Skipping
variable loading for optimizer 'adamw', because it has 2 variables whereas the saved optimizer has
52 variables.
    saveable.load_own_variables(weights_store.get(inner_path))

stage validation_loss validation_accuracy validation_recall_female validation_recall_male \
1 fine 0.253569 0.889762 0.905394 0.875908
0 head 0.487304 0.764964 0.781112 0.750654

validation_balanced_accuracy
1 0.890651
0 0.765883

/usr/local/lib/python3.13/dist-packages/keras/src/saving/saving_lib.py:797: UserWarning: Skipping
variable loading for optimizer 'loss_scale_optimizer', because it has 4 variables whereas the
saved optimizer has 56 variables.
    saveable.load_own_variables(weights_store.get(inner_path))
/usr/local/lib/python3.13/dist-packages/keras/src/saving/saving_lib.py:797: UserWarning: Skipping
variable loading for optimizer 'adamw', because it has 2 variables whereas the saved optimizer has
52 variables.
    saveable.load_own_variables(weights_store.get(inner_path))

Chosen stage: fine

```

22. Plot training, recall and validation results

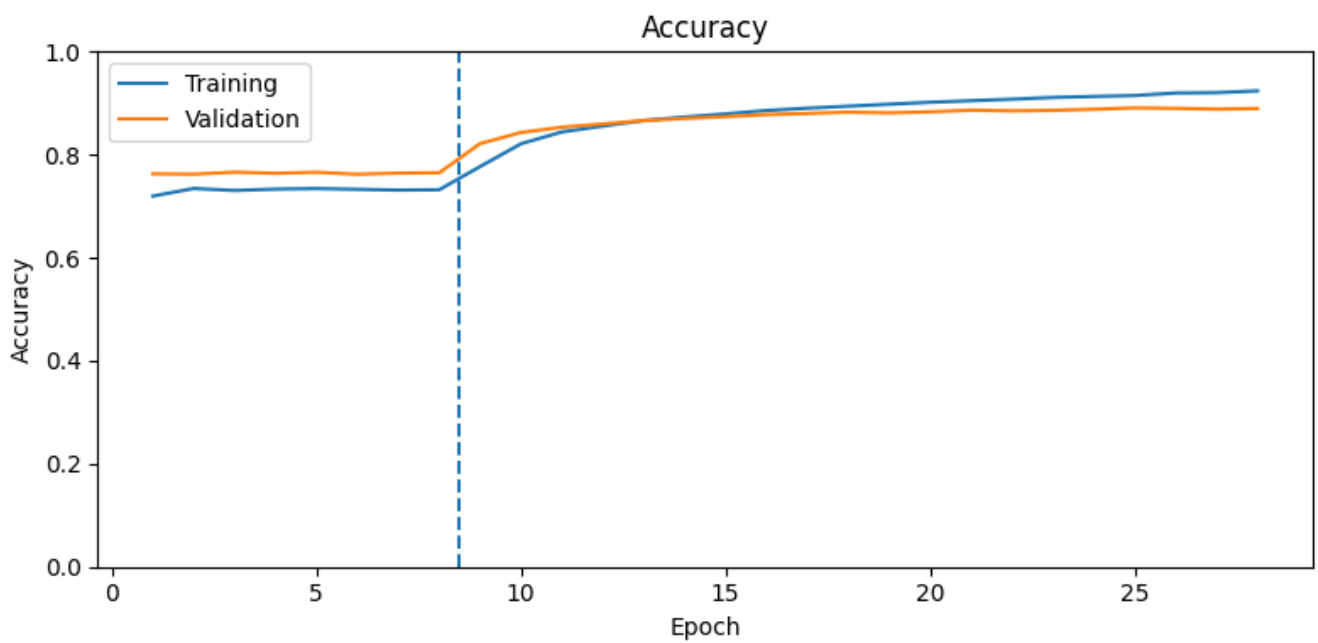
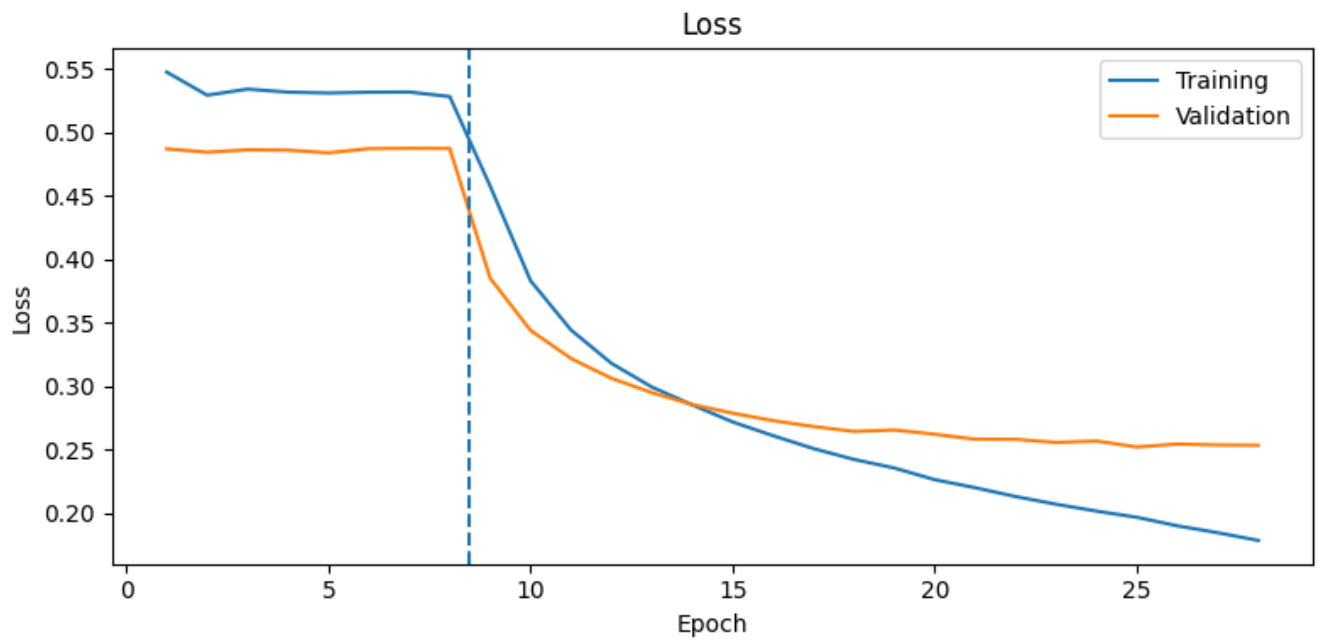
In this cell we plot loss, accuracy, Female recall, Male recall and balanced accuracy across both training stages.

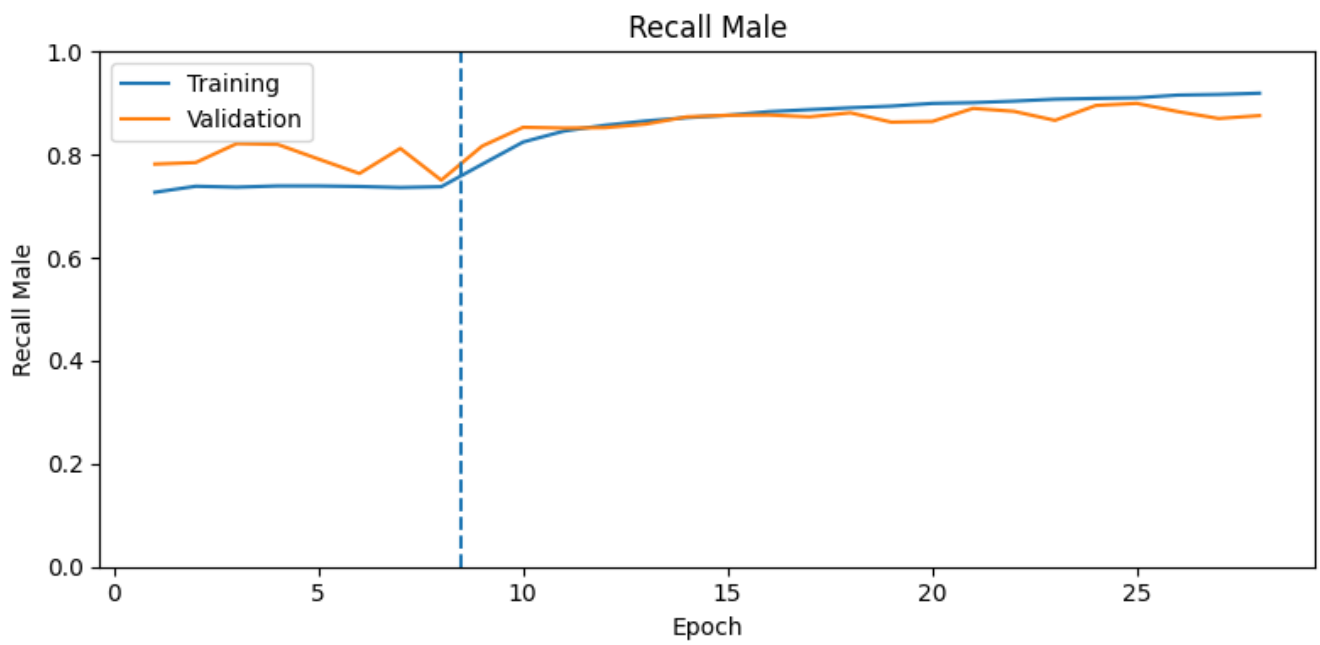
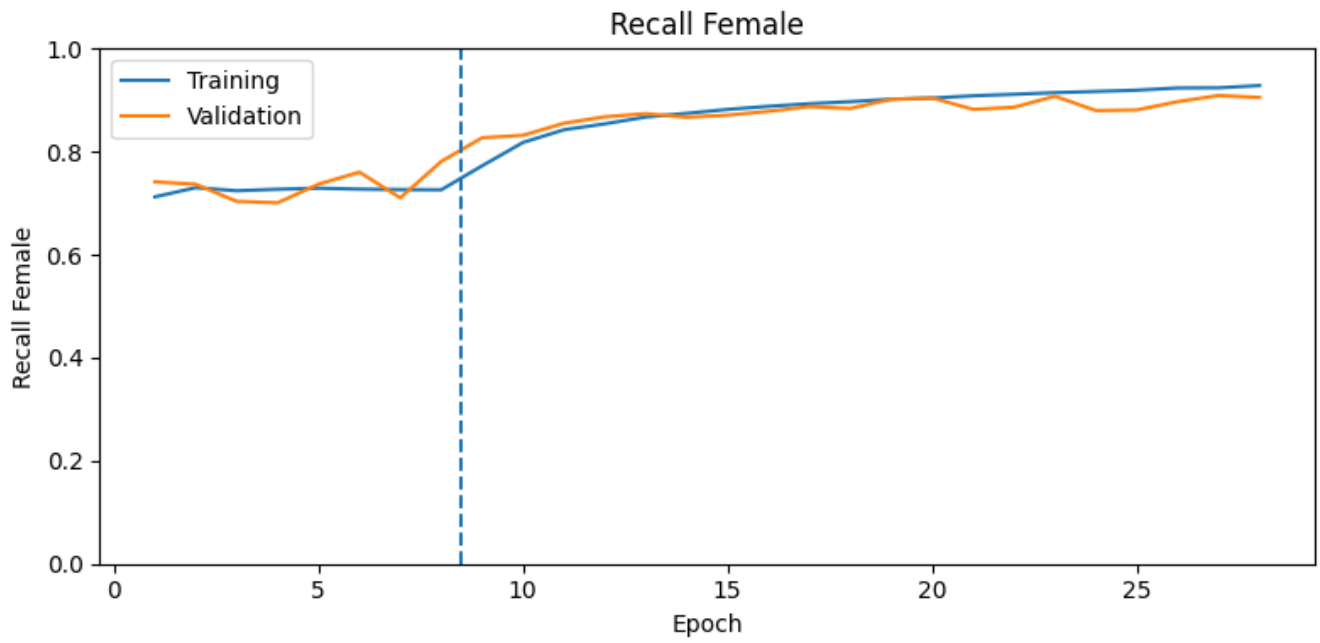
Code cell 44 - recorded

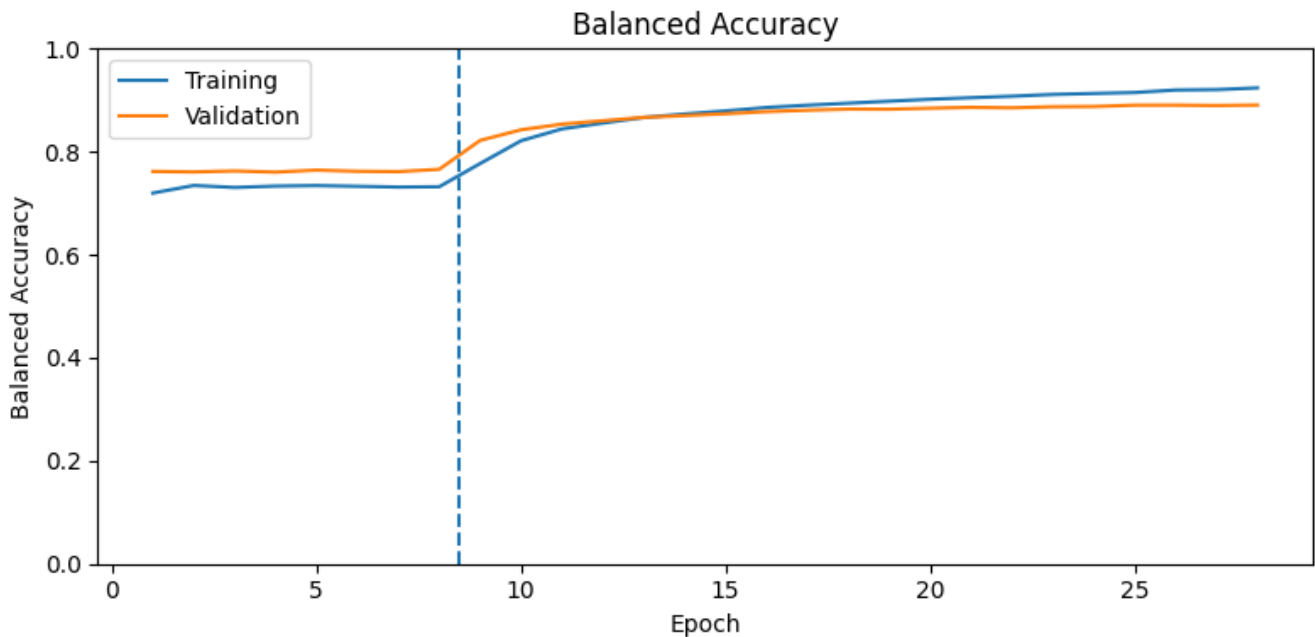
```

# cell:curves
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
history = pd.concat([pd.read_csv(OUTPUT_DIR / "head_training.csv"),
                    pd.read_csv(OUTPUT_DIR / "fine_training.csv")], ignore_index=True)
history.to_csv(OUTPUT_DIR / "training_history.csv", index=False)
for metric in ("loss", "accuracy", "recall_female", "recall_male", "balanced_accuracy"):
    fig, ax = plt.subplots(figsize=(8, 4))
    ax.plot(np.arange(1, len(history)+1), history[metric], label="Training")
    ax.plot(np.arange(1, len(history)+1), history["val_"+metric], label="Validation")
    ax.axvline(len(head_history.history["loss"])+0.5, linestyle="--")
    title = metric.replace("_", " ").title()
    ax.set(xlabel="Epoch", ylabel=title, title=title)
    if metric != "loss":
        ax.set_ylim(0, 1)
    ax.legend()
    fig.tight_layout()
    fig.savefig(OUTPUT_DIR / f"training_{metric}.png", dpi=150)
    plt.show()
    plt.close(fig)

```







23. Evaluate on the existing holdout

In this cell we test the chosen model on the untouched FairFace holdout. We report accuracy, balanced accuracy and macro F1. The holdout does not choose the checkpoint or mobile format.

Code cell 46 - recorded

```
# cell:evaluate
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
probabilities = model.predict(test_ds, verbose=0)
if not np.isfinite(probabilities).all():
    raise ValueError("The model produced invalid scores.")
y_true = test_df["label"].to_numpy()
y_pred = probabilities.argmax(axis=1)
majority_class = int(train_df["label"].value_counts().idxmax())
baseline_pred = np.full(len(y_true), majority_class)

def score_summary(truth, prediction):
    return {"accuracy": float(accuracy_score(truth, prediction)),
            "balanced_accuracy": float(balanced_accuracy_score(truth, prediction)),
            "macro_f1": float(f1_score(truth, prediction, average="macro",
                                         labels=list(range(N_CLASSES)), zero_division=0))}

model_scores = score_summary(y_true, y_pred)
results = pd.DataFrame([{"model": "Majority-class baseline", **score_summary(y_true,
baseline_pred)},
                        {"model": TASK + " model", **model_scores}])
display(results)
results.to_csv(OUTPUT_DIR / "holdout_results.csv", index=False)

predictions = test_df.copy()
predictions["true_class"] = [CLASS_NAMES[value] for value in y_true]
predictions["predicted_class"] = [CLASS_NAMES[value] for value in y_pred]
predictions["model_score"] = probabilities.max(axis=1)
predictions["correct"] = y_true == y_pred
predictions.to_csv(OUTPUT_DIR / "holdout_predictions.csv", index=False)

      model  accuracy  balanced_accuracy  macro_f1
0  Majority-class baseline  0.528643          0.500000  0.345825
1           gender model  0.894838          0.895379  0.894653
```

24. Check Female and Male separately

In this cell we report precision, recall and F1 for Female and Male. The confusion matrix shows the real label in each row and the model prediction in each column.

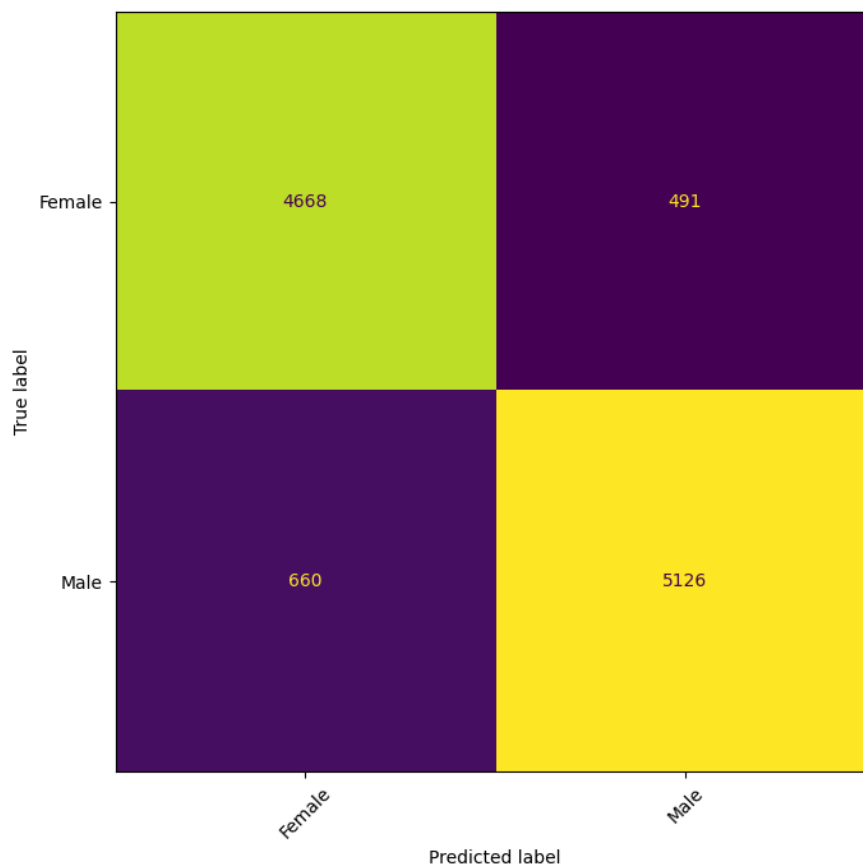
Code cell 48 - recorded

```
# cell:class_report
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
report = pd.DataFrame(classification_report(y_true, y_pred, labels=list(range(N_CLASSES)),
                                          target_names=CLASS_NAMES, output_dict=True, zero_division=0)).T

display(report)
report.to_csv(OUTPUT_DIR / "classification_report.csv")
cm = confusion_matrix(y_true, y_pred, labels=list(range(N_CLASSES)))
pd.DataFrame(cm, index=CLASS_NAMES, columns=CLASS_NAMES).to_csv(OUTPUT_DIR /
"confusion_matrix.csv")
fig, ax = plt.subplots(figsize=(8, 7))
ConfusionMatrixDisplay(cm, display_labels=CLASS_NAMES).plot(ax=ax, colorbar=False,
                                                            xticks_rotation=45)

fig.tight_layout()
fig.savefig(OUTPUT_DIR / "confusion_matrix.png", dpi=150)
plt.show()
```

	precision	recall	f1-score	support
Female	0.876126	0.904827	0.890245	5159.000000
Male	0.912587	0.885932	0.899062	5786.000000
accuracy	0.894838	0.894838	0.894838	0.894838
macro avg	0.894356	0.895379	0.894653	10945.000000
weighted avg	0.895401	0.894838	0.894906	10945.000000



25. Check gender performance across age groups

In this cell we check the model inside each original FairFace age range. We report Female recall and Male recall separately so one age group cannot hide a weak class.

Code cell 50 - recorded

```
# cell:task_check
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

age_rows = []

age_order = []
```

```

    "0-2",
    "3-9",
    "10-19",
    "20-29",
    "30-39",
    "40-49",
    "50-59",
    "60-69",
    "more than 70",
]

for age_range in age_order:
    mask = test_df["age"].eq(age_range).to_numpy()

    if not mask.any():
        continue

    truth = y_true[mask]
    pred = y_pred[mask]

    female_mask = truth == 0
    male_mask = truth == 1

    female_recall = (
        float(np.mean(pred[female_mask] == 0))
        if female_mask.any()
        else np.nan
    )

    male_recall = (
        float(np.mean(pred[male_mask] == 1))
        if male_mask.any()
        else np.nan
    )

    recalls = [
        value
        for value in (female_recall, male_recall)
        if np.isfinite(value)
    ]

    age_rows.append({
        "fairface_age_range": age_range,
        "images": int(mask.sum()),
        "female_images": int(female_mask.sum()),
        "male_images": int(male_mask.sum()),
        "accuracy": float(accuracy_score(truth, pred)),
        "female_recall": female_recall,
        "male_recall": male_recall,
        "balanced_accuracy": float(np.mean(recalls)),
    })

age_results = pd.DataFrame(age_rows)

display(age_results)

age_results.to_csv(
    OUTPUT_DIR / "gender_results_by_age.csv",
    index=False,
)

class_summary = pd.DataFrame({
    "class": CLASS_NAMES,
    "holdout_images": [
        int((y_true == class_id).sum())
        for class_id in range(N_CLASSES)
    ],
    "correct_predictions": [
        int(((y_true == class_id) & (y_pred == class_id)).sum())
        for class_id in range(N_CLASSES)
    ],
})

```

```

        "class_recall": [
            float((y_pred[y_true == class_id] == class_id).mean())
            for class_id in range(N_CLASSES)
        ],
    })

display(class_summary)

class_summary.to_csv(
    OUTPUT_DIR / "female_male_class_recall.csv",
    index=False,
)

fairface_age_range  images  female_images  male_images  accuracy  female_recall  male_recall
balanced_accuracy
0          0-2      199          76          123  0.738693      0.605263      0.821138
0.713201
1          3-9     1351         613          738  0.800148      0.853181      0.756098
0.804639
2         10-19    1180         632          548  0.819492      0.873418      0.757299
0.815358
3         20-29    3299        1831         1468  0.920279      0.943200      0.891689
0.917445
4         30-39    2329        1018         1311  0.940318      0.936149      0.943555
0.939852
5         40-49    1352         517          835  0.933432      0.901354      0.953293
0.927324
6         50-59     796         300          496  0.907035      0.860000      0.935484
0.897742
7         60-69     321         114          207  0.894081      0.850877      0.917874
0.884376
8        more than 70  118         58          60  0.864407      0.793103      0.933333
0.863218

class  holdout_images  correct_predictions  class_recall
0  Female           5159             4668         0.904827
1   Male           5786             5126         0.885932

```

26. Look at the mistakes

In this cell we look at some Female and Male faces that the model classified incorrectly. This helps us understand the model's errors.

Code cell 52 - recorded

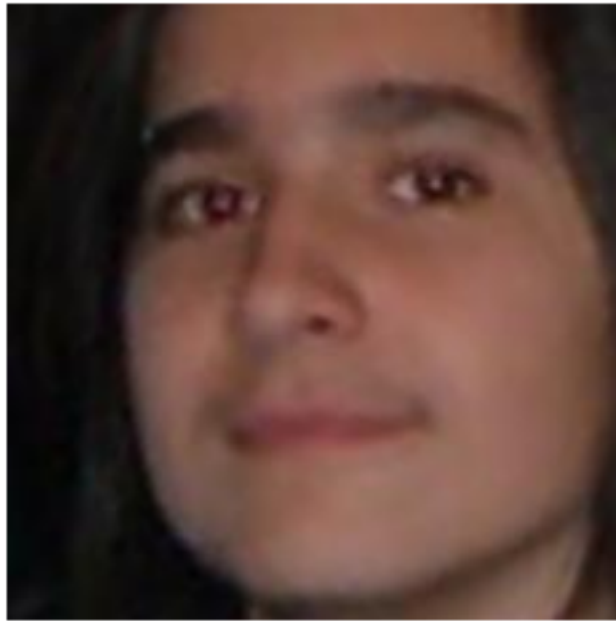
```

# cell:errors
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
mistakes = predictions.loc[~predictions["correct"]].sort_values("model_score", ascending=False)
display(mistakes[["true_class", "predicted_class", "model_score"]].head(10))
mistakes.to_csv(OUTPUT_DIR / "mistakes.csv", index=False)
if SHOW_FACE_IMAGES:
    for _, row in mistakes.head(3).iterrows():
        fig, ax = plt.subplots(figsize=(3, 3))
        ax.imshow(open_face(row["path"], row[BOX_COLUMNS]))
        ax.set_title(f"Label: {row['true_class']}\nPrediction: {row['predicted_class']}")
        ax.axis("off")
        plt.show()

true_class  predicted_class  model_score
94580      Female          Male      0.999909
89228      Female          Male      0.999675
97503      Male            Female     0.999600
95710      Female          Male      0.999556
94336      Female          Male      0.999410
90812      Male            Female     0.998942
93834      Female          Male      0.998384
91551      Female          Male      0.998276
97204      Male            Female     0.998181
94366      Female          Male      0.998119

```

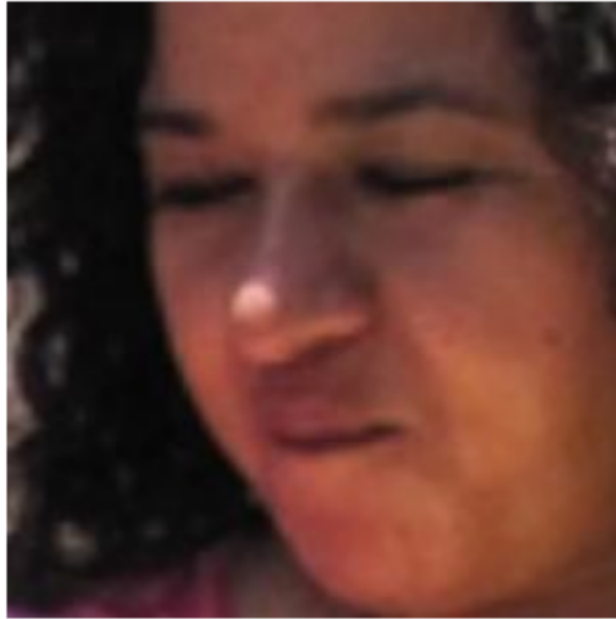
Label: Female
Prediction: Male



Label: Female
Prediction: Male



Label: Male
Prediction: Female



27. Export a full-precision Android model

In this cell we create a separate float32 CPU copy of the chosen model and convert it to TensorFlow Lite. This gives the conversion check a fair reference.

Code cell 54 - recorded

```
# cell:fp32
OUTPUT_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

# Training uses mixed precision on the GPU.
# For deployment checking we rebuild the same architecture in float32 on CPU.
training_policy = tf.keras.mixed_precision.global_policy()

tf.keras.mixed_precision.set_global_policy(
    "float32"
)

try:
    with tf.device("/CPU:0"):
        cpu_reference, _ = build_gender_model(
            weights=None
        )

        cpu_reference.set_weights(
            model.get_weights()
        )

        cpu_reference.trainable = False

        cpu_reference(
            tf.zeros(
                (
                    1,
                    IMAGE_SIZE,
                    IMAGE_SIZE,
                    3,
```

```

        ),
        dtype=tf.float32,
    ),
    training=False,
)

converter = tf.lite.TFLiteConverter.from_keras_model(
    cpu_reference
)

converter.target_spec.supported_ops = [
    tf.lite.OpsSet.TFLITE_BUILTINS
]

fp32_bytes = converter.convert()

finally:
    tf.keras.mixed_precision.set_global_policy(
        training_policy
    )

FP32_PATH = (
    OUTPUT_DIR
    / f"{TASK}_fp32.tflite"
)

FP32_PATH.write_bytes(
    fp32_bytes
)

print("Saved:", FP32_PATH)

print(
    "Size in MB:",
    round(
        FP32_PATH.stat().st_size
        / 1e6,
        2,
    ),
)

Saved artifact at '/tmp/tmpm11xhjzl'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32,
name='face_rgb')
Output Type:
  TensorSpec(shape=(None, 2), dtype=tf.float32, name=None)
Captures:
139794450174736: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450183952: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450182608: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450181264: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450173584: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450179536: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450183376: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450178384: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450179344: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450184528: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450184336: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450179728: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450180688: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450174544: TensorSpec(shape=(), dtype=tf.resource, name=None)

139794450176848: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450183568: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450178960: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450182416: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450183760: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450175696: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450176080: TensorSpec(shape=(), dtype=tf.resource, name=None)

```



```

139794469520592: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469520400: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469520784: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469519632: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469520016: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469521552: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469521360: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469521744: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469519440: TensorSpec(shape=(), dtype=tf.resource, name=None)

139794469520976: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469522512: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469522320: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469522704: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469520208: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469521936: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469523664: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469523472: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469524048: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469523088: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469524624: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469524432: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469524816: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469522896: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469523280: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469525584: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469525392: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469525776: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469523856: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469525008: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469526544: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469526352: TensorSpec(shape=(), dtype=tf.resource, name=None)

139794469526736: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469524240: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469525968: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469527696: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469527504: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469528080: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469527888: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469528272: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469527312: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469528464: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469525200: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469526928: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469529232: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469529040: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469529424: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469526160: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469529616: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469530576: TensorSpec(shape=(), dtype=tf.resource, name=None)
Saved: /content/project_edge_ai/results/gender/20260915_030921/gender_fp32.tflite
Size in MB: 3.73

```

28. Try a smaller Android model

In this cell we also try full integer quantisation. The calibration images come only from the training set.

Code cell 56 - recorded

```

# cell:int8
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
calibration_df = pd.concat([group.sample(min(200, len(group)), random_state=SEED)
                           for _, group in train_df.groupby("label")])
calibration_df.to_csv(OUTPUT_DIR / "calibration_images.csv", index=False)
assert set(calibration_df["path"]).issubset(set(train_df["path"]))

def representative_images():
    for images, _ in make_dataset(calibration_df):
        for image in images.numpy():
            yield [image[None].astype(np.float32)]

```

```

INT8_PATH = None
try:
    with tf.device("/CPU:0"):
        converter = tf.lite.TFLiteConverter.from_keras_model(cpu_reference)
        converter.optimizations = [tf.lite.Optimize.DEFAULT]
        converter.representative_dataset = representative_images
        converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
        converter.inference_input_type = tf.int8
        converter.inference_output_type = tf.int8
        converted = converter.convert()

        path = OUTPUT_DIR / f"{TASK}_int8.tflite"
        path.write_bytes(converted)
        INT8_PATH = path
        print("Saved:", INT8_PATH)
except Exception as error:
    print("INT8 conversion failed. The FP32 model is still available.")
    print(type(error).__name__+": "+str(error))
    (OUTPUT_DIR / "int8_error.txt").write_text(str(error), encoding="utf-8")

```

Saved artifact at '/tmp/tmp6ebitonz'. The following endpoints are available:

```

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32,
name='face_rgb')
Output Type:
  TensorSpec(shape=(None, 2), dtype=tf.float32, name=None)
Captures:
139794450174736: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450183952: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450182608: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450181264: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450173584: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450179536: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450183376: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450178384: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450179344: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450184528: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450184336: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450179728: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450180688: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450174544: TensorSpec(shape=(), dtype=tf.resource, name=None)

139794450176848: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450183568: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450178960: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450182416: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450183760: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450175696: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450176080: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450185488: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450175312: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794450182992: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462831248: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462823376: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462830672: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462826064: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462821072: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462819152: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462824144: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462828176: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462831824: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462820688: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462827408: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462832208: TensorSpec(shape=(), dtype=tf.resource, name=None)

139794462834512: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462831632: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462819344: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462829904: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462830480: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462829712: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794462830288: TensorSpec(shape=(), dtype=tf.resource, name=None)

```



```

139794469522896: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469523280: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469525584: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469525392: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469525776: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469523856: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469525008: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469526544: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469526352: TensorSpec(shape=(), dtype=tf.resource, name=None)

139794469526736: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469524240: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469525968: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469527696: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469527504: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469528080: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469527888: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469528272: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469527312: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469528464: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469525200: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469526928: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469529232: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469529040: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469529424: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469526160: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469529616: TensorSpec(shape=(), dtype=tf.resource, name=None)
139794469530576: TensorSpec(shape=(), dtype=tf.resource, name=None)

```

```

/usr/local/lib/python3.13/dist-packages/tensorflow/lite/python/convert.py:863: UserWarning:
Statistics for quantized inputs were expected, but not specified; continuing anyway.
warnings.warn(

```

Saved: /content/project_edge_ai/results/gender/20260915_030921/gender_int8.tflite

29. Check the converted models

In this cell we compare TensorFlow Lite against a separate float32 Keras model on the CPU. We check score differences, prediction agreement, Female recall, Male recall and balanced accuracy before choosing a phone model.

Code cell 58 - recorded

```

# cell:mobile_check
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

def balanced_check_sample(frame, per_class, seed):
    return pd.concat([g.sample(min(per_class, len(g)), random_state=seed)
                      for _, g in frame.groupby("label")]).reset_index(drop=True)

def probability_check(values, rows, tolerance=1e-4):
    values = np.asarray(values, dtype=np.float32)
    if values.shape != (rows, 2) or not np.isfinite(values).all():
        raise ValueError("Wrong prediction shape or non-finite model scores.")
    if (values < -tolerance).any() or (values > 1+tolerance).any():
        raise ValueError("Model scores are outside the probability range.")
    if not np.allclose(values.sum(axis=1), 1, rtol=0, atol=tolerance):
        raise ValueError("Class probabilities do not sum to one. Check output scaling.")
    return values

def class_recalls(truth, prediction):
    truth, prediction = np.asarray(truth), np.asarray(prediction)
    if set(np.unique(truth)) != {0, 1}:
        raise ValueError("The check sample must contain both gender classes.")
    return np.array([np.mean(prediction[truth == i] == i) for i in (0, 1)])

def conversion_report(truth, reference, candidate, limits, quantized=False):

```

```

truth = np.asarray(truth)
reference = probability_check(reference, len(truth))
candidate = probability_check(candidate, len(truth), tolerance=0.02 if quantized else 1e-4)
difference = np.abs(reference-candidate)
ref_labels, labels = reference.argmax(axis=1), candidate.argmax(axis=1)
ref_recall, recall = class_recalls(truth, ref_labels), class_recalls(truth, labels)
result = {
    "max_abs_difference": float(difference.max()),
    "mean_abs_difference": float(difference.mean()),
    "p99_abs_difference": float(np.quantile(difference, 0.99)),
    "prediction_agreement": float(np.mean(ref_labels == labels)),
    "accuracy": float(np.mean(truth == labels)),
    "recall_female": float(recall[0]), "recall_male": float(recall[1]),
    "balanced_accuracy": float(recall.mean()),
    "balanced_accuracy_drop": float(ref_recall.mean()-recall.mean()),
    "largest_class_recall_drop": float(np.max(ref_recall-recall)),
}

reasons = []
for key, limit_key in [("max_abs_difference", "max_abs"), ("mean_abs_difference", "mean_abs"),
    ("balanced_accuracy_drop", "max_balanced_accuracy_drop"),
    ("largest_class_recall_drop", "max_class_recall_drop")]:
    if result[key] > limits[limit_key]+1e-12:
        reasons.append(key+" exceeds the set limit")
    if result["prediction_agreement"]+1e-12 < limits["min_agreement"]:
        reasons.append("prediction agreement is below the set limit")
result["passed"] = not reasons
result["reason"] = "; ".join(reasons) or "All checks passed"
return result

def encode_input(value, detail):
    value = np.asarray(value, dtype=np.float32)
    if not np.isfinite(value).all():
        raise ValueError("Input contains invalid pixels.")
    if np.issubdtype(detail["dtype"], np.integer):
        scale, zero = detail["quantization"]
        if not np.isfinite(scale) or scale <= 0:
            raise ValueError("Invalid input quantization scale.")
        bounds = np.iinfo(detail["dtype"])

        value = np.clip(np rint(value/scale+zero), bounds.min, bounds.max)
    return value.astype(detail["dtype"])

def decode_output(value, detail):
    value = np.asarray(value)
    if np.issubdtype(detail["dtype"], np.integer):
        scale, zero = detail["quantization"]
        if not np.isfinite(scale) or scale <= 0:
            raise ValueError("Invalid output quantization scale.")
        value = (value.astype(np.float32)-zero)*scale
    return value.astype(np.float32)

def open_lite(path):
    interpreter = tf.lite.Interpreter(model_path=str(path), num_threads=4)
    interpreter.allocate_tensors()
    inputs, outputs = interpreter.get_input_details(), interpreter.get_output_details()
    if len(inputs) != 1 or len(outputs) != 1:
        raise ValueError("Expected one image input and one two-class output.")
    if list(inputs[0]["shape"]) != [1, IMAGE_SIZE, IMAGE_SIZE, 3] or list(outputs[0]["shape"]) !=
[1, 2]:
        raise ValueError("Unexpected TensorFlow Lite input/output shape.")
    return interpreter, inputs[0], outputs[0]

def lite_predictions(path, frame):
    interpreter, input_info, output_info = open_lite(path)
    values = []
    for images, _ in make_dataset(frame):

```

```

        for image in images.numpy():
            interpreter.set_tensor(input_info["index"], encode_input(image[None], input_info))
            interpreter.invoke()
            values.append(decode_output(interpreter.get_tensor(output_info["index"])[0],
output_info))
        return np.asarray(values, dtype=np.float32)

def cpu_predictions(frame):
    values = []
    for images, _ in make_dataset(frame):
        with tf.device("/CPU:0"):
            values.append(cpu_reference(images, training=False).numpy())
    return np.concatenate(values)

validation_sample = balanced_check_sample(val_df, MOBILE_IMAGES_PER_CLASS, SEED)
validation_sample.to_csv(OUTPUT_DIR / "conversion_validation_images.csv", index=False)
truth = validation_sample["label"].to_numpy()
keras_scores = cpu_predictions(validation_sample)
fp32_scores = lite_predictions(FP32_PATH, validation_sample)
fp32_report = conversion_report(truth, keras_scores, fp32_scores, FP32_LIMITS)
# This GPU/CPU comparison is diagnostic. It is not the conversion test reference.
gpu_scores = model.predict(make_dataset(validation_sample), verbose=0)
print("Largest GPU-Keras / CPU-Keras difference:", float(np.max(np.abs(gpu_scores-keras_scores))))
print("Largest CPU-Keras / FP32-Lite difference:", fp32_report["max_abs_difference"])
print("FP32 class-prediction agreement:", f"{fp32_report['prediction_agreement']:.2%}")
if fp32_report["max_abs_difference"] > 0.001 and fp32_report["passed"]:
    print("Small numerical drift recorded. Agreement and both recall checks passed.")

format_rows = [dict(format="FP32", size_MB=FP32_PATH.stat().st_size/1e6, **fp32_report)]
mobile_models = {"FP32": FP32_PATH}
CHOSEN_FORMAT = "FP32"
if INT8_PATH is not None:
    try:
        int8_scores = lite_predictions(INT8_PATH, validation_sample)

        int8_report = conversion_report(truth, fp32_scores, int8_scores, INT8_LIMITS,
quantized=True)
        format_rows.append(dict(format="INT8", size_MB=INT8_PATH.stat().st_size/1e6,
**int8_report))
        if fp32_report["passed"] and int8_report["passed"]:
            mobile_models["INT8"] = INT8_PATH
            if INT8_PATH.stat().st_size < FP32_PATH.stat().st_size:
                CHOSEN_FORMAT = "INT8"
    else:
        print("INT8 is not selected:", int8_report["reason"])
except Exception as error:
    print("INT8 checking failed; keeping FP32 as the candidate:", error)
    (OUTPUT_DIR / "int8_validation_error.txt").write_text(str(error), encoding="utf-8")

format_table = pd.DataFrame(format_rows).set_index("format")
display(format_table)
format_table.to_csv(OUTPUT_DIR / "format_validation.csv")
(OUTPUT_DIR / "conversion_limits.json").write_text(json.dumps({
    "FP32": FP32_LIMITS, "INT8": INT8_LIMITS,
    "sample_images_per_class": MOBILE_IMAGES_PER_CLASS,
    "fp32_reference": "Separate float32 Keras model on CPU, identical saved weights and
preprocessing",
}, indent=2), encoding="utf-8")
diagnostic = validation_sample[["path", "label"]].copy()
for i, name in enumerate(CLASS_NAMES):
    diagnostic["keras_cpu_"+name] = keras_scores[:, i]
    diagnostic["fp32_"+name] = fp32_scores[:, i]
diagnostic.to_csv(OUTPUT_DIR / "fp32_check_predictions.csv", index=False)
if not fp32_report["passed"]:
    raise RuntimeError("FP32 conversion failed the checks. See format_validation.csv:
"+fp32_report["reason"])
print("Candidate Android format:", CHOSEN_FORMAT)

/usr/local/lib/python3.13/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning:
Warning: tf.lite.Interpreter is deprecated and is scheduled for deletion in
TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.

```

See the [migration guide](https://ai.google.dev/edge/litert/migration) for details.

```
warnings.warn(_INTERPRETER_DELETION_WARNING)
```

```
Largest GPU-Keras / CPU-Keras difference: 0.04228395223617554
Largest CPU-Keras / FP32-Lite difference: 1.7344951629638672e-05
FP32 class-prediction agreement: 100.00%
```

```
/usr/local/lib/python3.13/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning:
  Warning: tf.lite.Interpreter is deprecated and is scheduled for deletion in
  TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
  See the [migration guide](https://ai.google.dev/edge/litert/migration)
  for details.
```

```
warnings.warn(_INTERPRETER_DELETION_WARNING)
```

```
INT8 checking failed; keeping FP32 as the candidate: failed to create XNNPACK runtimeNode number
117 (TfLiteXNNPackDelegate) failed to prepare.
```

	size_MB	max_abs_difference	mean_abs_difference	p99_abs_difference	prediction_agreement	accuracy	...	\	format
	...								
FP32	3.731884	0.000017	0.000001	0.00001					
1.0	0.890625	...							
	recall_male	balanced_accuracy	balanced_accuracy_drop	largest_class_recall_drop	passed				
	reason								
	format								
FP32	0.855469	0.890625	0.0	0.0	True				
	All checks passed								

```
[1 rows x 13 columns]
```

```
Candidate Android format: FP32
```

30. Compare models on the same holdout sample

In this cell we compare Keras, FP32 TensorFlow Lite and the accepted INT8 model on the same balanced sample. Computer CPU timing is recorded, but it is not presented as phone timing.

Code cell 60 - recorded

```
# cell:comparison
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

def computer_latency(path, sample):
    interpreter, details, _ = open_lite(path)
    interpreter.set_tensor(details["index"], encode_input(sample[None], details))
    for _ in range(10):
        interpreter.invoke()
    times = []
    for _ in range(50):
        started = time.perf_counter()
        interpreter.invoke()
        times.append((time.perf_counter()-started)*1000)
    return float(np.median(times)), float(np.percentile(times, 95))

comparison_df = balanced_check_sample(test_df, MOBILE_IMAGES_PER_CLASS, SEED)
comparison_df.to_csv(OUTPUT_DIR / "mobile_comparison_images.csv", index=False)
comparison_labels = comparison_df["label"].to_numpy()
keras_comparison = cpu_predictions(comparison_df)

def comparison_row(name, values, size, median=np.nan, p95=np.nan):
    predicted = values.argmax(axis=1)
    recall = class_recalls(comparison_labels, predicted)
    return {"format": name, "images": len(comparison_df),
            "accuracy": float(accuracy_score(comparison_labels, predicted)),
            "balanced_accuracy": float(recall.mean()), "recall_female": float(recall[0]),
```

```

        "recall_male": float(recall[1]),
        "macro_f1": float(f1_score(comparison_labels, predicted, average="macro",
zero_division=0)),
        "size_MB": size, "computer_CPU_median_ms": median, "computer_CPU_p95_ms": p95}

comparison_rows = [comparison_row("Keras CPU reference", keras_comparison,
                                (OUTPUT_DIR/"best_model.h5").stat().st_size/1e6)]
example, _ = next(iter(make_dataset(validation_sample.head(1))))
for name, path in mobile_models.items():
    values = lite_predictions(path, comparison_df)
    median_ms, p95_ms = computer_latency(path, example[0].numpy())
    comparison_rows.append(comparison_row(name, values, path.stat().st_size/1e6, median_ms,
p95_ms))
comparison = pd.DataFrame(comparison_rows)
display(comparison)
comparison.to_csv(OUTPUT_DIR / "model_comparison.csv", index=False)
print("CPU timing: 4 threads, 10 warm-up calls, 50 measured calls. Image loading is excluded.")
print("These are NOT phone timings. Holdout results did not choose the exported format.")

/usr/local/lib/python3.13/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning:
    Warning: tf.lite.Interpreter is deprecated and is scheduled for deletion in
    TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
    See the [migration guide](https://ai.google.dev/edge/litert/migration)
    for details.

warnings.warn(_INTERPRETER_DELETION_WARNING)
/usr/local/lib/python3.13/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning:
    Warning: tf.lite.Interpreter is deprecated and is scheduled for deletion in
    TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
    See the [migration guide](https://ai.google.dev/edge/litert/migration)
    for details.

warnings.warn(_INTERPRETER_DELETION_WARNING)

      format  images  accuracy  balanced_accuracy  recall_female  recall_male  macro_f1
size_MB  \
0  Keras CPU reference      512   0.882812          0.882812        0.902344        0.863281   0.882768
4.077616
1          FP32      512   0.882812          0.882812        0.902344        0.863281   0.882768
3.731884

      computer_CPU_median_ms  computer_CPU_p95_ms
0                NaN                NaN
1          12.348333          13.941746

CPU timing: 4 threads, 10 warm-up calls, 50 measured calls. Image loading is excluded.
These are NOT phone timings. Holdout results did not choose the exported format.

```

31. Save the Android model and its input rules

In this cell we save the chosen gender model, the Female/Male label order and the input rules. Android must use the same RGB face crop and image size.

Code cell 62 - recorded

```

# cell:save
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
CHOSEN_PATH = OUTPUT_DIR / f"{TASK}_android.tflite"
shutil.copyfile(mobile_models[CHOSEN_FORMAT], CHOSEN_PATH)
(OUTPUT_DIR / "labels.txt").write_text("\n".join(CLASS_NAMES) + "\n", encoding="utf-8")
interpreter = tf.lite.Interpreter(model_path=str(CHOSEN_PATH))
interpreter.allocate_tensors()

def tensor_rules(details):
    return {"shape": details["shape"].tolist(), "dtype": np.dtype(details["dtype"]).name,
            "scale": float(details["quantization"][0]), "zero_point":
int(details["quantization"][1])}

settings = {
    "course": "DLBAIPEAI", "task": "Task 2", "model_task": TASK,
    "architecture": "MobileNetV3Small",

```

```

"seed": SEED, "class_names": CLASS_NAMES,
"notebook_revision": NOTEBOOK_REVISION,
"training_sampling": "exact half Female / half Male; shuffled cycles; training augmentation
only",
"training_class_weights": None, "samples_per_epoch": STEPS_PER_EPOCH*BATCH_SIZE,
"checkpoint_rule": "highest validation balanced accuracy; validation loss breaks stage ties",

"decision_rule": "argmax of Female/Male scores; no holdout-tuned threshold",
"probability_note": "Balanced training changes the class prior; scores are model confidence
values, not identity claims.",
"gender_label_note": "The model predicts the two appearance labels supplied by FairFace.",
"conversion_fp32_limits": FP32_LIMITS, "conversion_int8_limits": INT8_LIMITS,
"dataset_handle": KAGGLE_HANDLE, "dataset_directory": str(DATA_DIR),
"input": tensor_rules(interpreter.get_input_details()[0]),
"output": tensor_rules(interpreter.get_output_details()[0]),
"image_rule": "Crop face, convert to RGB, resize to 224x224 using bilinear interpolation.",
"pixel_rule": "Float pixels 0-255. Scaling is already inside the model. Do not divide by
255.",
"integer_rule": "For integer tensors use the saved scale and zero point, with rounding and
clipping.",
"includes_face_detector": False, "measured_on_phone": False,
"split_sizes": {name: len(frame) for name, frame in frames.items()},
"chosen_stage": BEST_STAGE, "chosen_format": CHOSEN_FORMAT,
"training_seconds": head_seconds + fine_seconds,
"gpu_name": GPU_NAME, "gpu_forward_backward_test_passed": GPU_READY,
"holdout_scores": model_scores,
"versions": {
    "tensorflow": tf.__version__,
    **{name: metadata.version(name) for name in packages},
},
"system": platform.platform(), "tensorflow_devices": [str(device) for device in gpus],
}
(OUTPUT_DIR / "model_settings.json").write_text(json.dumps(settings, indent=2), encoding="utf-8")
print("Android model:", CHOSEN_PATH)
print("The training input starts with a cropped face. The app must do this crop too.")

/usr/local/lib/python3.13/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning:
  Warning: tf.lite.Interpreter is deprecated and is scheduled for deletion in
  TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
  See the [migration guide](https://ai.google.dev/edge/litert/migration)
  for details.

warnings.warn(_INTERPRETER_DELETION_WARNING)

Android model: /content/project_edge_ai/results/gender/20260915_030921/gender_android.tflite
The training input starts with a cropped face. The app must do this crop too.

```

32. Summarise this run

In this cell we summarise the measured Female/Male results. The notebook does not create results before training. The Android app and the required 20-image device experiment still need to be completed.

Code cell 64 - recorded

```

# cell:summary
display(Markdown(
    f"### Measured results\n\n"
    f"The gender model predicts the two appearance labels supplied by FairFace: "
    f"***Female** and ***Male**.\n\n"
    f"The training set contained **{len(train_df):,} different image files**."
    f"Each epoch used **{STEPS_PER_EPOCH*BATCH_SIZE:,} image presentations**,"
    f"half from each class."
    f"Repeats are not counted as new people."
    f"The validation set had **{len(val_df):,} images**."
    f"The final holdout had **{len(test_df):,} images**.\n\n"
    f"Holdout accuracy was **{model_scores['accuracy']:.2%**."
    f"Balanced accuracy was **{model_scores['balanced_accuracy']:.2%**."
    f"Macro F1 was **{model_scores['macro_f1']:.3f%**."
    f"The selected checkpoint was **{BEST_STAGE}**, and the candidate phone format was
    **{CHOSEN_FORMAT}**.\n\n"
    f"The age-group table above should also be reviewed because it shows whether "

```

```
f"Female or Male recall changes across age ranges. "  
f"These computer tests do not replace the final on-phone test."  
)
```

Measured results

The gender model predicts the two appearance labels supplied by FairFace: **Female** and **Male**.

The training set contained **73,555 different image files**. Each epoch used **78,016 image presentations**, half from each class. Repeats are not counted as new people. The validation set had **12,981 images**. The final holdout had **10,945 images**.

Holdout accuracy was **89.48%**. Balanced accuracy was **89.54%**. Macro F1 was **0.895**. The selected checkpoint was **fine**, and the candidate phone format was **FP32**.

The age-group table above should also be reviewed because it shows whether Female or Male recall changes across age ranges. These computer tests do not replace the final on-phone test.

What remains for Task 2

The final phone app must recognise age group, gender and expression on the device. For this model, the app will return **Female** or **Male**, following the two labels supplied by FairFace.

The final experiment must still use the 20 device-captured images required by the task brief. Those images must not be used for training or model selection. Real phone measurements and the final test results must be included in the report.

The source code, critical review of the results and AI-use statement still belong in the course submission.

Sources

IU International University. (n.d.). DLBAIPEAI - Project Edge AI: Task - Project report (Task 2, pp. 3-4). Course document supplied with this project.

Karkkainen, K., & Joo, J. (2021). FairFace: Face attribute dataset for balanced race, gender, and age for bias measurement and mitigation. Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, 1548-1558. <https://github.com/joojs/fairface>

Kaggle. (n.d.). *FAIRFACE DATASET*. Kaggle mirror used for the notebook download. <https://www.kaggle.com/datasets/mehmoodsheikh/fairface-dataset>

Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q. V., & Adam, H. (2019). Searching for MobileNetV3. arXiv. <https://arxiv.org/abs/1905.02244>

TensorFlow. (n.d.-a). *Install TensorFlow with pip*. <https://www.tensorflow.org/install/pip>

TensorFlow. (n.d.-b). *MobileNetV3Small*. https://www.tensorflow.org/api_docs/python/tf/keras/applications/MobileNetV3Small

TensorFlow. (n.d.-c). *Post-training integer quantization*. https://www.tensorflow.org/lite/performance/post_training_integer_quant

TensorFlow. (n.d.-d). *Classification on imbalanced data*. https://www.tensorflow.org/tutorials/structured_data/imbalanced_data

All result tables and graphs in this notebook are produced from the loaded data when the cells run. Face examples remain subject to their dataset terms.

Label note: The model predicts the two appearance labels supplied by FairFace. It should not be described as determining a person's gender identity.

33. Make the notebook PDF

In this cell we automatically create a PDF of the saved notes, code and recorded outputs. Save any changes to the notes first. Training is not run again. No LaTeX, browser installer or helper file is needed.

Code cell 68 - recorded

```
# cell:pdf
OUTPUT_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

import base64
import textwrap
import copy
import re
import io

from html import escape

import nbformat

from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import (
    getSampleStyleSheet,
    ParagraphStyle,
)
from reportlab.platypus import (
    SimpleDocTemplate,
    Paragraph,
    Preformatted,
    Spacer,
    Image as PDFImage,
)

# Get the current notebook directly from Google Colab.
try:
    from google.colab import _message

    response = _message.blocking_request(
        "get_ipynb",
        timeout_sec=30,
    )

    notebook_data = response.get(
        "ipynb"
    )

    if notebook_data is None:
        raise RuntimeError(
            "Colab did not return the current notebook."
        )

    notebook = nbformat.from_dict(
        notebook_data
    )
except Exception as error:
    raise RuntimeError(
        "Could not read the current Colab notebook."
    ) from error

# Colab can return cell source as either a string
# or a list of strings. Make every source one string.
for cell in notebook.cells:

    source = cell.get(
        "source",
```



```

        "",
    )
    if isinstance(
        source,
        list,
    ):
        source = "".join(
            source
        )

    elif not isinstance(
        source,
        str,
    ):
        source = str(
            source
        )

    cell["source"] = source

    if cell.cell_type == "code":

        key = (
            source.splitlines()[0].strip()
            if source
            else ""
        )

        cell.outputs = []
        cell.execution_count = None

        if key in pdf_recorder.records:

            record = copy.deepcopy(
                pdf_recorder.records[key]
            )

            cell.outputs = nbformat.from_dict(
                {
                    "outputs": record.get(
                        "outputs",
                        [],
                    )
                }
            ).outputs

            cell.execution_count = record.get(
                "execution_count"
            )

# PDF styles.
styles = getSampleStyleSheet()

styles.add(
    ParagraphStyle(
        "Notes",
        fontName="Helvetica",
        fontSize=10,
        leading=14,
        spaceAfter=8,
    )
)

styles.add(
    ParagraphStyle(
        "NotebookCode",
        fontName="Courier",
        fontSize=8,
        leading=10.5,
    )
)

```

```

        spaceAfter=7,
    )
)

styles["Heading1"].fontSize = 18
styles["Heading1"].leading = 22

styles["Heading2"].fontSize = 12
styles["Heading2"].leading = 16

story = []

def plain(value):

    if isinstance(
        value,
        list,
    ):

        text = "".join(
            value
        )

    else:

        text = str(
            value
        )

    # Remove terminal colour codes.
    text = re.sub(
        r"\x1b\[([0-?]*[ -/]*[@-~])",
        "",
        text,
    )

    replacements = {
        "-": "-",
        "_": "_",
        "->": "->",
        '"': '"',
        "'": "'",
        "`": "`",
    }

    for old, new in replacements.items():
        text = text.replace(
            old,
            new,
        )

    return (
        text
        .encode(
            "latin-1",
            errors="replace",
        )
        .decode(
            "latin-1"
        )
    )

def inline(text):

    text = escape(
        plain(text)
    )

    text = re.sub(
        r"\([([^\]]+)\)\((https?:\/\/[^\s]+)\)",

```

```

        r'<link href="\2">\1</link>',
        text,
    )

    text = re.sub(
        r"\*(.+?)\*",
        r"<b>\1</b>",
        text,
    )

    text = re.sub(
        r"`([^\`]+)`",
        r'<font name="Courier">\1</font>',
        text,
    )

    return text

def add_code(text):

    lines = []

    for line in plain(
        text
    ).splitlines():

        wrapped = textwrap.wrap(
            line.expandtabs(4),
            98,
            replace_whitespace=False,
            drop_whitespace=False,
            break_on_hyphens=False,
        )

        lines.extend(
            wrapped or [""]
        )

    for start in range(
        0,
        len(lines),
        22,
    ):

        story.append(
            Preformatted(
                "\n".join(
                    lines[
                        start:start + 22
                    ]
                ),
                styles["NotebookCode"],
            )
        )

def add_notes(text):

    for paragraph in plain(
        text
    ).split(
        "\n\n"
    ):

        paragraph = paragraph.strip()

        if not paragraph:
            continue

        heading = re.match(

```

```

        r"^(#{1,3})\s+(.*)$",
        paragraph,
        re.S,
    )

    if heading:

        style = styles[
            "Heading1"
            if len(
                heading[1]
            ) == 1
            else "Heading2"
        ]

        story.append(
            Paragraph(
                inline(
                    heading[2]
                ),
                style,
            )
        )

    else:

        story.append(
            Paragraph(
                inline(
                    paragraph
                ).replace(
                    "\n",
                    "<br/>",
                ),
                styles["Notes"],
            )
        )

# Add every notebook cell to the PDF.
for number, cell in enumerate(
    notebook.cells,
    1,
):

    if cell.cell_type == "markdown":

        add_notes(
            cell.source
        )

    elif cell.cell_type == "code":

        status = (
            "recorded"
            if cell.execution_count is not None
            else "output not recorded"
        )

        story.append(
            Paragraph(
                f"Code cell {number} - {status}",
                styles["Heading3"],
            )
        )

        add_code(
            cell.source
        )

```

```

for output in cell.outputs:

    if output.get(
        "output_type"
    ) == "stream":

        add_code(
            output.get(
                "text",
                "",
            )
        )

    else:

        data = output.get(
            "data",
            {},
        )

        if "image/png" in data:

            encoded = data[
                "image/png"
            ]

            if isinstance(
                encoded,
                list,
            ):
                encoded = "".join(
                    encoded
                )

            image = PDFImage(
                io.BytesIO(
                    base64.b64decode(
                        encoded
                    )
                )
            )

            scale = min(
                500 / image.imageWidth,
                330 / image.imageHeight,
                1.0,
            )

            image.drawWidth *= scale
            image.drawHeight *= scale

            story.append(
                image
            )

            story.append(
                Spacer(
                    1,
                    8,
                )
            )

        elif "text/markdown" in data:

            add_notes(
                data[
                    "text/markdown"
                ]
            )

        elif "text/plain" in data:

```

```

        add_code(
            data[
                "text/plain"
            ]
        )

# Save the PDF.
PDF_PATH = (
    OUTPUT_DIR
    / (
        NOTEBOOK_NAME
        + ".pdf"
    )
)

def page_number(
    canvas,
    document,
):
    canvas.setFont(
        "Helvetica",
        8,
    )

    canvas.drawString(
        40,
        22,
        "DLBAIPEAI - Task 2 - "
        + TASK.title()
        + " model",
    )

    canvas.drawRightString(
        A4[0] - 40,
        22,
        f"Page {document.page}",
    )

SimpleDocTemplate(
    str(PDF_PATH),

    pagesize=A4,
    leftMargin=40,
    rightMargin=40,
    topMargin=38,
    bottomMargin=38,
    title=NOTEBOOK_NAME,
    author="W. Pretorius",
).build(
    story,
    onFirstPage=page_number,
    onLaterPages=page_number,
)

# Save an executed notebook copy too.
EXECUTED_PATH = (
    OUTPUT_DIR
    / (
        NOTEBOOK_NAME
        + "_executed.ipynb"
    )
)

nbformat.write(
    notebook,
    EXECUTED_PATH,
)

```

```

print(
    "Notebook PDF saved:",
    PDF_PATH,
)

print(
    "Executed notebook saved:",
    EXECUTED_PATH,
)

```

Download the Colab results

In this cell we put the trained model, tables, graphs and PDF into one ZIP file and download it to the computer.

Code cell 70 - output not recorded

```

# cell:colab_download
import shutil
from google.colab import files as colab_files

archive_base = PROJECT_DIR / f"gender_results_{OUTPUT_DIR.name}"

archive_path = shutil.make_archive(
    str(archive_base),
    "zip",
    root_dir=OUTPUT_DIR,
)

print("Created:", archive_path)
colab_files.download(archive_path)

```